RUBY

CHEAT SHEETS

A Complete Reference for Modern Ruby

Strings & Symbols · Pattern Matching · Blocks & Lambdas · Arrays & Hashes · Classes & Mixins · Exceptions · Files & JSON · Gems & Bundler · Testing · Concurrency & Ractors · Enumerable · Metaprogramming

115 REFERENCE SHEETS

CONTENTS

FOUNDATIONS

001–004 Introduction & Setup
005–007 Output & Comments
008–015 Variables & Data Types
016–021 Strings
022–027 Operators
028–035 Control Flow

CORE LANGUAGE

036–039 Error Handling
040–049 Methods & Blocks
050–060 Data Structures
061–072 OOP
073–078 Files & I/O

ECOSYSTEM & ADVANCED

079–085 Modules & Packages
086–091 External Libraries
092–100 Advanced Topics
101–106 Enumerable & Enumerators
107–113 Metaprogramming
114–115 Quick Reference

RUBY

Table of Contents

115 sheets · syntax · examples · notes

INTRODUCTION & SETUP 001 – 004

001 Introduction & Philos…
002 Installation & Version …
003 Running Ruby
004 Program Structure & …

OUTPUT & COMMENTS 005 – 007

005 Output
006 Input & Command-Li…
007 Comments & Docum…

VARIABLES & DATA TYPES 008 – 015

008 Variables & Assignme…
009 Integers
010 Floats, Rational & Big…
011 Booleans & Truthiness
012 nil & Safe Navigation
013 Symbols
014 Type Conversion
015 Type Checking & Ide…

STRINGS 016 – 021

016 String Basics
017 Interpolation & Forma…
018 String Methods — Se…
019 String Methods — Tra…
020 Indexing, Slicing & Spl…
021 Heredocs, % Literals …

OPERATORS 022 – 027

022 Arithmetic Operators
023 Comparison & Space…
024 Logical Operators
025 Bitwise Operators
026 Assignment & Splat …
027 Special Operators

CONTROL FLOW 028 – 035

028 if, elsif, else & unless
029 Modifiers & Ternary
030 case / when
031 Pattern Matching — B…
032 Pattern Matching — A…
033 while, until & loop
034 Iterating: each, times …
035 Loop Controls

ERROR HANDLING 036 – 039

036 begin, rescue, else & …
037 Raising Exceptions
038 Exception Hierarchy …
039 retry & Cleanup Patte…

METHODS & BLOCKS 040 – 049

040 Defining & Calling Me…
041 Parameters & Keywor…
042 Variadic Params & Fo…
043 Variable Scope
044 Blocks & yield
045 Procs & &block
046 Lambdas
047 Closures
048 Method Objects & Co…
049 Recursion & Memoiza…

DATA STRUCTURES 050 – 060

050 Array Basics
051 Adding, Removing & S…
052 Array Operations
053 Array Slicing & Lookup
054 Hash Basics
055 Hash Methods
056 Hash Transformations
057 Ranges
058 Sets
059 Struct & Data
060 Sorting & Searching

OOP 061 – 072

061 Classes & Objects
062 Attribute Accessors
063 Class Methods & Clas…
064 Visibility
065 Inheritance & super
066 Modules as Mixins
067 Modules as Namespa…
068 Polymorphism & Duc…
069 Operator Overloading
070 Equality & Hash Keys
071 Conversion Protocols
072 Copying & Freezing

FILES & I/O 073 – 078

073 Reading Files
074 Writing & Appending
075 Paths & Directories
076 JSON
077 CSV
078 YAML

MODULES & PACKAGES 079 – 085

079 require & load
080 RubyGems
081 Bundler
082 Creating a Gem
083 Standard Library — M…
084 Standard Library — D…
085 Standard Library — O…

EXTERNAL LIBRARIES 086 – 091

086 Minitest
087 RSpec Basics
088 RSpec Matchers & M…
089 Rake
090 Debugging & Linting
091 HTTP Clients

ADVANCED TOPICS 092 – 100

092 Regular Expressions …
093 Regex in String Metho…
094 Threads & Mutex
095 Fibers & the Fiber Sc…
096 Ractors
097 Databases
098 Marshal & Serialization
099 Type Signatures: RBS …
100 Performance & Memo…

ENUMERABLE & ENUMERATORS 101 – 106

101 Enumerable — Iterate…
102 Enumerable — Filter …
103 Enumerable — Reduc…
104 Enumerable — Slice &…
105 Enumerators & Lazy E…
106 Making Classes Enum…

METAPROGRAMMING 107 – 113

107 Open Classes & Refin…
108 Introspection
109 Dynamic Dispatch & …
110 method_missing & D…
111 Singleton Classes
112 eval Family & Hooks
113 Building DSLs

QUICK REFERENCE 114 – 115

114 Quick Reference — S…
115 Quick Reference — C…

Sheet 001

Introduction & Philosophy

Matz · principle of least surprise · everything is an object · CRuby/MRI · JRuby · TruffleRuby · Christmas releases

WHAT RUBY IS

Ruby is a **dynamic, object-oriented** language created by **Yukihiro "Matz" Matsumoto** in 1995, designed for programmer happiness. **Everything is an object**, including integers, `nil` and classes themselves. Code reads close to English, blocks make iteration expressive, and the standard library ships batteries for files, JSON, dates and networking.

QUICK FACTS

- File extension `.rb` ; run with `ruby file.rb`
- **CRuby / MRI** is the reference implementation (written in C)
- New major/minor versions ship every **December 25**
- Ruby 4.0 (2025): ZJIT, `Ruby::Box` , core `Set` , `Ractor::Port`
- Best known for **Rails**, scripting, DevOps (Chef, Vagrant), CLIs

HELLO, RUBY

```ruby
# hello.rb
puts "Hello, World!"          # Hello, World!

5.times { |i| print i }       # 01234
puts

3.class                       # => Integer
nil.class                     # => NilClass
Integer.class                 # => Class
3.even?                       # => false
"ruby".upcase.reverse         # => "YBUR"
[3, 1, 2].sort.first          # => 1

RUBY_VERSION                  # => "4.0.0"
RUBY_ENGINE                   # => "ruby"
```

CODE EXPLAINED

- → `puts` — prints a line; no `main()` or boilerplate needed
- → `5.times { }` — integers are objects with methods
- → `nil.class` — even "nothing" is an object
- → `Integer.class` — classes are objects too (instances of `Class`)
- → `RUBY_VERSION` — constant for the running interpreter version

THE RUBY WAY

- ◆ `for i in 0...5` → `5.times { |i| }`
- ◆ `if (x == nil)` → `if x.nil?` or just `unless x`
- ◆ `return result` at the end → let the last expression return

IMPLEMENTATIONS

NAME	NOTES
`CRuby / MRI`	reference, C, YJIT / ZJIT
`JRuby`	runs on the JVM, Java interop
`TruffleRuby`	GraalVM, high peak speed
`mruby`	embeddable, tiny footprint

RELEASE TIMELINE

VERSION	HIGHLIGHT
`1.9 (2007)`	YARV VM, new hash syntax
`2.0 (2013)`	keyword args, refinements
`3.0 (2020)`	Ractors, RBS, pattern matching
`3.4 (2024)`	`it` block param, Prism parser
`4.0 (2025)`	ZJIT, core `Set`, `Ruby::Box`

⚠ COMMON MISTAKES

- Assuming Ruby is "just Rails"; Rails is a framework, Ruby is the language underneath it
- Following old 1.8/1.9 tutorials; hash syntax, keyword args and string encoding all changed since
- Expecting static types; Ruby checks types at runtime, so typos surface only when the line runs

✓ TIPS & BEST PRACTICES

- Check `ruby -v` before following a tutorial; pin the version per project with `.ruby-version`
- Use `irb` to try every snippet on these sheets; it prints each result for you
- Read the official docs at `docs.ruby-lang.org` , or offline with `ri String#split`

QUICK SUMMARY

- `everything is an object` — `3.even?` , `nil.class`
- `puts` — print with newline, no boilerplate
- `CRuby` — the default implementation
- `Dec 25` — yearly release day

Installation & Version Managers

GETTING RUBY INSTALLED

macOS and most Linux distros ship an old **system Ruby** that you should not use for projects. Install Ruby with a **version manager** so each project can pin its own version in a `.ruby-version` file and gems install without `sudo`.

BEFORE YOU START

- Never `sudo gem install` ; that writes into the system Ruby
- Version managers compile Ruby, so install build deps first (`libyaml` , `openssl`)
- On Windows, use **RubyInstaller** with the MSYS2 devkit
- `mise` and `asdf` manage Ruby, Node, Python, etc. with one tool

INSTALL WITH RBENV OR MISE

```
# macOS — Homebrew + rbenv
brew install rbenv ruby-build
rbenv init                    # adds shims to your shell
rbenv install 4.0.0
rbenv global 4.0.0            # default everywhere
rbenv local 3.4.7             # writes .ruby-version here

# mise (any OS)
mise use --global ruby@4.0
mise use ruby@3.4             # project-level (mise.toml)

# verify
ruby -v                       # ruby 4.0.0 (2025-12-25 ...)
which ruby                    # ~/.rbenv/shims/ruby
gem -v                        # RubyGems version
```

CODE EXPLAINED

- → `rbenv init` — puts shims ahead of system Ruby on `PATH`
- → `rbenv install` — downloads and compiles a version
- → `global` / `local` — default version vs per-folder pin
- → `mise use` — same idea; writes `mise.toml` or `.tool-versions`
- → `which ruby` — confirm a shim, not `/usr/bin/ruby`

THE RUBY WAY

- `sudo gem install x` → install Ruby with a version manager, then `gem install x`
- Hard-coding versions in docs → commit `.ruby-version`
- Mixing rvm and rbenv → pick one manager per machine

VERSION MANAGERS

TOOL	STYLE
`rbenv`	shims, minimal, popular
`mise`	multi-language, fast, `mise.toml`
`asdf`	multi-language plugins
`chruby`	tiny; pair with `ruby-install`
`rvm`	older, heavier, gemsets

VERSION FILES

FILE	READ BY
`.ruby-version`	rbenv, chruby, mise, asdf, Bundler
`.tool-versions`	asdf, mise
`mise.toml`	mise
`Gemfile ruby "4.0.0"`	Bundler (enforced)

⚠ COMMON MISTAKES

- Running `gem install` with `sudo` on macOS; it fails on modern macOS and pollutes system Ruby
- Forgetting to restart the shell after `rbenv init` , so `ruby -v` still shows the system version
- Missing `openssl` / `libyaml` headers, so the compile fails or `require "psych"` breaks later

✓ TIPS & BEST PRACTICES

- Run `rbenv rehash` (or reopen the terminal) after installing gems that add executables
- Put `ruby "~> 4.0"` in the Gemfile so Bundler warns when the wrong Ruby is active
- Use `rbenv install -l` / `mise ls-remote ruby` to see installable versions

QUICK SUMMARY

- `version manager` — never use system Ruby
- `.ruby-version` — per-project pin
- `ruby -v` / `which ruby` — verify the active install
- `no sudo` — gems install into your user Ruby

Running Ruby

WAYS TO RUN CODE

Ruby is **interpreted**: `ruby file.rb` parses and runs the file directly, with no separate compile step. **irb** is the interactive REPL for experiments, and `ruby -e` runs one-liners. Since 3.1 the **YJIT** compiler can speed up long-running programs; Ruby 4.0 adds the experimental **ZJIT**.

GOOD TO KNOW

- `irb` shows the return value of every expression (`=>`)
- A shebang plus `chmod +x` makes a script runnable directly
- `__FILE__ == $0` is true only when the file is run, not required
- `-w` turns on warnings; worth using while learning

RUNNING RUBY

```
ruby hello.rb                # run a file
ruby -e 'puts 2 ** 10'       # 1024
ruby -w app.rb               # with warnings
ruby --yjit server.rb        # enable YJIT
ruby -c app.rb               # Syntax OK (check only)
irb                          # interactive shell
```

SCRIPT.RB

```
#!/usr/bin/env ruby
def main
  puts "Args: #{ARGV.inspect}"
end

main if __FILE__ == $0       # only when run directly
```

CODE EXPLAINED

- → `ruby -e` — run code passed on the command line
- → `-c` — parse only; reports `Syntax OK` or the error
- → `--yjit` — JIT compile hot methods (also `RUBY_YJIT_ENABLE=1`)
- → `#!/usr/bin/env ruby` — finds the active Ruby on `PATH`
- → `__FILE__ == $0` — guard like Python's `if __name__ == "__main__"`

THE RUBY WAY

- Pasting into `ruby` to test → use `irb` and see each result
- `if __FILE__ == $0 then main end` → `main if __FILE__ == $0`
- `#!/usr/bin/ruby` → `#!/usr/bin/env ruby` so version managers work

COMMAND-LINE FLAGS

FLAG	EFFECT
`-e 'code'`	run inline code
`-n / -p`	loop over input lines (–p prints)
`-i.bak`	edit files in place
`-r lib`	require a library first
`-I dir`	add to $LOAD_PATH
`--yjit / --zjit`	enable a JIT

IRB COMMANDS

COMMAND	DOES
`_`	last result
`help`	list irb commands
`ls obj`	show methods / vars
`show_source m`	print method source
`exit`	quit

⚠ COMMON MISTAKES

- Forgetting `chmod +x` and then wondering why `./script.rb` says permission denied
- Running a library file directly and executing code meant only for scripts; use the `__FILE__` guard
- Enabling YJIT for a 1-second script; JIT warm-up costs more than it saves on short runs

✓ TIPS & BEST PRACTICES

- One-liners like `ruby -ne 'print if /ERROR/' app.log` replace many grep/awk tricks
- Use `irb -r ./lib/my_code` to load your code into a REPL session
- Set `export RUBY_YJIT_ENABLE=1` for long-running servers and workers

QUICK SUMMARY

- `ruby file.rb` — run a script
- `irb` — REPL with live results
- `ruby -e` — one-liner
- `__FILE__ == $0` — "run directly" guard

Sheet 004

Program Structure & Style

no main() · newline-terminated statements · end keyword · { } vs do…end · 2-space indent · ? and ! naming · RuboCop

HOW A RUBY FILE IS LAID OUT

Ruby has **no `main()`** and no required boilerplate: statements run top to bottom. Lines end statements (no semicolons needed), blocks close with **`end`**, and indentation is **two spaces** by convention but not enforced. Method names may end in **`?`** (returns a boolean) or **`!`** (dangerous or mutating variant).

STYLE CONVENTIONS

- `snake_case` methods/variables, `CamelCase` classes, `SCREAMING_CASE` constants
- `{ }` for one-line blocks, `do...end` for multi-line
- **RuboCop** (or **Standard**) enforces the community style guide

TYPICAL FILE STRUCTURE

```ruby
# frozen_string_literal: true
require "json"                  # libraries first

MAX_RETRIES = 3                 # constants

class Greeter                   # class / module definitions
  def initialize(name)
    @name = name
  end

  def greet!  = puts("Hi, #{@name}")   # endless method
  def valid?  = !@name.empty?
end

g = Greeter.new("Ada")          # script body runs top-down
g.greet! if g.valid?            # Hi, Ada
[1, 2].each { |n| puts n }      # one-line block: { }
[1, 2].each do |n|              # multi-line block: do...end
  puts n * 10
end
```

CODE EXPLAINED

- → `# frozen_string_literal: true` — magic comment; must be the first line(s)
- → `require` — load libraries before using them
- → `class ... end` — every block construct closes with `end`
- → `greet!` / `valid?` — naming signals intent, not enforced by Ruby
- → `def x = expr` — endless method for one-liners (3.0+)

THE RUBY WAY

- `print("hi");` → `puts "hi"` (no semicolons, parens optional)
- `def isValid()` → `def valid?`
- Multi-line `{ }` blocks → `do ... end`

NAMING RULES

KIND	EXAMPLE
local / method	user_name, save
predicate	empty?, valid?
bang	sort!, save!
class / module	HttpClient
constant	MAX_SIZE
instance / class var	@name, @@count

STATEMENT RULES

RULE	EXAMPLE
newline ends a statement	x = 1
; joins statements	a = 1; b = 2
trailing operator continues	total = a + ↵ b
leading . continues	list ↵ .map(&:to_s)

⚠ COMMON MISTAKES

- Putting the magic comment after `require`; it only works at the top of the file
- Writing `{ }` with a `do ... end` meaning: `puts [1].map do ... end` binds the block to `puts`
- Assuming `!` always means "mutates"; it means "the more dangerous version" (e.g. `save!` raises)

✓ TIPS & BEST PRACTICES

- Run `rubocop -a` to auto-fix most style issues
- Start every file with `# frozen_string_literal: true`; it catches accidental string mutation
- Use `?` methods that return true/false, never other values like `nil` or strings

QUICK SUMMARY

- `no main()` — code runs top to bottom
- `end` — closes class, def, if, do
- `?` / `!` — predicate / dangerous variant
- `2 spaces` — conventional indentation

Output

PRINTING OUTPUT

Ruby has three everyday output methods: **puts** adds a newline and calls `to_s`, **print** writes without a newline, and **p** prints the `inspect` form (quotes, nils, types visible) and **returns its argument**, which makes it ideal for debugging. **pp** pretty-prints large nested structures.

KEY DIFFERENCES

- `puts nil` prints an empty line; `p nil` prints `nil`
- `puts [1, 2]` prints each element on its own line
- `puts` and `print` return `nil`; `p` returns the object
- `warn` writes to **stderr**, not stdout

OUTPUT METHODS

```
puts "Hello"              # Hello⏎
puts "a", "b"             # a⏎b⏎
puts [1, [2, 3]]          # 1⏎2⏎3⏎ (flattened)
print "no", "line"        # noline
print "x\n"               # x⏎

p "Hello"                 # "Hello"
p 1, nil                  # 1⏎nil
x = p(42) + 1             # prints 42, x == 43
pp({ user: { id: 1, roles: [:admin] } })

printf("%-5s|%3d\n", "id", 7)   # id   |   7
$stdout.puts "explicit stdout"
$stdout.write("raw", "\n")      # returns bytes written
warn "Disk almost full"         # → stderr
$stderr.puts "error!"
```

CODE EXPLAINED

- → `puts` — `to_s` + newline; arrays print one item per line
- → `print` — no newline, no separator between args
- → `p` — `inspect` form; returns its argument so you can wrap expressions
- → `pp` — pretty-print, wraps long nested data
- → `warn` — stderr; skipped when run with `-W0`

THE RUBY WAY

- `puts x.inspect` → `p x`
- `print "text\n"` → `puts "text"`
- `$stdout.puts "err"` for errors → `warn "err"`

PUTS VS PRINT VS P

INPUT	PUTS	PRINT	P
`"hi"`	hi↵	hi	"hi"↵
`nil`	↵	(nothing)	nil↵
`[1, 2]`	1↵2↵	[1, 2]	[1, 2]↵
`:sym`	sym↵	sym	:sym↵
`returns`	nil	nil	the object

STREAMS

OBJECT	USE
`$stdout / STDOUT`	normal output
`$stderr / STDERR`	errors, warnings
`$stdout.sync = true`	unbuffered (logs, pipes)
`$stdout.flush`	force write now

⚠ COMMON MISTAKES

- Debugging with `puts`; `puts nil` and `puts ""` both print a blank line, so you can't tell them apart. Use `p`
- Expecting `print` to add spaces between arguments; `print "a", "b"` prints `ab`
- Leaving `p` calls in production code; they also change return values when used as the last line

✓ TIPS & BEST PRACTICES

- `p` returns its argument, so `result = p(compute)` logs without restructuring code
- Set `$stdout.sync = true` in scripts piped to other tools so output appears immediately
- Use `pp` for hashes from JSON; it wraps and indents automatically

QUICK SUMMARY

- `puts` — to_s + newline
- `print` — no newline
- `p` — inspect + returns arg
- `warn` — stderr

Input & Command-Line Arguments

gets · chomp · $stdin · STDIN.read · ARGV · ARGF · Integer(gets)

READING INPUT

`gets` reads one line from standard input **including the trailing newline**, so it is almost always followed by **`chomp`**. Command-line arguments arrive as strings in the **ARGV** array, and **ARGF** reads from files named in ARGV or from stdin when none are given, like Unix filters do.

INPUT NOTES

- `gets` returns `nil` at end of input (Ctrl-D)
- `ARGV` holds strings only; convert with `Integer()` / `Float()`
- Inside a script, `gets` reads from files in `ARGV` first; use `$stdin.gets` to force the keyboard
- `$stdin.read` slurps everything until EOF

PROMPTS, ARGV AND ARGF

```
print "Name: "
name = gets.chomp                  # "Ada"
print "Age: "
age = Integer(gets)                # raises on "abc"
age = gets.to_i                    # "abc" → 0 (silent)

# ruby greet.rb Ada 36 --loud
ARGV                               # ["Ada", "36", "--loud"]
first, *rest = ARGV                # "Ada", ["36", "--loud"]
loud = ARGV.delete("--loud")       # removes the flag

$stdin.each_line do |line|         # read until EOF
  puts line.chomp.upcase
end

# cat a.txt | ruby filter.rb   or   ruby filter.rb a.txt b.txt
ARGF.each_line { |l| print l if l =~ /TODO/ }
```

CODE EXPLAINED

- → `gets.chomp` — strip the `"\n"` that `gets` keeps
- → `Integer(gets)` — strict: raises `ArgumentError` on bad input
- → `gets.to_i` — lenient: returns `0` for junk
- → `ARGV.delete` — pull flags out before treating the rest as files
- → `ARGF` — files from ARGV, else stdin

THE RUBY WAY

- `gets.strip.to_i` everywhere → `Integer(gets, exception: false)` and handle `nil`
- Manually opening each ARGV file → iterate `ARGF`
- Hand-parsing many flags → `OptionParser` (sheet 085)

INPUT SOURCES

EXPRESSION	RETURNS
`gets`	next line or `nil`
`gets.chomp`	line without `\n`
`$stdin.read`	all of stdin
`$stdin.readlines(chomp: true)`	array of lines
`ARGV`	array of arg strings
`ARGF.read`	all input files / stdin

STRING → NUMBER

CALL	"42ABC"
`to_i`	42
`Integer(s)`	ArgumentError
`Integer(s, exception: false)`	nil
`to_f`	42.0
`Float(s)`	ArgumentError

⚠ COMMON MISTAKES

- Forgetting `chomp`, so `"yes\n" == "yes"` is false
- Calling `gets` in a script that received file args; it reads the file, not the keyboard. Use `$stdin.gets`
- Using `to_i` for validation; `"abc".to_i` is `0`, which looks like valid input

✓ TIPS & BEST PRACTICES

- `while (line = gets)` loops until EOF and works with pipes
- Use `Integer(str, exception: false)` to get `nil` instead of rescuing
- `STDIN.tty?` tells you whether input is a keyboard or a pipe

QUICK SUMMARY

- `gets.chomp` — read a clean line
- `ARGV` — argument strings
- `ARGF` — files or stdin
- `Integer()` — strict conversion

Comments & Documentation

COMMENTS AND DOCS

`#` starts a comment that runs to the end of the line. `=begin` / `=end` block comments exist but are rarely used; prefer consecutive `#` lines. Comments at the very top of a file can be **magic comments** that change how Ruby parses it, and comments directly above a method become **RDoc / YARD** documentation.

COMMENT FACTS

- `=begin` and `=end` must start at column 0
- Magic comments must be in the first two lines
- `# frozen_string_literal: true` makes string literals immutable
- `ri` and IDEs read RDoc/YARD comments for hover help

COMMENT STYLES

```ruby
# Single-line comment
x = 42 # trailing comment

=begin
Block comment. Must start at column 0.
Rarely used in real code.
=end

# frozen_string_literal: true   (only valid on line 1-2)
# encoding: utf-8               (default already)

# Calculates the area of a rectangle.
#
# @param width [Numeric] side length
# @param height [Numeric] side length
# @return [Numeric] the area
# @example
#   area(2, 3) #=> 6
def area(width, height) = width * height

# TODO: handle negative sizes
```

CODE EXPLAINED

- → `#` — everything after it on the line is ignored
- → `=begin/=end` — block comment; column 0 only
- → magic comments — parser directives, not documentation
- → `@param` / `@return` — YARD tags; `yard doc` builds HTML
- → `#=>` — convention for showing a result in examples

THE RUBY WAY

- `=begin ... =end` → a run of `#` lines (editors toggle these)
- Comments that restate code → explain **why**, not what
- Commented-out dead code → delete it; git remembers

MAGIC COMMENTS

COMMENT	EFFECT
`frozen_string_literal: true`	string literals frozen
`encoding: utf-8`	source encoding (default)
`warn_indent: true`	warn on mismatched `end`
`shareable_constant_value: literal`	Ractor-safe constants

YARD TAGS

TAG	DOCUMENTS
`@param name [Type]`	an argument
`@return [Type]`	return value
`@raise [Error]`	exceptions raised
`@example`	usage sample
`@deprecated`	marks old API

⚠ COMMON MISTAKES

- Indenting `=begin`; it is then a syntax error, not a comment
- Placing `# frozen_string_literal: true` below the first code line, where Ruby ignores it
- Leaving a blank line between a doc comment and its `def`; RDoc then won't attach it

✓ TIPS & BEST PRACTICES

- Run `yard stats --list-undoc` to find public methods without docs
- Use `ri Array#each_slice` in the terminal for stdlib docs offline
- Keep `TODO:` / `FIXME:` tags consistent; `rails notes` and editors can list them

QUICK SUMMARY

- `#` — line comment
- `=begin/=end` — block comment (col 0)
- `# frozen_string_literal: true` — magic comment
- `@param` / `@return` — YARD docs

Variables & Assignment

VARIABLES AND ASSIGNMENT

Variables are created by **assignment**, with no declaration keyword and no type. A variable is a **name pointing at an object**; reassigning points it elsewhere. The first character (sigil) sets the **kind**: lowercase for locals, @ instance, @@ class, $ global, and an uppercase first letter for **constants**.

RULES

- Names: letters, digits, `_` ; can't start with a digit
- Reading an unassigned local raises `NameError`
- Reassigning a constant only **warns**; it doesn't stop you
- Assignment returns the assigned value: `a = b = 0`

ASSIGNMENT FORMS

```
name = "Ada"                  # local variable
count = 0
count += 1                    # 1

a = b = 0                     # chained: both 0
x, y = 1, 2                   # parallel assignment
x, y = y, x                   # swap → x=2, y=1
first, *rest = [1, 2, 3]      # 1, [2, 3]
head, (m, n) = 1, [2, 3]      # nested: m=2, n=3

MAX_USERS = 100               # constant
MAX_USERS = 200               # warning: already initialized

@title = "Book"               # instance variable
$debug = true                 # global (avoid)

defined?(zzz)                 # => nil
s1 = "hi"; s2 = s1            # same object
s2 << "!"; s1                 # => "hi!"
```

CODE EXPLAINED

- → `x, y = y, x` — swap without a temp variable
- → `first, *rest` — splat collects the remainder
- → `MAX_USERS = 200` — reassignment warns; constants aren't locked
- → `defined?` — check without raising `NameError`
- → `s2 = s1` — copies the reference, not the string

THE RUBY WAY

- `temp = x; x = y; y = temp` → `x, y = y, x`
- `arr[0]` / `arr[1]` lookups → `first, second = arr`
- `$globals` for config → constants or a config object

VARIABLE KINDS

SIGIL	SCOPE	EXAMPLE
none	local (method / block)	`total`
@	one object	`@name`
@@	class + subclasses	`@@count`
$	whole program	`$stdout`
Uppercase	constant (module)	`PI`

MULTIPLE ASSIGNMENT

CODE	RESULT
`a, b = 1, 2`	a=1 b=2
`a, b = [1, 2]`	a=1 b=2
`a, b = 1`	a=1 b=nil
`a, *b = 1, 2, 3`	b=[2, 3]
`*a, b = 1, 2, 3`	a=[1, 2] b=3

⚠ COMMON MISTAKES

- Mutating a string through one variable and being surprised the other changed; both point at one object
- Expecting constants to be immutable; freeze them (`LIST = %w[a b].freeze`) to stop mutation
- Typos create new variables silently (`totl = 1`), then the real one is used unchanged

✓ TIPS & BEST PRACTICES

- Use `dup` when you need an independent copy of a mutable object
- Freeze constant collections; the warning only covers rebinding, not `<<`
- Prefer locals and instance variables; globals make code hard to follow

QUICK SUMMARY

- `x = 1` — assignment creates the variable
- `x, y = y, x` — swap
- `CONST` — uppercase = constant
- `@` / `@@` / `$` — instance / class / global

Integers

INTEGERS

Ruby has a single **`Integer`** class with **arbitrary precision**, so integers never overflow; huge values just use more memory. Integers are objects with methods (`10.even?`, `255.to_s(2)`). Division between two integers **floors** toward negative infinity.

INTEGER FACTS

- Underscores improve readability: `1_000_000`
- Prefixes: `0x` hex, `0b` binary, `0o` octal
- `7 / 2` is `3`; `-7 / 2` is `-4` (floors)
- Integers are immutable and frozen

INTEGER BASICS

```
big = 2 ** 100                  # 1267650600228229401496703205376
1_000_000.class                 # Integer
0xff                            # 255
0b1010                          # 10
0o17                            # 15
255.to_s(2)                     # "11111111"
"ff".to_i(16)                   # 255

7 / 2                           # 3
-7 / 2                          # -4 (floor division)
7 % 3                           # 1
7.divmod(2)                     # [3, 1]
7.fdiv(2)                       # 3.5

10.even?                        # true
1234.digits                     # [4, 3, 2, 1]
3.pow(4, 5)                     # 1  (3**4 % 5, fast)
Integer.sqrt(17)                # 4
-5.abs                          # 5
```

CODE EXPLAINED

- → `2 ** 100` — no overflow; Ruby switches to bignum math internally
- → `to_s(2)` / `to_i(16)` — convert between bases
- → `-7 / 2` — floors, unlike C or Java (which give `-3`)
- → `divmod` — quotient and remainder together
- → `pow(b, mod)` — modular exponent without huge intermediates

THE RUBY WAY

- `1000000` → `1_000_000`
- `x % 2 == 0` → `x.even?`
- `(a.to_f / b)` → `a.fdiv(b)`

IN RAILS

- **AS** · `5.minutes`, `2.hours`, `1.day` build `ActiveSupport::Duration` values
- **AS** · `3.days.ago` / `2.weeks.from_now` return times
- **Rails** · views use `number_with_delimiter(1234567)` → `"1,234,567"`
- **AS** · `1.kilobyte` / `5.megabytes` for size limits

USEFUL METHODS

METHOD	RESULT
`5.times { }`	runs 5 times
`1.upto(3)`	1, 2, 3
`10.downto(8)`	10, 9, 8
`12.gcd(8) / 12.lcm(8)`	4 / 24
`7.clamp(1, 5)`	5
`3.zero? / 3.positive?`	false / true

ROUNDING INTEGERS

CALL	RESULT
`1234.round(-2)`	1200
`1250.round(-2)`	1300
`1234.floor(-2)`	1200
`1234.ceil(-2)`	1300
`15.pred / 15.succ`	14 / 16

⚠ COMMON MISTAKES

- Expecting `5 / 2` to be `2.5`; integer division drops the fraction. Use `fdiv` or a float operand
- Porting code from C/Java and getting different results for negative division and modulo
- Calling `Math.sqrt` on huge integers; it returns a float and loses precision. Use `Integer.sqrt`

✓ TIPS & BEST PRACTICES

- Use `Integer#digits` for digit-sum style puzzles instead of string conversion
- `rand(1..6)` gives a random integer in a range
- `Integer("0x1A")` reads base prefixes (26), but `"0x1A".to_i` is `0`; use `"1A".hex` or `to_i(16)` for bare hex

QUICK SUMMARY

- `Integer` — arbitrary precision
- `/` — floor division for integers
- `divmod` / `fdiv` — both parts / float result
- `to_s(base)` — base conversion

Floats, Rational & BigDecimal

FLOATS AND EXACT NUMBERS

`Float` is a 64-bit IEEE 754 double, so `0.1 + 0.2` is not exactly `0.3`. For exact fractions use **`Rational`** (`3r`, `1/3r`); for money and decimal precision use **`BigDecimal`**. Any operation mixing an Integer and a Float returns a Float.

FLOAT FACTS

- `Float::INFINITY`, `-Float::INFINITY`, `Float::NAN` exist
- `1.0 / 0` is `Infinity`; `1 / 0` raises `ZeroDivisionError`
- `round` uses half-away-from-zero by default
- `BigDecimal` needs `require "bigdecimal"` (bundled gem)

FLOAT, RATIONAL, BIGDECIMAL

```
0.1 + 0.2                       # 0.30000000000000004
(0.1 + 0.2).round(2)            # 0.3
3.14159.round(2)                # 3.14
3.7.floor / 3.2.ceil            # 3 / 4
-3.7.truncate                   # -3
2.5.round                       # 3
2.5.round(half: :even)          # 2

1.0 / 0                         # Infinity
(0.0 / 0).nan?                  # true
(0.1 + 0.2 - 0.3).abs < Float::EPSILON  # true

1/3r + 1/6r                     # (1/2)
0.75.rationalize                # (3/4)
require "bigdecimal"
BigDecimal("0.1") + BigDecimal("0.2")   # 0.3e0
BigDecimal("19.99").to_s("F")           # "19.99"
Complex(3, 4).abs               # 5.0
```

CODE EXPLAINED

- → `0.30000000000000004` — binary floats can't represent 0.1 exactly
- → `round(half: :even)` — banker's rounding when you need it
- → `Float::EPSILON` — tolerance for comparing floats
- → `1/3r` — `r` suffix makes an exact `Rational`
- → `BigDecimal("0.1")` — build from strings, never from floats

THE RUBY WAY

- `a == b` for floats → `(a - b).abs < 1e-9`
- Money as `Float` → integer cents or `BigDecimal`
- `x.to_f / y` → `x.fdiv(y)`

IN RAILS

- **Rails** · `decimal` columns load as `BigDecimal`, so `price.class` is `BigDecimal`
- **Rails** · `number_to_currency(19.99)` → `"$19.99"` in views
- **AS** · `BigDecimal#to_fs` formats decimals for display

NUMERIC CLASSES

CLASS	LITERAL	USE
`Integer`	`42`	counting, indexes
`Float`	`3.14`, `1e-3`	measurements
`Rational`	`3r`, `2/3r`	exact fractions
`BigDecimal`	`BigDecimal("1.1")`	money
`Complex`	`2i`, `Complex(1, 2)`	math

FLOAT METHODS

CALL	RESULT
`4.0.to_i`	4
`3.99.to_i`	3
`1.5e3`	1500.0
`7.0.divmod(2)`	[3, 1.0]
`12.345.floor(1)`	12.3
`1.23.infinite?`	nil

⚠ COMMON MISTAKES

- Storing currency in floats; `0.1 + 0.2` errors accumulate across totals
- Building `BigDecimal(0.1)` from a float; the float's error is already baked in
- Comparing `x == Float::NAN`; NaN never equals anything, use `x.nan?`

✓ TIPS & BEST PRACTICES

- Use `round(2)` only for display; keep full precision while calculating
- `Integer#fdiv` avoids accidentally truncating integer division
- `Rational` keeps fractions exact through long calculations, then convert at the end with `to_f`

QUICK SUMMARY

- `Float` — fast, approximate
- `Rational` — exact fractions (`3r`)
- `BigDecimal` — exact decimals for money
- `nan?` / `EPSILON` — safe float checks

Booleans & Truthiness

true / false · TrueClass · only nil and false are falsy · 0 and "" are truthy · !! · predicate methods

TRUE, FALSE AND TRUTHINESS

`true` and `false` are singleton objects of `TrueClass` and `FalseClass`; there is no `Boolean` class. In conditions, **only `nil` and `false` are falsy**. Everything else is truthy, including `0`, `""`, `[]` and `{}`, which surprises people coming from Python, JavaScript or C.

TRUTHINESS RULES

- Falsy: `false`, `nil`. That's the whole list
- Truthy: `0`, `0.0`, `""`, `[]`, `{}`, `"false"`
- `!!value` converts anything to `true` / `false`
- Predicate methods end in `?` and should return a boolean

TRUTHINESS IN PRACTICE

```ruby
true.class                  # TrueClass
true & false                # false

puts "zero is truthy" if 0      # prints
puts "empty is truthy" if ""    # prints
puts "never" if nil             # (nothing)

!!nil                       # false
!!0                         # true
nil.to_a                    # []

[].empty?                   # true  — test emptiness explicitly
"".empty?                   # true
0.zero?                     # true
items = []
puts "none" if items.empty?     # none

flag = "false"
flag == "true"              # false — parse strings yourself
```

CODE EXPLAINED

→ `TrueClass` — `true` is the only instance; no `Boolean` type exists

→ `if 0` — runs; `0` is just an object

→ `!!x` — double negation gives a real boolean

→ `empty?` / `zero?` — ask the question you mean

→ `"false"` — a non-empty string, so truthy

THE RUBY WAY

- `if items.length > 0` → `if items.any?` or `unless items.empty?`
- `return x ? true : false` → `!!x` (or return `x` from a `?` method that already returns a boolean)
- `if flag == true` → `if flag`

IN RAILS

- **AS** · `blank?` is true for `nil`, `false`, `""`, `" "`, `[]`, `{}`
- **AS** · `present?` is `!blank?`; neither exists in plain Ruby
- **Rails** · boolean columns cast `"0"`, `"false"`, `"off"` to `false`

FALSY VS TRUTHY

VALUE	IN IF
`false`	falsy
`nil`	falsy
`0` / `0.0`	truthy
`""` / `"false"`	truthy
`[]` / `{}`	truthy

BOOLEAN OPERATORS

EXPR	RESULT
`true && false`	false
`true || false`	true
`!true`	false
`true & false`	false (no short-circuit)
`true ^ false`	true

⚠ COMMON MISTAKES

- Testing `if count` to mean "count is non-zero"; `0` is truthy, so use `count.positive?` or `count.zero?`
- Treating `""` as false when validating form input; check `str.empty?` or `str.strip.empty?`
- Comparing `x == true`; it fails for truthy non-boolean values like `1` or `"yes"`

✓ TIPS & BEST PRACTICES

- Name boolean-returning methods with `?` and keep them returning only `true` / `false`
- `Array#any?` with no block is false for `[nil, false]`; it checks truthiness of elements
- To parse a "boolean" string: `%w[true yes 1 on].include?(s.downcase)`

QUICK SUMMARY

- `nil` / `false` — the only falsy values
- `0`, `""`, `[]` — truthy
- `!!x` — force a boolean
- `empty?` / `zero?` — explicit checks

nil & Safe Navigation

NIL AND SAFE NAVIGATION

`nil` is the single instance of `NilClass` and means "no value". Calling a method on `nil` that it doesn't define raises **`NoMethodError`**, the most common Ruby error. The **safe navigation operator `&.`** returns `nil` instead of raising, and **`||` / `||=`** supply defaults.

NIL FACTS

- Many lookups return `nil` on a miss: `hash[:x]`, `arr[99]`, `str[/re/]`
- `nil.to_s` is `""`, `nil.to_a` is `[]`, `nil.to_i` is `0`
- `&.` skips the call **and any arguments**; it doesn't evaluate them
- `x ||= v` assigns only when `x` is `nil` or `false`

HANDLING NIL

```
user = nil
user.name                    # NoMethodError
user&.name                   # nil
user&.profile&.city          # nil (chain stops at first nil)
user.nil?                    # true

name = user&.name || "guest" # "guest"
@cache ||= {}                # set once, reuse after

h = { a: 1 }
h[:b]                        # nil  (silent miss)
h.fetch(:b)                  # KeyError: key not found: :b
h.fetch(:b, 0)               # 0
h.fetch(:b) { |k| "no #{k}" }  # "no b"

Array(nil)                   # []
Array([1, 2])                # [1, 2]  (no double wrap)
Array("x")                   # ["x"]
[1, nil, 2, nil].compact     # [1, 2]
```

CODE EXPLAINED

→ `&.` — call only if the receiver isn't `nil`

→ `|| "guest"` — default when the left side is `nil` / `false`

→ `||=` — memoize; common for lazy instance variables

→ `fetch` — raise or default instead of a silent `nil`

→ `Array(x)` — normalize "nil, one, or many" into an array

THE RUBY WAY

- `if user && user.profile && user.profile.city` → `user&.profile&.city`
- `@x = @x || compute` → `@x ||= compute`
- `hash[:key]` when the key must exist → `hash.fetch(:key)`

IN RAILS

- **AS** · `presence` returns the value or `nil` when blank: `params[:q].presence || "all"`
- **AS** · `user.try(:name)` is like `&.` but also skips methods it doesn't respond to
- **AS** · `blank?` treats `nil` and `""` alike

NIL CONVERSIONS

CALL	RESULT
`nil.to_a`	[]
`nil.to_h`	{}
`nil.to_i`	0
`nil.inspect`	"nil"

WHERE NIL COMES FROM

EXPRESSION	RESULT
`[1, 2][5]`	nil
`{ a: 1 }[:z]`	nil
`"abc"[/\d/]`	nil
`[].first`	nil
`puts "x"`	nil (return value)

⚠ COMMON MISTAKES

- Using `||=` for booleans: `@enabled ||= true` overwrites a stored `false`. Use `@enabled = true if @enabled.nil?`
- Chaining `&.` everywhere and hiding real bugs; a required value that is `nil` should raise early
- Forgetting `&.` doesn't help with `[]`: `nil&.[](0)` works, but `nil[0]` still raises

✓ TIPS & BEST PRACTICES

- Prefer `fetch` for config and required keys; the error message names the missing key
- Use `Array(x)` at method boundaries so callers can pass one item or many
- Ruby 3.4+ error messages show the receiver: `undefined method 'name' for nil`, which points to the nil source

QUICK SUMMARY

- `nil` — the only `NilClass` object
- `&.` — safe navigation
- `||=` — default/memoize
- `fetch` — loud lookup

Symbols

SYMBOLS

A **Symbol** (`:name`) is an immutable, interned identifier: every `:name` in a program is the **same object**. Symbols are used for hash keys, method names, and option flags where the value is a label rather than text. Strings are for data you manipulate; symbols are for names.

SYMBOL FACTS

- `:a.object_id == :a.object_id` is always true
- `{ name: "Ada" }` uses a symbol key; `{ "name" => "Ada" }` a string key
- Symbols are frozen and can't be mutated
- `%i[a b c]` builds an array of symbols

SYMBOLS VS STRINGS

```
:status.class                # Symbol
:ok.object_id == :ok.object_id   # true  (one object)

:name.to_s                   # "name"
"name".to_sym                # :name
:"first name"                # quoted symbol with a space
%i[draft live archived]      # [:draft, :live, :archived]

h = { name: "Ada" }          # symbol key
h[:name]                     # "Ada"
h["name"]                    # nil  — different key!

user.send(:name)             # call a method by name
[1, 2, 3].map(&:to_s)        # ["1", "2", "3"]
:upcase.to_proc.call("hi")   # "HI"
```

CODE EXPLAINED

→ one object — every `:ok` is identical, so comparisons are fast

→ `to_sym` / `to_s` — convert both ways

→ `h["name"]` — string and symbol keys never match each other

→ `&:to_s` — `Symbol#to_proc` turns a method name into a block

→ `send(:name)` — symbols name methods for dynamic calls

THE RUBY WAY

- `{ "status" => "active" }` for fixed labels → `{ status: :active }`
- `map { |x| x.to_s }` → `map(&:to_s)`
- `params[:x].to_sym` on user input → keep untrusted text as strings

IN RAILS

- **Rails** · `params` accept both: `params[:id]` and `params["id"]` work (`HashWithIndifferentAccess`)
- **AS** · `hash.with_indifferent_access` gives the same behavior to any hash
- **AS** · `hash.symbolize_keys` / `stringify_keys` convert keys

SYMBOL SYNTAX

LITERAL	VALUE
`:name`	simple symbol
`:"two words"`	symbol with spaces
`:"id_#{n}"`	interpolated symbol
`%i[a b]`	[:a, :b]

USE A SYMBOL WHEN

SITUATION	CHOICE
hash keys with fixed names	symbol
method / attribute names	symbol
states and options	symbol
user-entered text	string
anything you change	string

⚠ COMMON MISTAKES

- Looking up `h["name"]` in a hash built with `name:` keys; the result is `nil`, not an error
- JSON parsing returns string keys; access them with strings or pass `symbolize_names: true`
- Comparing a symbol to a string (`:ok == "ok"` is `false`); convert one side first

✓ TIPS & BEST PRACTICES

- Use `&:method` only for zero-argument methods; `map(&:round)` works, `map(&:round(2))` doesn't
- `Symbol#start_with?`, `#length`, `#upcase` exist, so many string ops work directly on symbols
- Symbols are garbage-collected since Ruby 2.2, but still avoid creating unbounded numbers of them

QUICK SUMMARY

- `:name` — interned, immutable label
- `to_sym` / `to_s` — convert
- `name:` — symbol hash key
- `&:method` — symbol as block

Type Conversion

to_s · to_i · to_f · Integer() / Float() strict · to_a · to_sym · implicit to_str / to_ary

CONVERTING BETWEEN TYPES

Ruby has two families of conversion. **Explicit** methods like `to_s`, `to_i`, `to_f`, `to_a` always try and return something, even if it's `0` or `""`. **Strict** Kernel functions like `Integer()`, `Float()`, `String()`, `Array()` raise on bad input, which is what you want for validation.

CONVERSION RULES

- `"12abc".to_i` is `12`; `"abc".to_i` is `0`
- `Integer("12abc")` raises `ArgumentError`
- Ruby never converts implicitly in `+`: `"a" + 1` raises `TypeError`
- `to_str` / `to_ary` are **implicit** protocols used by core methods

TO_* VS KERNEL()

```
"42".to_i                    # 42
"42.7".to_i                  # 42
"3.5kg".to_f                 # 3.5
"abc".to_i                   # 0
Integer("42")                # 42
Integer("4x")                # ArgumentError
Integer("4x", exception: false)  # nil
Float("1e3")                 # 1000.0

42.to_s                      # "42"
3.0.to_s                     # "3.0"
3.7.to_i                     # 3 (truncates)
3.7.round                    # 4
:id.to_s                     # "id"
"id".to_sym                  # :id
[[:a, 1], [:b, 2]].to_h      # {a: 1, b: 2}
{ a: 1 }.to_a                # [[:a, 1]]
(1..3).to_a                  # [1, 2, 3]
"Total: " + 5                # TypeError
"Total: #{5}"                # "Total: 5"
```

CODE EXPLAINED

- → `to_i` — reads leading digits, ignores the rest, never raises
- → `Integer()` — whole string must be a valid integer
- → `exception: false` — return `nil` instead of raising
- → `3.7.to_i` — truncates toward zero; use `round` to round
- → `"a" + 5` — no implicit coercion; interpolate instead

THE RUBY WAY

- `"Total: " + n.to_s` → `"Total: #{n}"`
- `Integer(s) rescue nil` → `Integer(s, exception: false)`
- `value.to_s.to_i` chains → convert once at the input boundary

IN RAILS

- **Rails** · ActiveRecord casts on assignment: `user.age = "42"` stores `42`
- **AS** · `{ q: "ruby", page: 2 }.to_query` → `"page=2&q=ruby"`
- **Rails** · `to_param` is what models return in URLs (the `id` by default)

LENIENT VS STRICT

INPUT	TO_I	INTEGER()
`"42"`	42	42
`" 42 "`	42	42
`"42abc"`	42	error
`""`	0	error
`"0x1F"`	0	31
`nil`	0 (nil.to_i)	TypeError

COMMON CONVERSIONS

FROM → TO	METHOD
`number → string`	`to_s`, `format`
`string → number`	`to_i`, `to_f`, `Integer()`
`pairs → hash`	`to_h`
`hash → pairs`	`to_a`
`range → array`	`to_a`, `[*1..3]`
`float → int`	`to_i`, `round`, `floor`

⚠ COMMON MISTAKES

- Validating input with `to_i`; junk becomes `0` and passes as a real number
- Expecting `"5" + 5` to coerce like JavaScript; Ruby raises `TypeError`
- Using `to_i` to round prices; `19.99.to_i` is `19`

✓ TIPS & BEST PRACTICES

- Convert at the edge (user input, files, env vars) and work with real types inside
- `Float(x, exception: false)` makes a quick "is this numeric?" check
- `Integer("0b101")`, `Integer("0o17")` and `Integer("0x1F")` read base prefixes

QUICK SUMMARY

- `to_i` / `to_f` / `to_s` — lenient conversion
- `Integer()` / `Float()` — strict conversion
- `exception: false` — nil on failure
- no implicit coercion — interpolate

Type Checking & Identity

.class · is_a? · kind_of? · instance_of? · respond_to? · == vs eql? vs equal? · object_id

CHECKING TYPES AND IDENTITY

Ruby is **duck typed**: you usually ask whether an object can do something (`respond_to?`) rather than what class it is. When you do need the class, `is_a?` checks the whole ancestry, `instance_of?` checks the exact class, and `===` powers `case/when`. Ruby has three different equality methods with different jobs.

EQUALITY METHODS

- `==` — same value (overridden by most classes)
- `eql?` — same value **and** type; used by Hash keys
- `equal?` — same object (identity); never override it
- `object_id` — unique integer per live object

TYPE CHECKS

```ruby
5.class                     # Integer
5.is_a?(Integer)            # true
5.is_a?(Numeric)            # true  (ancestor)
5.instance_of?(Numeric)     # false (exact class only)
Integer === 5               # true  (used by case/when)
Integer.ancestors.take(4)   # [Integer, Numeric, Comparable, Object]

"hi".respond_to?(:upcase)   # true
nil.respond_to?(:each)      # false

1 == 1.0                    # true
1.eql?(1.0)                 # false (Integer vs Float)
a = "x"; b = "x"
a == b                      # true
a.equal?(b)                 # false (two objects)
a.equal?(a)                 # true

case 5
when String  then "text"
when Integer then "number"  # => "number"
end
```

CODE EXPLAINED

- → `is_a?` / `kind_of?` — same method; includes superclasses and modules
- → `instance_of?` — exact class, ignores inheritance
- → `Integer === 5` — class `===` means "is an instance of"
- → `respond_to?` — duck-typing check
- → `eql?` — stricter `==`; why `1` and `1.0` are different hash keys

THE RUBY WAY

- `if x.class == String` → `if x.is_a?(String)` (or better, `respond_to?`)
- `if x.is_a?(A) ... elsif x.is_a?(B)` → `case x when A ... when B`
- Checking types before every call → call it and let `NoMethodError` surface

CHECK METHODS

CALL	INHERITANCE?	
`obj.class`	exact class	—
`is_a?(C)` / `kind_of?(C)`	yes	
`instance_of?(C)`	no	
`C === obj`	yes	
`respond_to?(:m)`	duck typing	

EQUALITY AT A GLANCE

EXPR	RESULT
`1 == 1.0`	true
`1.eql?(1.0)`	false
`1.equal?(1)`	true (small ints shared)
`"a" == "a"`	true
`"a".equal?("a")`	false

⚠ COMMON MISTAKES

- Writing `case x when "Integer"`; `when` needs the class constant, not a string
- Using `equal?` to compare values; it only checks identity, so equal strings return `false`
- Reversing `===`: `5 === Integer` is `false`; the pattern goes on the left

✓ TIPS & BEST PRACTICES

- Prefer `respond_to?(:each)` over `is_a?(Array)` so hashes, sets and custom collections also work
- Pass several classes to `when`: `when Integer, Float`
- `obj.class.ancestors` shows exactly what `is_a?` will accept

QUICK SUMMARY

- `is_a?` — class or ancestor
- `instance_of?` — exact class
- `respond_to?` — duck typing
- `==` / `eql?` / `equal?` — value / value+type / identity

String Basics

STRINGS

Ruby strings are **mutable** sequences of characters with an encoding (UTF-8 by default). **Double quotes** support interpolation and escapes; **single quotes** are literal except for `\'` and `\\`. With `# frozen_string_literal: true` at the top of a file, string literals are frozen and mutating them raises `FrozenError`.

STRING FACTS

- `length` counts characters; `bytesize` counts bytes
- `+` builds a new string; `<<` appends in place
- Ruby 3.4+ warns when you mutate an unfrozen literal (chilled strings)
- `+""` or `String.new` gives a mutable string under the magic comment

STRING BASICS

```
# frozen_string_literal: true
name = "Ada"
'Hi #{name}\n'                # literal: Hi #{name}\n
"Hi #{name}\n"                # Hi Ada⏎
"café".length                 # 4
"café".bytesize               # 5
"".empty?                     # true

a = "Hi"
b = a + "!"                   # new string; a unchanged
a << "!"                      # FrozenError (frozen literal)
buf = +""                     # mutable copy
buf << "Hello" << ", " << name   # "Hello, Ada"
buf.frozen?                   # false

"ab" * 3                      # "ababab"
"a" == "a"                    # true
"apple" < "banana"            # true (byte order)
"Ruby".dup.frozen?            # false
"%.2f" % 3.14159              # "3.14"
```

CODE EXPLAINED

- → `'...'` — no interpolation, no `\n`
- → `length` vs `bytesize` — `é` is 1 character, 2 bytes
- → `+` vs `<<` — copy vs append in place
- → `+""` — unary plus returns an unfrozen string
- → `"%.2f" %` — format operator (sheet 017)

THE RUBY WAY

- `s = s + more` in a loop → `s << more` on a `+""` buffer
- `"Hello " + name + "!"` → `"Hello #{name}!"`
- `str.length == 0` → `str.empty?`

QUOTES

LITERAL	INTERPOLATION	ESCAPES
`"..."`	yes	all
`'...'`	no	\' \\ only
`%Q(...)`	yes	all
`%q(...)`	no	minimal
`<<~TXT`	yes	all (sheet 021)

BUILDING STRINGS

CODE	EFFECT
`a + b`	new string
`a << b`	append, mutates a
`a.concat(b, c)`	append several
`a * 3`	repeat
`[a, b].join(", ")`	join with separator
`String.new(capacity: 1024)`	pre-sized buffer

⚠ COMMON MISTAKES

- Using single quotes and expecting `#{}` to interpolate
- Calling `<<` on a frozen literal and getting `FrozenError`; start from `+""` or `dup`
- Using `length` to size binary or network data; use `bytesize`

✓ TIPS & BEST PRACTICES

- Add `# frozen_string_literal: true` to every file; it catches accidental mutation and saves allocations
- Build big strings with `<<` or `Array#join`, not repeated `+`
- `str.dup` gives you an unfrozen copy of anything

QUICK SUMMARY

- `"..."` — interpolation + escapes
- `'...'` — literal
- `<<` / `+` — append / copy
- `length` / `bytesize` — chars / bytes

Interpolation & Formatting

#{} · format / sprintf · % operator · %05.2f · rjust / ljust / center · to_s(2) · inspect

INTERPOLATION AND FORMATTING

`"#{expr}"` evaluates any Ruby expression and calls `to_s` on it. For layout and number formatting use **`format`** (alias `sprintf`) or the `%` operator, which take printf-style specifiers. `rjust`, `ljust` and `center` pad strings for simple column alignment.

FORMATTING OUTPUT

```
name, score = "Ada", 93.456
"Hi #{name}, #{score.round(1)}%"   # "Hi Ada, 93.5%"
"Sum: #{1 + 2}"                    # "Sum: 3"

format("%.2f", score)              # "93.46"
format("%08.3f", score)            # "0093.456"
format("%5d|", 42)                 # "   42|"
format("%-5d|", 42)                # "42   |"
format("%+d", 5)                   # "+5"
format("%x %o %b", 255, 8, 5)      # "ff 10 101"
format("%s has %d", "box", 3)      # "box has 3"
format("%<x>s-%<y>s", x: 1, y: 2)  # "1-2"
"%05.1f" % 3.14159                 # "003.1"
"%s & %s" % ["a", "b"]             # "a & b"

"id".ljust(6, ".") + "|"           # "id....|"
"7".rjust(3, "0")                  # "007"
1234567.to_s.reverse.scan(/\d{1,3}/).join(",").reverse  # "1,234,567"
```

FORMAT NOTES

- `format` and `%` return strings; `printf` prints
- `%-10s` left-aligns, `%10s` right-aligns
- Named references: `format("%<n>05.1f", n: 3.14)`
- `to_s(2)` / `to_s(16)` format integers in other bases

CODE EXPLAINED

- → `#{}` — any expression; result goes through `to_s`
- → `%08.3f` — zero-pad to width 8, 3 decimals
- → `%-5d` — left-align in a 5-char field
- → `%<x>s` — named arguments from a hash
- → `%` with an array — multiple values

THE RUBY WAY

- `"Total: " + total.to_s` → `"Total: #{total}"`
- `x.to_s.rjust(3, "0")` for numbers → `format("%03d", x)`
- Hand-rolled decimal rounding for display → `format("%.2f", x)`

IN RAILS

- **AS** · `Time#to_fs(:db)` → `"2026-10-17 14:05:00"`
- **Rails** · `number_to_percentage(93.456, precision: 1)` → `"93.5%"`
- **Rails** · `number_to_human(1_234_567)` → `"1.23 Million"`

SPECIFIERS

SPEC	MEANING
`%s`	string (`to_s`)
`%p`	`inspect`
`%d`	integer
`%f` / `%.2f`	float / 2 decimals
`%e`	scientific
`%x` `%o` `%b`	hex / octal / binary
`%%`	literal %

FLAGS & WIDTH

SPEC	EXAMPLE
`%5s`	right-align width 5
`%-5s`	left-align
`%05d`	zero-pad: `00042`
`%+d`	always show sign
`% d`	space for positive

⚠ COMMON MISTAKES

- Writing `"#{name}"` inside single quotes; it prints literally
- Passing a single array to `%` when the format has one `%s`: `"%s" % [[1, 2]]` is needed to print the array itself
- Using `round(2)` and expecting trailing zeros; `3.10.round(2)` displays `3.1`. Use `format("%.2f")`

✓ TIPS & BEST PRACTICES

- `%p` inside `format` is handy for debug strings: `format("got %p", value)`
- `Integer#digits`, `to_s(2)`, and `format("%08b", n)` cover most binary display needs
- `inspect` in interpolation (`"#{x.inspect}"`) shows quotes and nils clearly

QUICK SUMMARY

- `"#{x}"` — interpolation
- `format` / `%` — printf-style
- `%.2f` / `%05d` — precision / padding
- `rjust` / `ljust` / `center` — simple alignment

String Methods — Search

include? · start_with? · end_with? · index / rindex · count · match? · =~ · scan

SEARCHING STRINGS

Ruby's string search methods are mostly **predicates** (`include?`, `start_with?`, `match?`) that return booleans, plus **locators** (`index`, `rindex`) that return a position or `nil`. Most accept either a plain string or a **Regexp**, so one method covers both literal and pattern searches.

SEARCH NOTES

- Everything is case-sensitive; use `casecmp?` or `/i`
- `index` returns `nil` when not found, not `-1`
- `match?` doesn't set `$~`, so it's the fastest regex test
- `start_with?` and `end_with?` accept multiple arguments

SEARCH METHODS

```ruby
s = "Hello, Ruby world"
s.include?("Ruby")            # true
s.start_with?("Hell")         # true
s.start_with?("Hi", "He")     # true (any)
s.end_with?("world")          # true
s.index("o")                  # 4
s.index("o", 5)               # 13 (search from 5)
s.rindex("o")                 # 13
s.index("zzz")                # nil

s.count("lo")                 # 5 (counts chars l or o)
s.scan("o").size              # 2
s.match?(/r\w+y/i)            # true
s =~ /Ruby/                   # 7
s[/w\w+/]                     # "world"
s.scan(/\b\w/)                # ["H", "R", "w"]

"RUBY".casecmp?("ruby")       # true
"b".between?("a", "c")        # true
```

CODE EXPLAINED

- → `include?` — substring test; returns a boolean
- → `start_with?("Hi", "He")` — true if any prefix matches
- → `index(str, start)` — first position at or after `start`
- → `count("lo")` — counts **characters** in the set, not the substring
- → `=~` — match position or `nil`; sets `$~`

THE RUBY WAY

- `s.index("x") != nil` → `s.include?("x")`
- `s =~ /re/ ? true : false` → `s.match?(/re/)`
- `s.downcase == t.downcase` → `s.casecmp?(t)`

IN RAILS

- **AS** · `exclude?` is the inverse of `include?`: `"abc".exclude?("z")` → `true`
- **AS** · `"production".inquiry.production?` → `true`
- **Rails** · that's how `Rails.env.production?` works

PREDICATES

METHOD	RETURNS
`include?(s)`	true / false
`start_with?(*s)`	true / false
`end_with?(*s)`	true / false
`match?(re)`	true / false
`casecmp?(s)`	case-insensitive ==
`empty?`	true / false

LOCATORS

METHOD	RETURNS
`index(s, from)`	position or nil
`rindex(s)`	last position
`s =~ re`	position or nil
`s[re]`	matched text or nil
`scan(re)`	array of all matches
`byteindex(s)`	byte position

⚠ COMMON MISTAKES

- Checking `index(x) > 0`; a match at position `0` fails the test, and `nil > 0` raises
- Using `count("ab")` to count occurrences of `"ab"`; use `scan("ab").size`
- Forgetting `/i` and missing `"ruby"` vs `"Ruby"`

✓ TIPS & BEST PRACTICES

- Use `match?` in conditions; it's faster than `=~` because it doesn't build MatchData
- `s[/(\d+)-(\d+)/, 2]` grabs one capture group directly
- `each_char.with_index.select { |c, _| c == "o" }.map(&:last)` finds all positions

QUICK SUMMARY

- `include?` / `start_with?` — substring tests
- `index` / `rindex` — position or nil
- `match?` — fast regex test
- `scan` — all matches

String Methods — Transform

upcase / downcase · capitalize · strip · sub / gsub · tr · squeeze · delete · bang ! variants

TRANSFORMING STRINGS

Transform methods return a **new string**; their **bang (!) variants** modify the receiver in place and return `nil` when nothing changed. `sub` replaces the first match and `gsub` replaces all, accepting a string, a hash, or a block for the replacement.

TRANSFORM NOTES

- `tr` maps characters one-to-one, like Unix `tr`
- `squeeze` collapses runs of repeated characters
- Bang methods return `nil` if no change was made; don't chain them

TRANSFORM METHODS

```
s = "  Hello Ruby  "
s.strip                          # "Hello Ruby"
s.lstrip / s.rstrip              # "Hello Ruby  " / "  Hello Ruby"
"ruby".upcase                    # "RUBY"
"RuBy".downcase                  # "ruby"
"ruby lang".capitalize           # "Ruby lang"

"a-b-c".sub("-", "+")            # "a+b-c"
"a-b-c".gsub("-", "+")           # "a+b+c"
"a1b2".gsub(/\d/) { |d| d.to_i * 2 }      # "a2b4"
"cat".gsub(/[ac]/, "a" => "4", "c" => "<")  # "<4t"
"hello".tr("el", "ip")           # "hippo"
"aaabbb".squeeze                 # "ab"
"hello".delete("l")              # "heo"
"chomp\n".chomp                  # "chomp"

name = +"ada"
name.capitalize!                 # "Ada"
name.capitalize!                 # nil (already capitalized)
```

CODE EXPLAINED

- → `sub` / `gsub` — first / all replacements
- → block form — compute each replacement from the match
- → hash form — map each matched string to its replacement
- → `tr("el", "ip")` — e→i, l→p, character by character
- → `capitalize!` — returns `nil` on no change, so `name.capitalize!.strip` can raise

THE RUBY WAY

- `s.gsub(" ", "")` → `s.delete(" ")`
- `s = s.strip` → `s.strip!` only when you own the string
- Chained `gsub` calls for single characters → one `tr`

IN RAILS

- AS · `" a b ".squish` → `"a b"` (strip + collapse spaces)
- AS · `"user_name".titleize` → `"User Name"`, `"Hello World".parameterize` → `"hello-world"`
- AS · `"post".pluralize` → `"posts"`, `"long text".truncate(6)` → `"lon..."`

CASE METHODS

CALL	RESULT
`"ruby".upcase`	"RUBY"
`"RUBY".downcase`	"ruby"
`"ruby on".capitalize`	"Ruby on"
`"Ab".swapcase`	"aB"

TRIM & CLEAN

CALL	RESULT
`" x ".strip`	"x"
`"x\n".chomp`	"x"
`"path/".chomp("/")`	"path"
`"a  b".squeeze(" ")`	"a b"
`"1,234".delete(",")`	"1234"
`"txt.rb".delete_suffix(".rb")`	"txt"

⚠ COMMON MISTAKES

- Writing `s.upcase!` and then using the return value; it's `nil` when the string was already uppercase
- Using `chop` to remove a newline; it removes the last character whatever it is. Use `chomp`

✓ TIPS & BEST PRACTICES

- `delete_prefix` / `delete_suffix` are clearer and faster than `sub(/\A.../, "")`
- `gsub` with a hash is a quick lookup-table replacer for templates
- `unicode_normalize` before comparing user text that may contain accents

QUICK SUMMARY

- `upcase` / `downcase` / `capitalize` — case
- `strip` / `chomp` — trim
- `sub` / `gsub` — replace
- `!` variants — in place, `nil` if unchanged

Indexing, Slicing & Splitting

INDEXING, SLICING AND SPLITTING

`str[...]` accepts an index, a start and length, a range, a substring, or a regex. Negative indexes count from the end. **`split`** breaks a string into an array, and **`chars`**, **`each_char`** and **`each_line`** walk through it piece by piece.

SLICING NOTES

- Indexes count **characters**, not bytes
- Out-of-range reads return `nil`, never raise
- `split` with no argument splits on whitespace and drops empties
- `str[i] = "x"` assigns into a mutable string

INDEX, SLICE, SPLIT

```
s = "Ruby on Rails"
s[0]                          # "R"
s[-1]                         # "s"
s[0, 4]                       # "Ruby" (start, length)
s[0..3]                       # "Ruby"
s[5..]                        # "on Rails" (endless range)
s[-5..]                       # "Rails"
s[100]                        # nil
s["on"]                       # "on" (or nil)
s[/R\w+$/]                    # "Rails"

"a,b,,c".split(",")           # ["a", "b", "", "c"]
"a b  c".split                # ["a", "b", "c"]
"a,b,c".split(",", 2)         # ["a", "b,c"]
"2026-10-17".split("-").map(&:to_i)  # [2026, 10, 17]
"hey".chars                   # ["h", "e", "y"]
"one\ntwo".lines              # ["one\n", "two"]
"one\ntwo".each_line(chomp: true) { |l| puts l }

t = +"cat"
t[0] = "b"                    # t == "bat"
```

CODE EXPLAINED

- → `s[0, 4]` — start and length
- → `s[5..]` / `s[-5..]` — endless ranges for "to the end"
- → `s["on"]` — returns the substring if present, else `nil`
- → `split(",", 2)` — limit the number of parts
- → `each_line(chomp: true)` — lines without trailing newlines

THE RUBY WAY

- `s[s.length - 1]` → `s[-1]`
- `s[3, s.length - 3]` → `s[3..]`
- `s.split("")` → `s.chars`

IN RAILS

- AS · `"Ruby".first(2)` → `"Ru"`, `"Ruby".last(2)` → `"by"`
- AS · `"hello".from(2)` → `"llo"`, `"hello".to(1)` → `"he"`
- AS · `"one two three".truncate_words(2)` → `"one two..."`

SLICE FORMS

FORM	EXAMPLE → RESULT
`[i]`	"ruby"[1] → "u"
`[start, len]`	"ruby"[1, 2] → "ub"
`[range]`	"ruby"[1..2] → "ub"
`[-n..]`	"ruby"[-2..] → "by"
`[str]`	"ruby"["by"] → "by"
`[regex, n]`	"a1b2"[/(\d)b(\d)/, 2] → "2"

SPLIT VARIANTS

CALL	RESULT
`"a b".split`	["a", "b"]
`"a-b".split("-")`	["a", "b"]
`"a1b22c".split(/\d+/)`	["a", "b", "c"]
`"a,b,".split(",")`	["a", "b"]
`"a,b,".split(",", -1)`	["a", "b", ""]

⚠ COMMON MISTAKES

- Expecting `split(",")` to keep trailing empty fields; pass `-1` as the limit for CSV-like data
- Confusing `s[1, 2]` (start + length) with `s[1..2]` (start..end)
- Slicing by byte offsets from `index` on multibyte text; use `byteslice` for byte positions

✓ TIPS & BEST PRACTICES

- `partition("=")` splits once into `[before, sep, after]`; great for `key=value`
- Use `each_char` instead of `chars` on huge strings to avoid building an array
- Real CSV has quoting rules; use the `csv` library (sheet 077) rather than `split(",")`

QUICK SUMMARY

- `s[i]` / `s[a, n]` / `s[a..b]` — index / length / range
- negative index — from the end
- `split` — to an array
- `chars` / `lines` — per char / per line

Heredocs, % Literals & Encoding

HEREDOCS, % LITERALS AND ENCODING

Heredocs hold multi-line text; the **squiggly** form <<~ strips the common leading indentation so the text can be indented with your code. **Percent literals** (%w, %q, %Q, %i) avoid escaping quotes. Every string carries an **encoding**, UTF-8 by default.

HEREDOC RULES

- `<<~TAG` strips indentation; `<<-TAG` only lets the closing tag be indented
- `<<~'TAG'` (quoted) disables interpolation
- You can call methods on the opener: `<<~SQL.strip`
- Any delimiter pair works for % literals: `%w[]` , `%w()` , `%w{}`

HEREDOCS AND % LITERALS

```ruby
def report(name, total)
  <<~TEXT
    Report for #{name}
      Total: #{total}
    Done.
  TEXT
end
report("Ada", 3)   # "Report for Ada\n  Total: 3\nDone.\n"

sql = <<~SQL.squeeze(" ").strip
  SELECT *  FROM users
  WHERE id = 1
SQL

%w[red green blue]        # ["red", "green", "blue"]
%i[get post]              # [:get, :post]
%q(It's "quoted")         # It's "quoted"  (no interpolation)
%Q(Hi #{1 + 1})           # "Hi 2"
"café".encoding           # #<Encoding:UTF-8>
```

CODE EXPLAINED

→ `<<~TEXT` — common indent removed; relative indent kept

→ `<<~SQL.squeeze(" ").strip` — methods apply to the whole heredoc

→ `%w[]` / `%i[]` — word / symbol arrays, no commas or quotes

→ `%q` / `%Q` — single / double-quote rules with free delimiters

THE RUBY WAY

- `"line1\n" + "line2\n"` → a `<<~TEXT` heredoc
- `["a", "b", "c"]` → `%w[a b c]`
- `"He said \"hi\""` → `%(He said "hi")`

IN RAILS

- **Rails** · ERB templates are where most multi-line HTML lives, not heredocs
- **AS** · `strip_heredoc` predates `<<~` ; seen in older Rails code

HEREDOC FORMS

OPENER	INDENT	INTERP.
`<<TAG`	none stripped	yes
`<<-TAG`	closing tag may indent	yes
`<<~TAG`	stripped	yes
`<<~'TAG'`	stripped	no

ENCODING METHODS

CALL	USE
`str.encoding`	current encoding
`str.valid_encoding?`	detect bad bytes
`str.force_encoding("UTF-8")`	relabel bytes
`str.encode("ISO-8859-1")`	convert bytes
`str.scrub("?")`	replace invalid bytes

⚠ COMMON MISTAKES

- Using `<<-` and expecting the body's indentation to be removed; only `<<~` does that
- Mixing tabs and spaces in a squiggly heredoc; indentation stripping then looks wrong
- Using `force_encoding` to convert text; it only relabels bytes. Use `encode` to convert

✓ TIPS & BEST PRACTICES

- Name heredoc tags after the content (`SQL` , `HTML` , `JSON`); editors highlight the body
- Use `scrub` on text from files or sockets before regex work to avoid `ArgumentError: invalid byte sequence`
- `%w[]` with interpolation is `%W[#{a} b]`

QUICK SUMMARY

- `<<~TAG` — indented heredoc
- `%w[]` / `%i[]` — word / symbol arrays
- `%q` / `%Q` — custom-quoted strings
- `encode` vs `force_encoding` — convert vs relabel

Arithmetic Operators

+ - * / % ** · integer vs float division · fdiv · divmod · negative modulo · precedence

ARITHMETIC OPERATORS

Arithmetic operators are **methods** on numeric objects: `a + b` is `a.+(b)`. The result type follows the operands: Integer with Integer stays Integer (division floors), and anything mixed with a Float becomes a Float. ** is exponentiation and binds tighter than unary minus.

ARITHMETIC RULES

- `/` on two integers floors toward −∞
- `%` takes the sign of the **divisor**
- `-2 ** 2` is `-4` (`**` before unary `-`)
- Division by integer `0` raises; by `0.0` gives `Infinity`

ARITHMETIC IN ACTION

```ruby
10 + 3                  # 13
10 - 3                  # 7
10 * 3                  # 30
10 / 3                  # 3
10 / 3.0                # 3.3333333333333335
10 % 3                  # 1
2 ** 10                 # 1024
2 ** -1                 # (1/2)  Rational
4 ** 0.5                # 2.0

-7 / 2                  # -4
-7 % 2                  # 1
7.remainder(-2)         # 1   (sign of dividend)
7.divmod(-2)            # [-4, -1]
10.fdiv(4)              # 2.5
-2 ** 2                 # -4
(-2) ** 2               # 4
1 / 0                   # ZeroDivisionError
1.0 / 0                 # Infinity
3.+(4)                  # 7 — operators are methods
```

CODE EXPLAINED

→ `10 / 3.0` — one float operand makes the result a float

→ `2 ** -1` — negative integer powers return a `Rational`

→ `-7 % 2` — sign follows the divisor, so result is `1`

→ `remainder` — C-style: sign follows the dividend

→ `3.+(4)` — every operator is a method call

THE RUBY WAY

- `(a.to_f / b)` → `a.fdiv(b)`
- `Math.sqrt(x)` for perfect squares → `Integer.sqrt(x)`
- `x = x + 1` → `x += 1` (Ruby has no `++`)

OPERATORS

OP	NAME	EXAMPLE
`+`	add	2 + 3 → 5
`-`	subtract	5 − 8 → -3
`*`	multiply	"ab" * 2 → "abab"
`/`	divide	7 / 2 → 3
`%`	modulo	−7 % 3 → 2
`**`	power	2 ** 3 → 8
`-@ / +@`	unary	−x

PRECEDENCE (HIGH → LOW)

LEVEL	OPERATORS
1	! ~ unary +
2	**
3	unary −
4	* / %
5	+ −
6	<< >>

⚠ COMMON MISTAKES

- Writing `x++` ; Ruby has no increment operator. Use `x += 1`
- Expecting `-7 % 3` to be `-1` as in C or JavaScript; Ruby gives `2`
- Computing averages as `sum / count` with integers and losing the fraction

✓ TIPS & BEST PRACTICES

- Use `divmod` when you need both quotient and remainder (time conversion, pagination)
- `Integer#pow(exp, mod)` is the fast way to do modular exponentiation
- Parenthesize negative bases: `(-2) ** 2`

QUICK SUMMARY

- `/` — floors for integers
- `%` — sign of divisor
- `**` — power, binds tighter than unary `-`
- `fdiv` — float division

Comparison & Spaceship

COMPARISON AND THE SPACESHIP

Comparison operators return `true` / `false`, except the **spaceship `<=>`**, which returns `-1`, `0`, `1`, or `nil` when values can't be compared. `<=>` is what sorting uses, and including **`Comparable`** turns one `<=>` method into `<`, `>`, `between?`, `clamp` and more.

COMPARISON NOTES

- `==` compares values; `!=` is defined as `!(a == b)`
- `1 == 1.0` is `true`; `1.eql?(1.0)` is `false`
- Strings compare byte-wise: `"Z" < "a"` is `true`
- `<=>` returns `nil` for unrelated types: `1 <=> "a"`

COMPARING VALUES

```ruby
5 == 5.0                  # true
5 != 6                    # true
3 < 5                     # true
"apple" < "banana"        # true
"Zebra" < "apple"         # true (uppercase sorts first)
[1, 2] == [1, 2]          # true (element-wise)
[1, 2] <=> [1, 3]         # -1

1 <=> 2                   # -1
2 <=> 2                   # 0
3 <=> 2                   # 1
1 <=> "a"                 # nil

5.between?(1, 10)         # true
15.clamp(1, 10)           # 10
-3.clamp(0..)             # 0
"b".clamp("a", "c")       # "b"

%w[pear Apple fig].sort  # ["Apple", "fig", "pear"]
%w[pear Apple fig].sort_by(&:downcase)  # ["Apple", "fig", "pear"]
[3, 1, 2].max             # 3
```

CODE EXPLAINED

- → `<=>` — -1 / 0 / 1 tells sort which comes first
- → `nil` — returned when the two values can't be ordered
- → `between?` / `clamp` — come from `Comparable`
- → `clamp(0..)` — clamp with an endless range
- → `sort_by(&:downcase)` — case-insensitive ordering

THE RUBY WAY

- `x >= 1 && x <= 10` → `x.between?(1, 10)`
- `[[x, lo].max, hi].min` → `x.clamp(lo, hi)`
- `sort { |a, b| a.age <=> b.age }` → `sort_by(&:age)`

OPERATORS

OP	MEANING
`== / !=`	equal / not equal
`< <= > >=`	ordering
`<=>`	-1, 0, 1 or nil
`===`	case equality (sheet 027)
`eql?`	value + type equality
`equal?`	same object

COMPARABLE GIVES YOU

METHOD	FROM <=>
`< <= > >=`	ordering operators
`==`	when <=> returns 0
`between?(a, b)`	inclusive range test
`clamp(a, b)`	limit to range
`sort / max / min`	via Enumerable

⚠ COMMON MISTAKES

- Sorting mixed types; `[1, "a"].sort` raises `ArgumentError` because `<=>` returns `nil`
- Forgetting that `"10" < "9"` is `true` for strings; convert numbers before comparing
- Defining `<` and `>` by hand instead of `<=>` plus `include Comparable`

✓ TIPS & BEST PRACTICES

- Compare multiple keys with arrays: `[a.last, a.first] <=> [b.last, b.first]`
- Use `casecmp?` for case-insensitive equality and `casecmp` for case-insensitive ordering
- `Float::NAN <=> 1` is `nil`, so NaN breaks sorts; filter it out first

QUICK SUMMARY

- `==` / `!=` — value equality
- `<=>` — three-way compare
- `Comparable` — free operators from `<=>`
- `clamp` / `between?` — range helpers

Logical Operators

LOGICAL OPERATORS

&&, || and ! work on truthiness and **short-circuit**: the right side runs only if needed. They return **one of the operands**, not necessarily `true` / `false`, which is why `x || default` works. The word forms `and`, `or`, `not` have **much lower precedence** and are meant for control flow, not expressions.

LOGIC RULES

- `a && b` returns `a` if `a` is falsy, otherwise `b`
- `a || b` returns `a` if `a` is truthy, otherwise `b`
- `&&` binds tighter than `||`
- `and` / `or` bind looser than `=`, which causes bugs

&& || ! AND PRECEDENCE

```ruby
true && "yes"              # "yes"
nil && "yes"               # nil
nil || "default"           # "default"
"set" || "default"         # "set"
!nil                       # true
!"text"                    # false

name = params[:name] || "Guest"
user && user.admin?        # safe even if user is nil
@config ||= load_config    # assign once
count &&= count + 1        # only if count is truthy

x = true and false         # x is true!  (= runs first)
x = (true and false)       # x is false
x = true && false          # x is false

file = File.open(path) or raise "can't open"
save or return             # control-flow style
not true                   # false
```

CODE EXPLAINED

→ `||` default — returns the first truthy value

→ `&&` guard — stops at `nil` before calling `admin?`

→ `||=` / `&&=` — conditional assignment

→ `x = true and false` — `=` binds tighter than `and`

→ `or raise` — idiomatic "do this or bail out"

THE RUBY WAY

- `if a == true && b == true` → `if a && b`
- `x = y and z` in assignments → `x = y && z`
- `if !(x.nil?)` → `if x` or `unless x.nil?`

TRUTH TABLE

A	B	A && B	A || B
true	true	true	true
true	nil	nil	true
nil	true	nil	true
false	nil	false	nil

OPERATOR FORMS

SYMBOL	WORD	PRECEDENCE
&&	and	high / very low
||	or	high / very low
!	not	highest / low
? :	—	below ||

⚠ COMMON MISTAKES

- Using `and` / `or` in assignments and conditions; precedence below `=` gives surprising results
- Relying on `||` for defaults when `false` is a valid value; `false || true` is `true`
- Expecting `&&` to return `true`; it returns the last evaluated operand

✓ TIPS & BEST PRACTICES

- Reserve `and` / `or` for control flow like `do_it or raise`; RuboCop flags other uses
- Use `x.nil? ? default : x` (or `fetch`) when `false` must be preserved
- Short-circuiting means expensive checks belong on the right side

QUICK SUMMARY

- `&&` / `||` — short-circuit, return an operand
- `!` — negate truthiness
- `||=` / `&&=` — conditional assignment
- `and` / `or` — low precedence, control flow only

Bitwise Operators

BITWISE OPERATORS

Integers support bitwise AND, OR, XOR, NOT and shifts, operating on **two's-complement** bit patterns of any size. Because Ruby integers never overflow, left shifts grow the number instead of wrapping. `Integer#[]` reads single bits, and `to_s(2)` shows the pattern.

BIT NOTES

- `~n` equals `-n - 1` (two's complement)
- `>>` on negatives fills with sign bits (arithmetic shift)
- `bit_length` is bits needed, excluding the sign

BIT OPERATIONS

```
a = 0b1100                # 12
b = 0b1010                # 10
a & b                     # 8   (0b1000)
a | b                     # 14  (0b1110)
a ^ b                     # 6   (0b0110)
~a                        # -13
a << 2                    # 48  (0b110000)
a >> 2                    # 3   (0b11)

12.to_s(2)                # "1100"
format("%08b", 12)        # "00001100"
12[2]                     # 1  (bit 2 is set)
12[0]                     # 0
255.bit_length            # 8
0b1111.allbits?(0b0101) # true
0b1000.anybits?(0b1100) # true

READ, WRITE, EXEC = 1, 2, 4
perms = READ | WRITE      # 3
perms & WRITE != 0        # true
perms &= ~WRITE           # 1 (clear WRITE)
```

CODE EXPLAINED

- → `&` `|` `^` — AND, OR, XOR per bit
- → `~a` — invert all bits (`-a - 1`)
- → `<<` — grows without overflow
- → `12[2]` — read bit 2 directly
- → `allbits?` / `anybits?` / `nobits?` — mask tests

THE RUBY WAY

- `(n & mask) == mask` → `n.allbits?(mask)`
- `(n >> i) & 1` → `n[i]`
- `n.to_s(2).rjust(8, "0")` → `format("%08b", n)`

OPERATORS

OP	NAME	12 OP 10
`&`	AND	8
`|`	OR	14
`^`	XOR	6
`~`	NOT	~12 → -13
`<<`	left shift	12 << 1 → 24
`>>`	right shift	12 >> 1 → 6

WORKED EXAMPLE

BITS	VALUE
`1100`	12
`1010`	10
`1000 (&)`	8
`0110 (^)`	6

⚠ COMMON MISTAKES

- Expecting 32-bit wrap-around from C or Java; `1 << 40` is just a big number. Mask with `& 0xFFFF_FFFF`
- Printing `~n` and expecting an unsigned value; `~12` is `-13`. Mask it: `~12 & 0xFF` → `243`
- Using `^` as exponent; it's XOR. Power is `**`

✓ TIPS & BEST PRACTICES

- For a set of named flags, consider `Set` of symbols; use bits when storing or sending compactly
- `Integer("0b1010")` and `"1010".to_i(2)` both parse binary strings
- See the Bitwise set for algorithms (popcount, masks, subsets)

QUICK SUMMARY

- `&` `\|` `^` `~` — bit logic
- `<<` `>>` — shifts, no overflow
- `n[i]` — read a bit
- `allbits?` / `anybits?` — mask tests

Sheet 026

Assignment & Splat Operators

+= -= · ||= · parallel assignment · *splat · **double splat · nested destructuring · *nil (4.0)

ASSIGNMENT AND SPLAT

Ruby's assignment goes beyond =: **compound operators** (+=, ||=), **parallel assignment** (`a, b = 1, 2`), and **splats** (*, **) that collect or spread arrays and hashes. Destructuring works in assignment, block parameters and method calls.

ASSIGNMENT NOTES

- `a += 1` expands to `a = a + 1`
- `a ||= x` is roughly `a || a = x`
- `*` collects into an array; `**` into a hash
- Ruby 4.0: `*nil` no longer calls `nil.to_a`; it splats to nothing directly

COMPOUND, PARALLEL, SPLAT

```ruby
n = 10
n += 5; n -= 3; n *= 2    # 24
n /= 4                    # 6
n **= 2                   # 36
s = +"a"; s << "b"        # "ab"

a, b, c = 1, 2            # a=1 b=2 c=nil
first, *rest = [1, 2, 3]        # 1, [2, 3]
*init, last = [1, 2, 3]         # [1, 2], 3
a, (b, c), d = 1, [2, 3], 4     # nested
_, second = ["skip", "keep"]    # ignore with _

nums = [1, 2, 3]
[0, *nums, 4]             # [0, 1, 2, 3, 4]
def add(a, b, c) = a + b + c
add(*nums)                # 6
opts = { color: "red" }
{ size: 2, **opts }       # {size: 2, color: "red"}
def paint(color:) = color
paint(**opts)             # "red"
[*1..3, *"a"]             # [1, 2, 3, "a"]
```

CODE EXPLAINED

- → `a, b, c = 1, 2` — missing values become `nil`
- → `*rest` / `*init` — splat can sit anywhere, once
- → `(b, c)` — nested destructuring
- → `*nums` in a call — spreads array into positional args
- → `**opts` — spreads a hash into keywords or another hash

THE RUBY WAY

- `arr[0], arr[1..]` → `first, *rest = arr`
- `a.merge(b)` for literals → `{ **a, **b }`
- `method(arr[0], arr[1])` → `method(*arr)`

COMPOUND OPERATORS

OP	EXPANDS TO
`+= -= *= /=`	a = a op b
`%= **=`	a = a op b
`||=`	assign if nil / false
`&&=`	assign if truthy
`<<= >>= &= |= ^=`	bitwise forms

SPLAT USES

CONTEXT	EXAMPLE
collect args	def f(*args)
spread args	f(*list)
collect keywords	def f(**opts)
spread keywords	f(**opts)
array literal	[*a, *b]
hash literal	{ **h, x: 1 }

⚠ COMMON MISTAKES

- Expecting `a, b = [1, 2, 3]` to fail; extra values are silently dropped
- Using `||=` with hash entries that may hold `false`; the `false` gets overwritten
- Writing two splats in one assignment (`*a, *b = ...`); only one splat is allowed per side

✓ TIPS & BEST PRACTICES

- Use `_` (or `_name`) for values you deliberately ignore; Ruby won't warn about it
- Block params destructure too: `pairs.each { |(k, v)| ... }`
- `{ **defaults, **options }` gives you a merged options hash with later keys winning

QUICK SUMMARY

- `+=` etc. — compound assignment
- `a, b = b, a` — parallel assignment
- `*` / `**` — collect or spread
- `_` — ignore a value

Special Operators

SPECIAL OPERATORS

Some of Ruby's most-used operators don't fit the usual categories: **===** (case equality, used by `case/when`), **=~** (regex match), **&.** (safe navigation), **.. / ...** (ranges), **::** (constant lookup), **defined?** and **->** (lambda literal).

NOTES

- `===` is a method; each class defines what "matches" means
- `..` includes the end, `...` excludes it
- `defined?` returns a description string or `nil`
- `::` with nothing on the left means top-level: `::File`

=== =~ &. .. :: DEFINED? ->

```ruby
Integer === 5            # true  (is-a)
(1..10) === 5            # true  (cover)
/ab/ === "cab"           # true  (match)

"price: 42" =~ /\d+/     # 7
$~[0]                    # "42"
"no" !~ /\d/             # true

user&.email              # nil if user is nil
(1..5).to_a              # [1, 2, 3, 4, 5]
(1...5).to_a             # [1, 2, 3, 4]

Math::PI                 # 3.141592653589793
::File                   # top-level constant

defined?(foo)            # nil
defined?(puts)           # "method"
defined?(String)         # "expression"
square = ->(x) { x * x }
square.(4)               # 16
```

CODE EXPLAINED

- → `===` — "does the right side belong to the left side?"
- → `=~` — match position or `nil`; sets `$~`
- → `!~` — true when there's no match
- → `..` / `...` — inclusive / exclusive range
- → `->(x) { }` — lambda literal; call with `.()` or `.call`

THE RUBY WAY

- `if x.is_a?(Integer) || x.is_a?(Float)` → `case x when Integer, Float`
- `if user != nil && user.email` → `user&.email`
- `lambda { |x| ... }` → `->(x) { ... }`

OPERATORS

OP	MEANS
`===`	case equality
`=~ / !~`	regex match / no match
`&.`	call unless nil
`.. / ...`	inclusive / exclusive range
`::`	constant / namespace lookup
`->`	lambda literal
`defined?`	what is this name?

WHAT === MEANS

LEFT SIDE	TRUE WHEN
`Class`	right is an instance
`Range`	right is covered
`Regexp`	right string matches
`Proc`	proc returns truthy
`other object`	same as ==

⚠ COMMON MISTAKES

- Flipping `===`: `5 === Integer` is `false`; the pattern always goes on the left
- Using `defined?(@var)` to test for `nil`; it only tells you whether the variable exists
- Writing `1..5.each`; `..` has low precedence, so wrap it: `(1..5).each`

✓ TIPS & BEST PRACTICES

- Lambdas work in `when`: `when ->(n) { n > 100 } then "big"`
- Use `::` only for constants and nested modules; use `.` to call methods
- `defined?(yield)` (or `block_given?`) checks whether a block was passed

QUICK SUMMARY

- `===` — case equality
- `=~` — regex match position
- `&.` — safe call
- `..` / `...` / `::` / `->` — range / scope / lambda

if, elsif, else & unless

IF, ELSIF, ELSE, UNLESS

Conditionals test **truthiness** (only `nil` and `false` are falsy). `if` is an **expression** that returns the value of the branch that ran, so you can assign its result. `unless` means "if not" and reads well for a single negative condition.

CONDITIONAL NOTES

- It's `elsif`, not `elseif` or `else if`
- `then` lets a branch fit on one line
- An `if` with no matching branch returns `nil`
- Avoid `unless ... else`; flip it to `if`

CONDITIONALS

```ruby
temp = 72
if temp > 85
  puts "hot"
elsif temp > 60
  puts "nice"             # nice
else
  puts "cold"
end

status = if temp > 60 then "warm" else "cool" end  # "warm"

unless logged_in?
  redirect_to_login
end

grade =
  if    score >= 90 then "A"
  elsif score >= 80 then "B"
  else                   "C"
  end

if (m = line.match(/(\d+)/))   # assign + test
  puts m[1]
end
```

CODE EXPLAINED

→ `elsif` — the only spelling Ruby accepts

→ `status = if ...` — `if` returns the chosen branch's value

→ `unless` — runs when the condition is falsy

→ `then` — single-line branches

→ `if (m = ...)` — parentheses mark intentional assignment

THE RUBY WAY

- `if !valid?` → `unless valid?`
- `x = nil; if c then x = 1 else x = 2 end` → `x = if c then 1 else 2 end`
- Deep nesting → guard clauses: `return unless valid?` (sheet 029)

FORMS

FORM	USE
`if / elsif / else`	multi-branch
`unless`	single negative test
`if c then a else b end`	one-liner
`a if c`	modifier (sheet 029)
`c ? a : b`	ternary (sheet 029)
`case / when`	many values (sheet 030)

RETURN VALUES

CODE	RESULT
`if true then 1 end`	1
`if false then 1 end`	nil
`if nil then 1 else 2 end`	2
`unless false then :ok end`	:ok

⚠ COMMON MISTAKES

- Writing `else if`; Ruby parses it as a nested `if` that needs its own `end`
- Using `unless` with `else` or with `&&` / `\|\|` conditions; double negatives are hard to read
- Writing `if x = 5` by accident; Ruby warns, and the condition is always truthy

✓ TIPS & BEST PRACTICES

- Prefer `case` once you have more than two `elsif` branches comparing one value
- Return early from methods instead of wrapping the whole body in `if`
- Assigning from `if` keeps the variable defined in one place

QUICK SUMMARY

- `if / elsif / else` — branches
- `unless` — if not
- `if` is an expression — assign its result
- `then` — one-line branches

Modifiers & Ternary

if / unless modifiers · if as expression · ? : · leading && / || continuation (4.0) · guard clauses

MODIFIERS, TERNARY, GUARD CLAUSES

Short conditions can go **after** a statement as **modifiers**: `do_it if ready`. The **ternary** `cond ? a : b` picks one of two values. **Guard clauses** return early at the top of a method so the main logic isn't nested. Ruby 4.0 also lets a line **start** with && or `\|\|` to continue the previous condition.

NOTES

- Modifiers work with `if`, `unless`, `while`, `until`, `rescue`
- A modifier `if` returns `nil` when the condition is false
- Nested ternaries are legal but hard to read
- Leading `&&` / `\|\|` line continuation is new in 4.0

MODIFIERS AND GUARDS

```ruby
puts "ready" if ready
save unless dirty?
retries += 1 while retries < 3

label = count == 1 ? "item" : "items"
"#{count} #{count == 1 ? 'item' : 'items'}"

def ship(order)
  return :empty   if order.items.empty?   # guard clauses
  return :blocked unless order.paid?
  raise ArgumentError, "no address" if order.address.nil?

  order.ship!                             # main path, unnested
  :shipped
end

valid = user.active?
  && user.verified?           # Ruby 4.0: leading &&
  || user.admin?

value = Integer(input) rescue 0   # rescue modifier (use sparingly)
```

CODE EXPLAINED

→ `puts "ready" if ready` — statement first, condition after

→ `count == 1 ? "item" : "items"` — ternary returns one value

→ `return ... if ...` — guard clauses handle edge cases first

→ leading `&&` — continue a condition on the next line (4.0+)

→ `rescue 0` — inline fallback; it rescues every `StandardError`

THE RUBY WAY

- `if ready then puts "ready" end` → `puts "ready" if ready`
- Nested `if` around a method body → guard clauses at the top
- `a ? (b ? 1 : 2) : 3` → a `case` or `if/elsif`

MODIFIER FORMS

FORM	RUNS WHEN
`x if c`	c is truthy
`x unless c`	c is falsy
`x while c`	repeat while truthy
`x until c`	repeat until truthy

TERNARY VS IF

USE	CHOOSE
`pick between two values`	ternary
`side effects in branches`	if / else
`3+ outcomes`	case / if-elsif
`long expressions`	if / else

⚠ COMMON MISTAKES

- Writing long statements before a modifier; readers miss the condition at the end of the line
- Using `rescue` modifiers to hide real errors; it swallows every `StandardError`, including typos
- Running Ruby 4.0-style leading `&&` lines on 3.x; it's a syntax error there. Use trailing `&&` for older versions

✓ TIPS & BEST PRACTICES

- One guard clause per edge case reads like a checklist of preconditions
- Keep ternaries on one line with simple operands; otherwise use `if`
- Prefer `Integer(input, exception: false) \|\| 0` over `rescue 0`

QUICK SUMMARY

- `x if c` — modifier condition
- `c ? a : b` — ternary
- `return x if c` — guard clause
- leading `&&` / `\|\|` — 4.0 line continuation

case / when

CASE / WHEN

`case` compares one value against several patterns using **`===`**, so a `when` can match a value, a class, a range, a regex, or a lambda. The first matching branch runs and its value is returned; there is **no fall-through**, so no `break` is needed. Without a subject, `case` acts like an `if/elsif` chain.

CASE NOTES

- `when a, b` matches either value
- `else` handles no match; otherwise the result is `nil`
- `case/in` is pattern matching (sheets 031–032), a different feature

CASE / WHEN

```ruby
def classify(x)
  case x
  when nil             then "nothing"
  when Integer, Float  then "number"
  when /\A\d+\z/       then "numeric string"
  when String          then "text"
  else "other"
  end
end
classify("42")       # "numeric string"

grade = case score
        when 90.. then "A"
        when 80...90 then "B"
        when ->(s) { s < 0 } then "invalid"
        else "C"
        end

case                  # no subject: if/elsif style
when temp > 30 then :hot
when temp < 5  then :cold
else :mild
end
```

CODE EXPLAINED

→ `/\A\d+\z/` — regex `===` tests a match

→ `90..` — endless range catches 90 and up

→ `->(s) { s < 0 }` — a lambda's `===` calls it

→ no subject — each `when` is a plain condition

THE RUBY WAY

- `if x == :a \|\| x == :b` → `case x when :a, :b`
- `when x > 5` with a subject → subjectless `case` (or a lambda)
- Hash lookup tables for pure value maps → `{ a: 1 }.fetch(x)` when no logic is needed

WHAT A WHEN CAN HOLD

PATTERN	MATCHES
`42 / "str" / :sym`	equal value
`Integer`	instances of the class
`1..10`	values in range
`/regex/`	matching strings
`->(x) { }`	lambda returns truthy

CASE VS OTHERS

NEED	USE
`one value, many options`	case / when
`destructure data`	case / in
`value → value map`	Hash#fetch

⚠ COMMON MISTAKES

- Writing `when x > 5` in a `case x`; that compares `x` to `true` / `false`. Drop the subject or use a lambda
- Adding `break` out of habit from C or JavaScript; there's no fall-through to stop
- Ordering `when String` before a more specific regex; the first match wins

✓ TIPS & BEST PRACTICES

- `when *VALID_STATES` splats an array of options into the branch
- `case` returns a value, so assign it directly instead of setting a variable in each branch
- Put `nil` checks first so later `when`s can assume a value

QUICK SUMMARY

- `case x when ...` — compares with `===`
- `when a, b` — either
- no fall-through — no `break` needed
- subjectless case — if/elsif replacement

Pattern Matching — Basics

PATTERN MATCHING BASICS

`case/in` matches the **shape** of data and binds variables in one step. Array patterns check length and elements, hash patterns check keys, and plain values or classes are compared with ===. Unlike `case/when`, a `case/in` with no matching branch and no `else` raises **`NoMatchingPatternError`**.

BASICS

- Hash patterns match **symbol keys** and ignore extra keys
- Array patterns must match the **whole** length (use `*rest` for more)
- Bare names bind values: `in [x, y]`
- `=>` (rightward assignment) destructures or raises

CASE / IN

```ruby
def describe(data)
  case data
  in []                    then "empty"
  in [Integer => a, Integer => b] then "pair #{a + b}"
  in [first, *rest]        then "#{first} +#{rest.size}"
  in { type: "user", name: String => name } then "user #{name}"
  in { status: 200 | 201 } then "ok"
  in nil                   then "nil"
  end
end
describe([1, 2])                 # "pair 3"
describe(["a", 2, 3])            # "a +2"
describe({ type: "user", name: "Ada", id: 1 })  # "user Ada"
describe("?")                    # NoMatchingPatternError

config = { db: { host: "localhost", port: 5432 } }
config => { db: { host:, port: } }   # binds host, port
port                                 # 5432
```

CODE EXPLAINED

- → `Integer => a` — check with `===`, then bind to `a`
- → `[first, *rest]` — any length ≥ 1
- → `{ type: "user" }` — key must exist and match; extra keys OK
- → `host:` — shorthand binds the value to a local named `host`

THE RUBY WAY

- `if h[:type] == "user" && h[:name].is_a?(String)` → `in { type: "user", name: String }`
- `a = arr[0]; b = arr[1]` checks → `case arr in [a, b]`
- `h.dig(:db, :host)` plus `nil` checks → `h => { db: { host: } }`

PATTERN KINDS

PATTERN	MATCHES
`42 / "x" / :ok`	value via ===
`Integer`	class via ===
`1..5`	range via ===
`[a, b]`	array of length 2
`[a, *]`	array of length ≥ 1
`{ k: v }`	hash with key `:k`

CASE/IN VS CASE/WHEN

FEATURE	WHEN	IN
binds variables	no	yes
destructures	no	yes
no match	nil	raises

⚠ COMMON MISTAKES

- Expecting string-keyed hashes to match `{ name: }`; hash patterns only match symbol keys. Use `JSON.parse(s, symbolize_names: true)`
- Forgetting `else` and getting `NoMatchingPatternError` in production
- Writing `in x` as a catch-all early; a bare name matches everything, so later branches never run

✓ TIPS & BEST PRACTICES

- Use `case/in` for JSON-like data, API responses, and tuples; keep `case/when` for simple values
- `**nil` in a hash pattern requires no other keys: `in { id:, **nil }`
- `in [Integer, Integer] => pair` binds the whole match as well as checking it

QUICK SUMMARY

- `case x in pattern` — match shape + bind
- `[a, *rest]` — array pattern
- `{ key: }` — hash pattern + binding
- `=>` — destructure or raise

Pattern Matching — Advanced

PATTERN MATCHING ADVANCED

Beyond basic shapes, patterns support **find patterns** (`[*, x, *]`), the **pin operator** (^) for comparing against an existing value, **guards** (`if` / `unless`), and **alternatives** (`\|`). Your own classes join in by defining **`deconstruct`** (array patterns) and **`deconstruct_keys`** (hash patterns).

ADVANCED NOTES

- `^name` compares with a local; `^(expr)` with any expression
- Guards run after the pattern matches
- `x in pattern` returns `true` / `false` and never raises

FIND, PIN, GUARDS, CUSTOM CLASSES

```
case [1, 42, "x", 7]
in [*, String => s, *] then "found #{s}"   # "found x"
end

expected = 5
case 5
in ^expected then "match"                  # pin, don't rebind
end
case order
in { total: Integer => t } if t > 100 then "big"   # guard
in { total: }                         then "normal"
end

Point = Struct.new(:x, :y)
case Point.new(0, 3)
in [0, y] then "on y-axis at #{y}"         # deconstruct
end
class Temp
  def initialize(c) = @c = c
  def deconstruct_keys(keys) = { celsius: @c }
end
Temp.new(30) in { celsius: 25.. }          # true
```

CODE EXPLAINED

→ `^expected` — compare, don't rebind

→ `if t > 100` — guard filters after matching

→ `deconstruct_keys(keys)` — return a hash for hash patterns

THE RUBY WAY

- `arr.find { |e| e.is_a?(String) }` inside a `case` → `in [*, String => s, *]`
- Re-checking a bound value in the branch body → a guard `if ...`
- `respond_to?` checks on custom objects → define `deconstruct_keys`

OPERATORS

SYNTAX	MEANING
`^var`	pin existing value
`pat if cond`	guard
`p1 | p2`	alternative
`pat => name`	bind whole match
`[*, x, *]`	find pattern

ONE-LINE FORMS

FORM	ON NO MATCH
`x in pattern`	returns false
`x => pattern`	NoMatchingPatternError
`case x in p … else … end`	else branch

⚠ COMMON MISTAKES

- Forgetting `^` and accidentally rebinding: `in expected` matches anything and overwrites `expected`
- Using alternatives with bindings: `in [x] \| [x, _]` is a syntax error; alternatives can't bind names
- Returning all fields from `deconstruct_keys` on hot paths; use the `keys` argument to compute only what's asked

✓ TIPS & BEST PRACTICES

- `deconstruct_keys(keys)` receives `nil` when the pattern uses `**rest`; return everything then
- `Data.define` value objects work with both array and hash patterns for free
- Use `in` as a boolean filter: `events.select { it in { type: :click } }`

QUICK SUMMARY

- `[*, x, *]` — find pattern
- `^var` — pin
- `if` / `unless` — guards
- `deconstruct` / `deconstruct_keys` — custom matching

while, until & loop

WHILE, UNTIL AND LOOP

`while` repeats while a condition is truthy and `until` repeats until it becomes truthy. **`loop do ... end`** runs forever until `break`, and it quietly stops when an enumerator raises `StopIteration`. In everyday Ruby these are less common than iterators like `each` and `times` (sheet 034).

LOOP NOTES

- `while` / `until` don't create a new variable scope
- `loop` **does** (it takes a block)
- `begin ... end while cond` runs the body at least once
- `loop` returns the enumerator's result on `StopIteration`

CONDITION LOOPS

```ruby
i = 0
while i < 3
  print i                  # 012
  i += 1
end

n = 10
n -= 3 until n <= 0        # n == -2

begin
  input = gets&.chomp
end while input&.empty?    # do-while: body runs first

loop do
  line = $stdin.gets
  break if line.nil? || line.chomp == "quit"
  puts line.upcase
end

e = [1, 2].each
loop { puts e.next }       # 1, 2, then StopIteration ends it
```

CODE EXPLAINED

- → `while i < 3` — checks before each pass
- → `begin ... end while` — post-test loop (runs at least once)
- → `loop do` — infinite loop; leave with `break`
- → `e.next` — `loop` rescues `StopIteration` for you

THE RUBY WAY

- `while i < arr.size; arr[i]; i += 1; end` → `arr.each`
- `while true` → `loop do`
- `while !done` → `until done`

LOOP FORMS

FORM	USE
`while cond`	repeat while truthy
`until cond`	repeat while falsy
`loop do … end`	forever, `break` out
`begin … end while`	at least once

SCOPE DIFFERENCE

LOOP	NEW SCOPE?
`while / until`	no
`loop do`	yes
`each / times`	yes

⚠ COMMON MISTAKES

- Forgetting to update the counter in a `while` loop and hanging the program
- Expecting variables first assigned inside `loop do` to exist after the loop; they're block-local
- Using `while` to walk arrays by index; off-by-one errors are easy and `each` avoids them

✓ TIPS & BEST PRACTICES

- `loop` + external enumerators (`next`) is a clean way to read paired streams
- Put the exit condition near the top of `loop do` with `break if`
- For retries with limits, combine `loop` with a counter or use `retry` (sheet 039)

QUICK SUMMARY

- `while` / `until` — condition loops
- `loop do` — infinite loop + `break`
- `begin … end while` — do-while
- prefer iterators for collections

Iterating: each, times & for

each · times · upto / downto · step · each_with_index · for…in (and why it's rare)

ITERATING WITH EACH, TIMES, STEP

Ruby code loops with **iterator methods** that take a block rather than with counters. `each` walks a collection, `times` repeats, `upto` / `downto` / `step` count, and `each_with_index` adds a position. A `for ... in` loop exists, but it leaks its variable and is rarely used.

ITERATOR NOTES

- Every iterator returns its receiver (or an enumerator without a block)
- Block variables are local to the block
- `for x in list` is sugar for `each` without the new scope
- `with_index(1)` starts counting at 1

ITERATION METHODS

```ruby
3.times { |i| print i }           # 012
1.upto(4) { |i| print i }         # 1234
5.downto(3) { |i| print i }       # 543
0.step(10, 5) { |i| print i, " " }   # 0 5 10
1.0.step(2.0, 0.5).to_a           # [1.0, 1.5, 2.0]
(1..9).step(4).to_a               # [1, 5, 9]

%w[a b c].each { |x| print x }    # abc
%w[a b c].each_with_index { |x, i| puts "#{i}:#{x}" }
%w[a b c].each.with_index(1) { |x, i| puts "#{i}. #{x}" }
{ a: 1, b: 2 }.each { |k, v| puts "#{k}=#{v}" }
[[1, 2], [3, 4]].each { |(a, b)| print a + b, " " }  # 3 7

for x in [1, 2]                   # legal, but…
end
x                                 # 2 — leaks out of the loop
```

CODE EXPLAINED

- → `3.times` — yields 0, 1, 2
- → `0.step(10, 5)` — start, limit, increment
- → `each_with_index` — element, then index
- → `each.with_index(1)` — index offset for numbered lists
- → `for x in` — `x` survives after the loop

THE RUBY WAY

- `for i in 0...arr.size` → `arr.each_with_index`
- `i = 0; arr.each { ...; i += 1 }` → `each_with_index`
- `(0...n).each` → `n.times`

COUNTING

CALL	YIELDS
`n.times`	0 … n−1
`a.upto(b)`	a … b
`b.downto(a)`	b … a
`a.step(b, s)`	a, a+s … ≤ b
`(a..b).step(s)`	same via range
`(a..b).each`	a … b

COLLECTIONS

CALL	YIELDS
`arr.each`	element
`arr.each_with_index`	element, index
`hash.each`	key, value
`hash.each_with_index`	[k, v], index
`str.each_char`	character
`arr.each_slice(2)`	groups of 2

⚠ COMMON MISTAKES

- Using `for` and then reusing the loop variable name later; it still holds the last value
- Reversing block params on `each_with_index` (`|i, x|`); the element comes first
- Calling `each` when you want a new array; `each` returns the original. Use `map`

✓ TIPS & BEST PRACTICES

- Call an iterator without a block to get an Enumerator: `arr.each_with_index.map { |x, i| ... }`
- `Integer#step` with floats avoids accumulating rounding error that `+=` loops have
- Prefer `each` over `for` ; it's the idiom every Ruby reader expects

QUICK SUMMARY

- `times` / `upto` / `step` — counting
- `each` — walk a collection
- `each_with_index` / `with_index(1)` — positions
- `for` — legal, leaks its variable

Loop Controls

LOOP CONTROLS

Inside loops and blocks, **next** skips to the next iteration, **break** exits the loop (optionally with a value that becomes the loop's result), and **redo** repeats the current pass. **throw / catch** exit several levels of nesting at once. In a block, `return` returns from the **enclosing method**, not just the block.

CONTROL NOTES

- `break value` makes the iterator return `value`
- `next value` sets the block's result for that element (useful in `map`)
- Ruby has no labeled `break`; use `throw` / `catch` or a method `return`

BREAK, NEXT, THROW/CATCH, RETURN

```ruby
[1, 2, 3, 4].each do |n|
  next if n.even?            # skip evens
  print n                    # 13
end

found = [5, 8, 12].each { |n| break n if n > 6 }   # 8
[1, 2, 3].map { |n| next 0 if n == 2; n * 10 }    # [10, 0, 30]

result = catch(:done) do
  (1..3).each do |i|
    (1..3).each { |j| throw :done, [i, j] if i * j == 6 }
  end
  nil
end
result                       # [2, 3]

def first_negative(nums)
  nums.each { |n| return n if n.negative? }  # leaves the method
  nil
end
```

CODE EXPLAINED

- → `next if` — skip the rest of this iteration
- → `break n` — stop and make `each` return `n`
- → `next 0` — the block's value for this element
- → `throw :done, value` — unwinds to the matching `catch`
- → `return` in a block — exits `first_negative` itself

THE RUBY WAY

- `break` loops to search → `find { |n| n > 6 }`
- Flag variables to escape nested loops → `catch` / `throw` or extract a method
- `next unless cond` around the whole body → `select` or `filter_map`

KEYWORDS

KEYWORD	EFFECT
`next`	skip to next iteration
`next v`	block returns v for this item
`break`	exit loop, returns nil
`break v`	exit loop, returns v
`return`	exit the enclosing method

WHERE THEY WORK

CONSTRUCT	BREAK / NEXT
`while / until`	yes
`each / map / times`	yes
`proc`	break raises LocalJumpError
`lambda`	return / next leave the lambda

⚠ COMMON MISTAKES

- Using `return` inside a block passed to a method defined elsewhere; you exit your own method, which surprises readers
- Using `break` inside a `proc` that outlives its method; it raises `LocalJumpError`
- Writing `break` in `map` and expecting a partial array; `break` returns `nil` (or its value) instead

✓ TIPS & BEST PRACTICES

- Most explicit loops with `break` are really `find`, `any?`, `take_while` or `detect`
- `next value` in `map` is cleaner than wrapping the block body in `if/else`
- `throw` / `catch` is for rare deep exits; usually extracting a method and using `return` reads better

QUICK SUMMARY

- `next` — skip
- `break v` — exit with a value
- `throw` / `catch` — multi-level exit
- `return` in block — leaves the method

begin, rescue, else & ensure

BEGIN, RESCUE, ELSE, ENSURE

Wrap risky code in `begin ... end` (or a whole method body) and handle failures in **`rescue`** clauses, matched top to bottom by exception class. **`else`** runs only when nothing was raised, and **`ensure`** always runs, which makes it the place for cleanup. A bare `rescue` catches **`StandardError`** and its subclasses.

RESCUE RULES

- `rescue => e` binds the exception object
- List several classes: `rescue IOError, SystemCallError => e`
- `def ... rescue ... end` needs no `begin`

HANDLING EXCEPTIONS

```ruby
def read_number(path)
  Integer(File.read(path).strip)
rescue Errno::ENOENT => e
  warn "missing: #{e.message}"
  0
rescue ArgumentError, TypeError
  0
ensure
  puts "checked #{path}"      # always runs
end

begin
  result = 10 / divisor
rescue ZeroDivisionError => e
  puts e.message              # divided by 0
else
  puts "no error"             # only when nothing raised
end
```

CODE EXPLAINED

→ `rescue Errno::ENOENT => e` — specific class, bound to `e`

→ multiple classes — comma-separated in one clause

→ `ensure` — runs on success, error, or `return`

THE RUBY WAY

- `begin ... end` wrapping a whole method → method-level `rescue`
- `rescue Exception` → `rescue StandardError` (the default)
- `rescue => e; nil` that hides errors → rescue only what you can handle

IN RAILS

- **Rails** · controllers use `rescue_from ActiveRecord::RecordNotFound, with: :not_found`
- **Rails** · `find` raises `RecordNotFound`; `find_by` returns `nil`

COMMON EXCEPTIONS

CLASS	RAISED WHEN
`NoMethodError`	method missing (often on nil)
`ArgumentError`	bad argument / count
`TypeError`	wrong type (`"a" + 1`)
`ZeroDivisionError`	integer `/ 0`
`Errno::ENOENT`	file not found

EXCEPTION METHODS

METHOD	RETURNS
`e.message`	error text
`e.backtrace`	array of frames
`e.full_message`	formatted report
`e.cause`	wrapped exception

⚠ COMMON MISTAKES

- Rescuing `Exception`; it also traps `Interrupt` (Ctrl-C) and `SystemExit`, so scripts won't stop
- Putting a broad `rescue` before a specific one; the first matching clause wins
- Using the rescue modifier on big expressions; it silently hides `NoMethodError` typos too

✓ TIPS & BEST PRACTICES

- Rescue as narrowly as possible and as close to the failing call as you can
- Log `e.full_message` (or `e.backtrace.first(5)`) so you can find where it failed
- Put cleanup in `ensure`, or better, use block APIs like `File.open { }` that clean up for you

QUICK SUMMARY

- `rescue Class => e` — handle a specific error
- `else` — success branch
- `ensure` — always runs
- bare `rescue` — StandardError only

Raising Exceptions

RAISING EXCEPTIONS

`raise` creates and throws an exception. With just a string it raises `RuntimeError`; with a class and message it raises that class. Inside a `rescue`, a bare `raise` **re-raises** the current exception, and raising a new one automatically records the original as its **cause**.

RAISE NOTES

- `raise "msg"` → `RuntimeError`
- `cause` is set automatically when you raise inside `rescue`

RAISING AND RE-RAISING

```ruby
def withdraw(amount)
  raise ArgumentError, "must be positive" unless amount.positive?
  raise "insufficient funds" if amount > @balance  # RuntimeError
  @balance -= amount
end

def load_config(path)
  JSON.parse(File.read(path))
rescue Errno::ENOENT
  raise ConfigError, "no config at #{path}"   # wraps original
end

begin
  load_config("x.json")
rescue ConfigError => e
  e.cause.class               # Errno::ENOENT
end

begin
  risky
rescue => e
  log(e)
  raise                       # re-raise the same exception
end
```

CODE EXPLAINED

- → `raise ArgumentError, "..."` — class plus message
- → `raise "..."` — defaults to `RuntimeError`
- → raising in `rescue` — the original becomes `e.cause`
- → bare `raise` — re-raises the exception being handled

THE RUBY WAY

- `raise "invalid"` for bad input → `raise ArgumentError, "invalid"`
- `raise e` after logging → bare `raise` (keeps the original backtrace)
- Returning error strings → raise a specific exception class

FORMS

FORM	RAISES
`raise`	re-raise current (or RuntimeError)
`raise "msg"`	RuntimeError
`raise Cls`	Cls with default message
`raise Cls, "msg"`	Cls with message
`raise obj`	the exception object

INSPECTING

METHOD	GIVES
`e.message`	text
`e.backtrace`	frames (strings)
`e.cause`	original exception
`e.full_message(highlight: false)`	printable report

⚠ COMMON MISTAKES

- Raising bare strings for programmer errors; callers can't rescue `RuntimeError` precisely
- `rescue => e; raise e.message` ; you lose the class and the backtrace
- Raising inside `ensure` ; it replaces whatever exception was already in flight

✓ TIPS & BEST PRACTICES

- Use built-in classes when they fit: `ArgumentError` , `TypeError` , `KeyError` , `NotImplementedError`
- Validate arguments at the top of the method with guard-clause `raise` s
- Error messages should include the bad value: `"expected positive, got #{amount}"`

QUICK SUMMARY

- `raise Cls, "msg"` — raise a specific error
- `raise "msg"` — RuntimeError
- bare `raise` — re-raise
- `e.cause` — wrapped original

Exception Hierarchy & Custom Errors

EXCEPTION HIERARCHY AND CUSTOM ERRORS

All exceptions inherit from **`Exception`**, but application errors should inherit from **`StandardError`**, because that's what a bare `rescue` catches. Define custom errors as small subclasses, ideally under one base class per library or app so callers can rescue all of them at once.

HIERARCHY NOTES

- `Exception` also covers `Interrupt`, `SystemExit`, `NoMemoryError`
- Custom errors: `class MyError < StandardError; end`
- A class can carry extra data via `attr_reader`
- `rescue` clauses match subclasses: `rescue MyLib::Error` gets them all

CUSTOM EXCEPTIONS

```ruby
module Shop
  class Error < StandardError; end          # library base
  class OutOfStock < Error; end
  class PaymentFailed < Error
    attr_reader :code
    def initialize(msg = "payment failed", code: nil)
      super(msg)
      @code = code
    end
  end
end

begin
  raise Shop::PaymentFailed.new(code: 402)
rescue Shop::OutOfStock
  puts "restock"
rescue Shop::Error => e                     # catches PaymentFailed
  puts "#{e.message} (#{e.code})"           # payment failed (402)
end
```

CODE EXPLAINED

→ `Error < StandardError` — one base for the whole library

→ `OutOfStock < Error` — specific, but still rescuable as `Shop::Error`

→ `super(msg)` — keeps `message` working

→ rescue order — specific classes first, base class last

THE RUBY WAY

- `class MyError < Exception` → `< StandardError`
- Many unrelated top-level error classes → nest them under `MyLib::Error`
- Encoding details in the message string → expose them with `attr_reader`

IN RAILS

- **Rails** · app errors often live in `app/errors/` and inherit from a shared base
- **Rails** · `rescue_from Shop::Error, with: :render_error` maps them to HTTP responses

BUILT-IN HIERARCHY

CLASS	PARENT
`StandardError`	Exception
`RuntimeError`	StandardError
`ArgumentError`	StandardError
`KeyError`	IndexError
`Interrupt`	SignalException

RESCUE SCOPE

CLAUSE	CATCHES
`rescue`	StandardError + subclasses
`rescue Exception`	everything (avoid)
`rescue Shop::Error`	all Shop errors

⚠ COMMON MISTAKES

- Inheriting from `Exception`; a plain `rescue` then misses your error entirely
- Forgetting `super(msg)` in `initialize`; `message` falls back to the class name
- Rescuing the base class before specific subclasses; specific handlers never run

✓ TIPS & BEST PRACTICES

- Give each gem or app one `Error` base class; it's the first thing callers look for
- A default message in `initialize` keeps `raise OutOfStock` short at call sites
- Name errors for what went wrong (`OutOfStock`), not where (`InventoryServiceError`)

QUICK SUMMARY

- `< StandardError` — custom error base
- `MyLib::Error` — one base per library
- `attr_reader` — extra error data
- specific first — rescue order matters

retry & Cleanup Patterns

RETRY AND CLEANUP PATTERNS

retry inside a `rescue` jumps back to the start of the `begin` block, which is handy for flaky network calls when combined with an attempt counter. For cleanup, prefer **block-based APIs** (`File.open { }`, `Mutex#synchronize { }`) that release resources automatically; `ensure` and `at_exit` cover the rest.

NOTES

- `retry` without a limit can loop forever
- Back off between attempts (`sleep 2 ** attempts`)
- `at_exit` blocks run in reverse order of registration

RETRY AND CLEANUP

```ruby
attempts = 0
begin
  attempts += 1
  response = fetch_data          # may raise Net::OpenTimeout
rescue Net::OpenTimeout, Errno::ECONNRESET
  if attempts < 3
    sleep(2 ** attempts)         # 2s, 4s
    retry                        # back to `begin`
  end
  raise                          # give up after 3 tries
end

def with_tempfile
  file = File.open("tmp.dat", "w")
  yield file
ensure
  file&.close
  File.delete("tmp.dat") if File.exist?("tmp.dat")
end

Timeout.timeout(5) { slow_call }  # require "timeout"
at_exit { puts "bye" }
```

CODE EXPLAINED

- → `retry` — restarts the `begin` block; counter lives outside it
- → `sleep(2 ** attempts)` — exponential backoff
- → `raise` after the limit — re-raise the last error
- → `yield` + `ensure` — build your own cleanup helper

THE RUBY WAY

- `f = File.open(...); ...; f.close` → `File.open(...) { |f| ... }`
- Copy-pasted retry loops → one `with_retries(times: 3) { }` helper
- `Timeout.timeout` around IO → the library's own timeout options

IN RAILS

- **Rails** · ActiveJob has `retry_on Net::OpenTimeout, wait: :polynomially_longer, attempts: 5`
- **Rails** · `discard_on ActiveJob::DeserializationError` drops jobs that can't succeed

CLEANUP TOOLS

TOOL	USE
`ensure`	always-run cleanup
`File.open { }`	auto-close files
`Mutex#synchronize { }`	auto-unlock
`at_exit { }`	run at program exit

RETRY CHECKLIST

STEP	WHY
`count attempts`	avoid infinite loops
`rescue specific errors`	don't retry bugs
`back off`	don't hammer a failing service
`re-raise at the end`	surface the failure

⚠ COMMON MISTAKES

- Retrying on every `StandardError` ; bugs like `NoMethodError` will be retried pointlessly
- Initializing the counter inside `begin` ; it resets on every `retry` , so the loop never ends
- Relying on `Timeout.timeout` for code that holds locks; it can interrupt at unsafe points

✓ TIPS & BEST PRACTICES

- Use `Net::HTTP` options (`open_timeout` , `read_timeout`) instead of wrapping in `Timeout`
- Write cleanup helpers with `yield` inside `begin/ensure` , like the stdlib does
- Add jitter (`sleep(2 ** n + rand)`) when many clients retry at once

QUICK SUMMARY

- `retry` — re-run the begin block
- attempt counter — limit retries
- block APIs — automatic cleanup
- `ensure` / `at_exit` — always-run code

Defining & Calling Methods

def · implicit return · optional parens · ? predicates · ! bang methods · setter= · endless def

DEFINING AND CALLING METHODS

Methods are defined with `def` and **return the value of the last expression**; `return` is only needed to leave early. Parentheses are optional at both the definition and the call site. Naming conventions add meaning: **?** for predicates, **!** for dangerous variants, and **name=** for setters.

METHOD NOTES

- Every method returns something (`nil` if the body is empty)
- `def x = expr` defines an endless one-line method (3.0+)
- Top-level `def` methods become private methods of `Object`

DEF, RETURN VALUES, NAMING

```ruby
def greet(name)
  "Hello, #{name}"          # implicit return
end
greet "Ada"                 # "Hello, Ada" (parens optional)

def sign(n)
  return :zero if n.zero?   # early return
  n.positive? ? :pos : :neg
end

def square(x) = x * x       # endless method
def adult?(age) = age >= 18 # predicate
def min_max(a) = [a.min, a.max]
lo, hi = min_max([3, 9, 1]) # multiple return via array

class Account
  def balance=(v)           # setter: acct.balance = 10
    raise ArgumentError if v.negative?
    @balance = v
  end
  def reset! = @balance = 0 # bang: dangerous / mutating
end
```

CODE EXPLAINED

→ implicit return — the last line's value comes back

→ `return :zero if` — early exit, then the normal path

→ `def square(x) = x * x` — endless definition

→ `min_max` — "multiple returns" are just an array

→ `balance=` — called as `acct.balance = 10`

THE RUBY WAY

- `return x` on the last line → just `x`
- `def is_adult(age)` → `def adult?(age)`
- `def get_name` / `set_name` → `name` / `name=`

IN RAILS

- **Rails** · boolean columns get `?` methods for free: `user.admin?`
- **AS** · `delegate :email, to: :user` defines a forwarding method

METHOD NAMING

FORM	MEANING
`name`	normal method / getter
`name?`	returns true / false
`name!`	dangerous or in-place variant
`name=`	setter (`obj.name = v`)
`+ [] <=>`	operator methods

CALLING

CALL	NOTES
`greet("Ada")`	standard
`greet "Ada"`	no parens
`obj.greet`	explicit receiver
`greet`	implicit `self`

⚠ COMMON MISTAKES

- Putting a space before the parenthesis: `greet ("a", "b")` parses the parens as one argument expression and fails
- Expecting a setter's return value; `obj.x = 5` always returns `5`, whatever the method returns
- Writing `return a, b` and forgetting it returns an array `[a, b]`

✓ TIPS & BEST PRACTICES

- Keep methods short; Ruby's style guide suggests under ~10 lines
- Use parentheses when there are arguments, except for DSL-style calls like `puts` and `raise`
- `?` methods should return `true` / `false`, not truthy values, so callers can store them safely

QUICK SUMMARY

- `def` — define; last expression returns
- `def x = expr` — endless method
- `?` / `!` / `=` — naming conventions
- arrays — multiple return values

Parameters & Keyword Arguments

PARAMETERS AND KEYWORD ARGUMENTS

Ruby methods take **positional** parameters (required or with defaults) and **keyword** parameters (`name:`), which can also be required or optional. Keywords make call sites self-documenting and order-independent. Since Ruby 3.0, keywords and positional hashes are **fully separated**.

PARAMETER RULES

- Defaults can use earlier params: `def f(a, b = a * 2)`
- Required keyword: `def f(name:)` (no default)
- Wrong count → `ArgumentError (wrong number of arguments)`
- Unknown keyword → `ArgumentError (unknown keyword: :x)`

POSITIONAL AND KEYWORD PARAMS

```ruby
def greet(name, greeting = "Hello")
  "#{greeting}, #{name}!"
end
greet("Ada")                      # "Hello, Ada!"
greet("Ada", "Hi")                # "Hi, Ada!"

def connect(host, port: 5432, ssl: false)
  "#{host}:#{port} ssl=#{ssl}"
end
connect("db")                     # "db:5432 ssl=false"
connect("db", ssl: true)          # "db:5432 ssl=true"
connect("db", ssl: true, port: 1) # order doesn't matter

def create_user(email:, role: :member)   # email required
  [email, role]
end
create_user(email: "a@b.c")       # ["a@b.c", :member]
create_user                       # ArgumentError: missing keyword: :email
create_user(email: "x", admin: 1) # ArgumentError: unknown keyword: :admin
```

CODE EXPLAINED

- → `greeting = "Hello"` — optional positional with a default
- → `port: 5432` — optional keyword
- → `email:` — required keyword (no default)
- → unknown keywords — rejected, which catches typos

THE RUBY WAY

- `def resize(w, h, keep_ratio, crop)` → keywords: `resize(w, h, crop: true)`
- `def f(opts = {}) + opts[:x]` → `def f(x: nil)`
- Boolean positional args (`save(true)`) → `save(validate: true)`

PARAMETER KINDS

SYNTAX	KIND
`a`	required positional
`a = 1`	optional positional
`*args`	rest positional (sheet 042)
`k:`	required keyword
`k: 1`	optional keyword
`**opts`	rest keywords

REQUIRED ORDER

POSITION	PARAMETERS
1	required positional
2	optional positional
3	`*rest`
5	keywords, then `**opts`
6	`&block`

⚠ COMMON MISTAKES

- Passing a hash where keywords are expected: `f(opts)` doesn't work since 3.0; use `f(**opts)`
- Mutable defaults are rebuilt each call, so `def f(list = [])` is safe in Ruby (unlike Python); don't add workarounds
- Too many positional params; past three, callers can't tell which argument is which

✓ TIPS & BEST PRACTICES

- Make any boolean or "mode" argument a keyword
- `method(:connect).parameters` lists every parameter and its kind
- Required keywords give clear errors: `missing keyword: :email`

QUICK SUMMARY

- `a = default` — optional positional
- `k:` / `k: v` — required / optional keyword
- keywords — order-independent, self-documenting
- 3.0+ — keywords and hashes separated

Variadic Params & Forwarding

*args · **opts · &block · … forwarding · anonymous * ** & · splat at call site

VARIADIC PARAMS AND FORWARDING

A splat `*args` collects extra positional arguments into an array, `**opts` collects extra keywords into a hash, and `&block` captures the block as a Proc. `...` forwards everything (positional, keywords and block) to another method, and anonymous `*`, `**`, `&` forward just one kind.

NOTES

- Only one `*rest` and one `**rest` per method
- `...` must be the last parameter (leading args allowed: `def f(a, ...)`)
- Anonymous `*` / `**` / `&` forwarding: Ruby 3.2+

SPLATS AND FORWARDING

```ruby
def sum(*nums) = nums.sum
sum(1, 2, 3)                # 6
sum(*[4, 5])                # 9 (splat at call)

def tag(name, *classes, **attrs)
  [name, classes, attrs]
end
tag(:div, "a", "b", id: 1)  # [:div, ["a", "b"], {id: 1}]

def each_twice(&block)      # capture block as a Proc
  2.times(&block)
end
each_twice { print "hi " }  # hi hi

def log_call(...)           # forward everything
  puts "calling"
  target(...)
end

def wrap(*, **, &)          # anonymous forwarding (3.2+)
  inner(*, **, &)
end
```

CODE EXPLAINED

- → `*nums` — any number of positional args, as an array
- → `**attrs` — extra keywords, as a hash
- → `&block` — the block as a `Proc`; pass it on with `&block`
- → `...` — forwards args, keywords and block unchanged

THE RUBY WAY

- `def f(*args, **kw, &blk) = g(*args, **kw, &blk)` → `def f(...) = g(...)`
- `def f(list)` called as `f([a, b])` → `def f(*items)` called as `f(a, b)`
- Capturing `&block` just to `call` it → use `yield`

IN RAILS

- **AS** · `delegate :name, :email, to: :user` writes these forwarding methods for you
- **Rails** · service objects often use `def self.call(...) = new(...).call`

COLLECTORS

PARAM	RECEIVES
`*args`	Array of extra positionals
`**opts`	Hash of extra keywords
`&blk`	Proc (or nil)
`...`	everything, forwarded
`* / ** / &`	anonymous, forwarded

CALL-SITE SPREADS

CALL	PASSES
`f(*arr)`	elements as positionals
`f(**hash)`	pairs as keywords
`f(&proc)`	proc as the block

⚠ COMMON MISTAKES

- Passing an array without `*` to a `*args` method; it arrives as one element: `[[1, 2]]`
- Mixing `...` with other forwarding; you can't name `*args` and also use `...`
- Using `&block` everywhere; it allocates a Proc on each call, while `yield` doesn't

✓ TIPS & BEST PRACTICES

- `...` is the cleanest way to write wrappers and decorators
- Use `*args` for genuinely variable lists (like `puts`), keywords for options
- `block_given?` works with `yield`; with `&block` you can test `block.nil?`

QUICK SUMMARY

- `*args` / `**opts` — collect extras
- `&block` — capture the block
- `...` — forward everything
- `f(*arr)` / `f(**h)` — spread at call site

Variable Scope

VARIABLE SCOPE

Ruby has **scope gates**: `def`, `class` and `module` each start a fresh local scope, so a method **cannot see** local variables from outside it. **Blocks are different**: they see and can change the locals around them, which is what makes closures work. Instance variables belong to `self`, constants are looked up lexically, and globals are visible everywhere.

SCOPE RULES

- `def` can't read outer locals (use args or `@ivars`)
- Blocks read and write surrounding locals
- Block params and `|x; y|` block-locals shadow outer names
- `define_method` with a block crosses the `def` gate

SCOPE GATES AND BLOCKS

```ruby
x = 10
def show_x
  x                       # NameError: undefined local variable
end

def show_ivar = @name     # instance variables live on self
@name = "top"
show_ivar                 # "top" (same self at top level)

total = 0
[1, 2, 3].each { |n| total += n }
total                     # 6 — block changed outer local

y = 1
[5].each { |y| y += 1 }   # block param shadows outer y
y                         # 1
z = 1
[5].each { |n; z| z = n } # z is block-local
z                         # 1

LIMIT = 3
def under_limit?(n) = n < LIMIT   # constants pass scope gates
```

CODE EXPLAINED

- → `def show_x` — new scope; outer `x` isn't visible
- → `@name` — belongs to `self`, not to a scope
- → `total += n` — blocks share outer locals
- → `|n; z|` — declare a block-local variable
- → `LIMIT` — constants are visible inside methods

THE RUBY WAY

- `$counter` to share state between methods → an instance variable or object
- Reading outer locals in `def` → pass them as arguments
- Reusing outer names as block params → choose distinct names

VARIABLE VISIBILITY

KIND	SEEN IN DEF?	SEEN IN BLOCK?
`local`	no	yes
`@ivar`	yes (same self)	yes
`@@cvar`	yes (in class)	yes
`CONST`	yes	yes

SCOPE GATES

KEYWORD	OPENS NEW SCOPE
`def`	yes
`class / module`	yes
`do … end / { }`	no (sees outer)
`define_method { }`	no (closure)

⚠ COMMON MISTAKES

- Expecting a top-level local to be visible in a method; Ruby isn't JavaScript or Python here
- Accidentally overwriting an outer local inside a block because names collide
- Treating `@ivar` at top level as global state; it belongs to the `main` object and gets confusing fast

✓ TIPS & BEST PRACTICES

- Pass data into methods explicitly; it keeps them testable and reusable
- Use `define_method` or `Class.new` when you deliberately need a closure over locals (sheet 047)
- `binding.local_variables` shows what's in scope while debugging

QUICK SUMMARY

- `def` / `class` / `module` — scope gates
- blocks — see outer locals
- `@ivar` — tied to `self`
- `|x; y|` — block-local variable

Blocks & yield

BLOCKS AND YIELD

A **block** is a chunk of code passed to a method, written with `{ }` (one line) or `do ... end` (multi-line). The method runs it with **`yield`**, passing values that arrive as block parameters, and gets the block's last value back. **`block_given?`** checks whether a block was supplied. `it` (3.4+) and `_1` name the parameter implicitly.

BLOCK FACTS

- Every method can take one block; it isn't in the param list
- `yield` with no block raises `LocalJumpError`
- `{ }` binds tighter than `do ... end`
- `it` refers to the single block parameter (Ruby 3.4+)

WRITING AND CALLING BLOCKS

```ruby
[1, 2, 3].each { |n| print n }     # 123

def twice
  yield 1
  yield 2
end
twice { |x| print x }              # 12

def transform(value)
  return value unless block_given?
  yield(value)                     # block's result returned
end
transform(5)                       # 5
transform(5) { |v| v * 3 }         # 15


[1, 2, 3].map { it * 2 }           # [2, 4, 6] (3.4+)
[1, 2, 3].map { _1 * 2 }           # [2, 4, 6] (2.7+)
```

CODE EXPLAINED

→ `{ }` / `do ... end` — same thing; style differs by length

→ `yield 1` — call the block with an argument

→ `block_given?` — make the block optional

→ `it` / `_1` — implicit parameter names

THE RUBY WAY

- `{ }` blocks spanning many lines → `do ... end`
- `map { |x| x * 2 }` → `map { it * 2 }` (3.4+), or keep `|x|` when clearer
- Methods returning arrays only to iterate → `yield` each item

BLOCK SYNTAX

FORM	USE
`{ |x| … }`	one-liner
`do |x| … end`	multi-line
`|a, b|`	two params
`|(a, b), c|`	destructure
`it / _1`	implicit param

INSIDE THE METHOD

CALL	DOES
`yield`	run block
`yield a, b`	pass values
`block_given?`	was a block passed?
`yield(x) || default`	use block result

⚠ COMMON MISTAKES

- `puts [1, 2].map do |x| x end`; `do...end` binds to `puts`, so `map` gets no block. Use `{ }`
- Calling `yield` in a method that's sometimes called without a block; guard with `block_given?`
- Mixing `it` with explicit params, or using `it` in nested blocks where it's ambiguous

✓ TIPS & BEST PRACTICES

- Return `to_enum(__method__)` when no block is given so your iterator chains with `map` etc. (sheet 106)
- Prefer `yield` over `&block` + `call`; it's faster and reads better
- Use `it` for short single-parameter blocks; name the parameter when the block grows

QUICK SUMMARY

- `{ }` / `do … end` — block syntax
- `yield` — run the block
- `block_given?` — optional block
- `it` / `_1` — implicit param

Procs & &block

PROCS AND &BLOCK

A **Proc** is a block turned into an object you can store, pass around and call later. `&block` in a parameter list converts the caller's block into a Proc; `&obj` at a call site converts a Proc (or anything with `to_proc`) back into a block. Procs are **lenient** about arguments: extra args are dropped and missing ones become `nil`.

PROC FACTS

- Create with `proc { }`, `Proc.new { }`, or capture with `&block`
- Call with `.call`, `.()`, `[]`, or `yield`
- `&:name` uses `Symbol#to_proc`
- `return` inside a proc returns from the **enclosing method**

CREATING AND CALLING PROCS

```
square = proc { |x| x * x }
square.call(4)               # 16
square.(4)                   # 16
square.call(4, 99)           # 16  (extra arg ignored)
proc { |a, b| [a, b] }.call(1)  # [1, nil]

def run(&block)              # block → Proc
  block.call(10)
end
run { |n| n + 1 }            # 11

shout = proc { |s| s.upcase }
%w[a b].map(&shout)          # ["A", "B"]  (Proc → block)
%w[a b].map(&:upcase)        # ["A", "B"]  (Symbol#to_proc)

callbacks = []
callbacks << proc { puts "saved" }
callbacks.each(&:call)       # saved

add = proc { |a, b| a + b }
add.curry[1][2]              # 3
```

CODE EXPLAINED

→ `.call` / `.()` / `[]` — three ways to call

→ lenient arity — extras dropped, missing are `nil`

→ `&block` param — the caller's block, as a Proc

→ `map(&shout)` — turn a Proc back into a block

→ `callbacks.each(&:call)` — store blocks and run them later

THE RUBY WAY

- `map { |x| x.upcase }` → `map(&:upcase)`
- Storing a block via `Proc.new` with no block → capture with `&block`
- Procs for argument-checked callbacks → lambdas (sheet 046)

WAYS TO MAKE A PROC

CODE	KIND
`proc { |x| }`	proc
`Proc.new { |x| }`	proc
`def m(&b)`	proc from block
`lambda { |x| }`	lambda
`->(x) { }`	lambda
`:sym.to_proc`	lambda

CALLING

FORM	NOTE
`p.call(1)`	most explicit
`p.(1)`	short
`p[1]`	shortest
`p === 1`	used by `case/when`

⚠ COMMON MISTAKES

- Using `return` in a proc stored for later; once its method has returned, it raises `LocalJumpError`
- Relying on lenient arity and hiding bugs where callers pass the wrong number of values
- Writing `map(&:method(arg))`; `&:sym` only works for methods without arguments

✓ TIPS & BEST PRACTICES

- Use `&block` when you need to store or forward a block; use `yield` when you only call it
- Procs respond to `===`, so they work as `when` conditions
- `Method#to_proc` lets you pass named methods: `items.each(&method(:puts))`

QUICK SUMMARY

- `proc { }` — block as an object
- `&block` — block → Proc
- `&obj` — Proc → block
- lenient arity — extras ignored

Lambdas

LAMBDAS

A **lambda** is a Proc with **strict argument checking** and **method-like `return`**: `return` exits only the lambda itself. Write them with the stabby literal `->(x) { }` or `lambda { |x| }`. Lambdas are the right choice for small functions you store in variables, hashes or constants.

LAMBDA FACTS

- Wrong number of args → `ArgumentError`
- `return` leaves the lambda, not the surrounding method
- `.curry` builds partially applied versions

LAMBDAS IN PRACTICE

```
square = ->(x) { x * x }
square.(5)                    # 25
square.call(5, 6)             # ArgumentError (given 2, expected 1)

greet = ->(name, greeting: "Hi") { "#{greeting} #{name}" }
greet.("Ada")                 # "Hi Ada"
greet.("Ada", greeting: "Yo") # "Yo Ada"

def find_first_even(nums)
  check = ->(n) { return n.even? }   # return leaves lambda only
  nums.find { |n| check.(n) }
end
find_first_even([1, 4, 6])    # 4

TAX = { food: ->(p) { p * 1.02 }, other: ->(p) { p * 1.08 } }
TAX[:food].(100)              # 102.0

add = ->(a, b, c) { a + b + c }
add5 = add.curry[5]
add5[1][2]                    # 8
compose = square >> ->(n) { n + 1 }
compose.(3)                   # 10
```

CODE EXPLAINED

- → `->(x) { }` — stabby lambda literal
- → strict arity — extra or missing args raise
- → `return` — exits just the lambda
- → hash of lambdas — a dispatch table
- → `curry` / `>>` — partial application and composition

THE RUBY WAY

- `lambda { |x| ... }` → `->(x) { ... }`
- `case` -heavy dispatch on symbols → a hash of lambdas
- Tiny private helper methods used once → a local lambda

IN RAILS

- **Rails** · scopes are lambdas: `scope :active, -> { where(active: true) }`
- **Rails** · `validates :age, if: -> { adult? }` takes a lambda condition

PROC VS LAMBDA

FEATURE	PROC	LAMBDA
`arg count`	lenient	strict
`return`	from method	from lambda
`lambda?`	false	true
`literal`	proc { }	->() { }
`&:sym`	—	is a lambda

LAMBDA SYNTAX

FORM	EXAMPLE
`no args`	`-> { 42 }`
`one arg`	`->(x) { x }`
`default`	`->(x = 1) { x }`
`keyword`	`->(k:) { k }`

⚠ COMMON MISTAKES

- Calling a lambda with the wrong number of args from a callback API that passes extras (e.g. `each_with_index`)
- Writing `-> (x)` with a space before the paren; it works but warns in some linters and reads oddly
- Defining lambdas inside hot loops; build them once outside

✓ TIPS & BEST PRACTICES

- Use lambdas for anything stored in a variable or constant; arity errors catch bugs early
- `method(:name)` gives a Method object that behaves like a lambda
- `>>` and `<<` compose lambdas and methods into pipelines

QUICK SUMMARY

- `->(x) { }` — lambda literal
- strict arity — raises on mismatch
- `return` — exits the lambda
- `curry` / `>>` — functional helpers

Closures

captured variables · counter factory · define_method closures · binding · scope-gate workaround

CLOSURES

Blocks, procs and lambdas are **closures**: they capture the **variables** (not just the values) from where they were created and keep them alive. Changing a captured variable later is visible inside the closure, and the closure can change it too. This is how counters, memoizers and callbacks keep private state without classes.

CLOSURE FACTS

- Captures bindings, not copies
- Each call of a factory method creates new captured variables
- `def` doesn't close over outer locals; `define_method` does

CAPTURING STATE

```ruby
def make_counter
  count = 0
  increment = -> { count += 1 }
  current   = -> { count }
  [increment, current]
end
inc, cur = make_counter
inc.(); inc.()
cur.()                          # 2

x = 1
show = -> { x }
x = 99
show.()                         # 99 (sees the variable, not a copy)

def multiplier(n) = ->(v) { v * n }
triple = multiplier(3)
[1, 2].map(&triple)             # [3, 6]

class Greeter
  name = "Ada"
  define_method(:hello) { "Hi #{name}" }  # closes over name
end
Greeter.new.hello               # "Hi Ada"
```

CODE EXPLAINED

- → `count` — lives on after `make_counter` returns
- → `x = 99` — closures see later changes
- → `multiplier(3)` — returns a configured function
- → `define_method { }` — a method body that's a closure

THE RUBY WAY

- Global counters → a closure or a small object
- Building similar methods by copy-paste → `define_method` in a loop
- Long-lived closures over big objects → capture only what's needed

WHAT CLOSES OVER

CONSTRUCT	CAPTURES OUTER LOCALS?
`block`	yes
`proc / lambda`	yes
`define_method { }`	yes
`def`	no

USES

PATTERN	EXAMPLE
`counter / generator`	`make_counter`
`configured function`	`multiplier(3)`
`callback with context`	`on_save { log(user) }`
`memoizer`	cache hash captured

⚠ COMMON MISTAKES

- Creating lambdas in a loop that capture one shared variable; with `each` each iteration gets its own, but a `while` loop variable is shared
- Holding a closure that captures a huge object and wondering why memory isn't freed
- Expecting a closure to snapshot a value; it reads the variable's current value

✓ TIPS & BEST PRACTICES

- Prefer a class once a closure needs more than two or three pieces of state
- `each` creates a fresh block variable per iteration, which avoids the classic loop-closure bug
- `define_method` closures can't be seen by `def`, which makes them handy for metaprogramming (sheet 109)

QUICK SUMMARY

- closure — captures variables
- factory method — fresh state per call
- `define_method` — closure as a method
- `def` — not a closure

Sheet 048

Method Objects & Composition

method(:name) · Method#call · &method(:puts) · >> / << · send / public_send · unbind / bind

METHOD OBJECTS AND COMPOSITION

`method(:name)` returns a **Method** object you can call, pass as a block with &, inspect, or compose with **>>** and **<<**. `send` and `public_send` call a method whose name is only known at runtime. Unbinding and rebinding (`instance_method`, `bind`) attach a method to another object of the same type.

NOTES

- `send` ignores visibility; `public_send` respects it
- `Method#arity`, `#parameters`, `#source_location` describe a method
- `f >> g` means "call f, then g"
- `Method#to_proc` lets you pass methods where blocks are expected

METHODS AS OBJECTS

```ruby
m = 5.method(:+)
m.call(3)                          # 8

%w[1 2 3].map(&method(:Integer))  # [1, 2, 3]
[1, 2].each(&method(:puts))       # prints 1, 2

double = ->(x) { x * 2 }
inc    = ->(x) { x + 1 }
(double >> inc).(5)               # 11  (double first)
(double << inc).(5)               # 12  (inc first)
(method(:Integer) >> double).("21")  # 42

"hello".send(:upcase)             # "HELLO"
"hello".public_send(:length)      # 5
op = :+
3.public_send(op, 4)              # 7

um = String.instance_method(:upcase)   # UnboundMethod
um.bind_call("abc")               # "ABC"
def greet(name, greeting: "Hi") = nil
method(:greet).parameters         # [[:req, :name], [:key, :greeting]]
```

CODE EXPLAINED

- → `5.method(:+)` — a bound Method object
- → `&method(:Integer)` — pass a method as a block
- → `>>` / `<<` — compose left-to-right / right-to-left
- → `public_send(op, 4)` — dynamic call, visibility respected
- → `bind_call` — run an unbound method on another receiver

THE RUBY WAY

- `map { |s| Integer(s) }` → `map(&method(:Integer))`
- `obj.send(user_input)` → `public_send`, with an allow-list
- Nested calls `inc.(double.(x))` → `(double >> inc).(x)`

METHOD INTROSPECTION

CALL	RETURNS
`m.arity`	arg count (-n = variadic)
`m.parameters`	kinds and names
`m.owner`	defining class/module
`m.receiver`	bound object
`m.source_location`	[file, line]

DYNAMIC CALLS

CALL	VISIBILITY
`send(:m, *args)`	ignores private
`__send__(:m)`	same, never overridden
`public_send(:m)`	public only
`respond_to?(:m)`	check first

⚠ COMMON MISTAKES

- Using `send` with user-controlled names; it can call private methods like `system`
- Expecting `&method(:name)` to be fast in hot loops; it allocates, so a plain block can be quicker
- Confusing `>>` and `<<` order; `f >> g` runs `f` first

✓ TIPS & BEST PRACTICES

- `source_location` jumps straight to where a method is defined while debugging
- Prefer `public_send` in application code; reserve `send` for tests and metaprogramming
- Composition with `>>` keeps small transformation steps reusable

QUICK SUMMARY

- `method(:x)` — Method object
- `&method(:x)` — method as block
- `>>` / `<<` — composition
- `public_send` — safe dynamic call

Recursion & Memoization

base case · recursive methods · Hash.new { |h, k| } memo · @memo ||= · SystemStackError · no default TCO

RECURSION AND MEMOIZATION

A recursive method calls itself on a smaller input until it reaches a **base case**. CRuby does not optimize tail calls by default, so very deep recursion raises **SystemStackError**. **Memoization** caches results so repeated subproblems are computed once; a Hash with a default block is the classic Ruby way to do it.

NOTES

- Always handle the base case first
- Default stack depth is roughly 10k frames (varies)
- `Hash.new { |h, k| h[k] = ... }` computes and stores on first lookup
- `@x ||= compute` memoizes a single value per object

RECURSION AND CACHING

```ruby
def factorial(n)
  return 1 if n <= 1          # base case
  n * factorial(n - 1)        # recursive case
end
factorial(5)                  # 120

def fib(n) = n < 2 ? n : fib(n - 1) + fib(n - 2)
fib(30)                       # 832040 (slow: ~2.7M calls)

fib = Hash.new { |h, n| h[n] = n < 2 ? n : h[n - 1] + h[n - 2] }
fib[90]                       # 2880067194370816120 (instant)

def flat(x) = x.is_a?(Array) ? x.flat_map { flat(it) } : [x]
flat([1, [2, [3, [4]]]])      # [1, 2, 3, 4]

class Report
  def totals = @totals ||= expensive_query   # memoize once
end

def depth(n) = depth(n + 1)
depth(0)                      # SystemStackError: stack level too deep
```

CODE EXPLAINED

- → `return 1 if n <= 1` — stops the recursion
- → `fib(30)` — exponential calls without caching
- → `Hash.new { |h, n| h[n] = ... }` — compute once, store forever
- → `flat_map` + recursion — walk nested structures
- → `@totals ||=` — per-object memoization

THE RUBY WAY

- Recursive `fib` → memoized hash or a loop
- Deep recursion over long lists → iterate with `each` / `reduce`
- Recomputing a value in many methods → `@value ||= compute`

IN RAILS

- **Rails** · `Rails.cache.fetch("stats", expires_in: 1.hour) { compute }` memoizes across requests
- **Rails** · controllers use `@current_user ||= User.find_by(id: session[:user_id])`

MEMOIZATION OPTIONS

TECHNIQUE	SCOPE
`@x ||= v`	one value per object
`Hash.new { }`	many keys, computed on demand
`@cache[k] ||= v`	manual per-key cache
`constant hash`	shared, process-wide

RECURSION CHECKLIST

STEP	WHY
`base case first`	stop condition
`shrink the input`	guarantees progress
`watch depth`	~10k frames max
`cache overlaps`	avoid exponential time

⚠ COMMON MISTAKES

- Forgetting the base case, or one that is never reached (e.g. `n == 0` with negative input)
- Using `||=` to memoize methods that can return `nil` or `false`; they recompute every time
- Recursing over large inputs and hitting `SystemStackError`; convert to a loop or explicit stack

✓ TIPS & BEST PRACTICES

- Memoize falsy values with `defined?(@x) ? @x : (@x = compute)`
- Use an explicit array as a stack for deep tree walks
- `Hash#dig` and `flat_map` handle many nested-data tasks without hand-written recursion

QUICK SUMMARY

- base case + smaller input — recursion pattern
- `SystemStackError` — too deep
- `Hash.new { }` — memo table
- `@x \|\|=` — memoize one value

Array Basics

[] · Array.new(3, 0) · Array.new(3) { } · %w[] · indexing · first / last · dig · length / empty?

ARRAY BASICS

Arrays are **ordered, integer-indexed** collections that can hold any mix of objects and grow automatically. Indexing with an out-of-range position returns `nil` instead of raising. Most array work is done with methods (`first`, `dig`, `each`) rather than manual index loops.

ARRAY FACTS

- Negative indexes count from the end
- `Array.new(3, obj)` shares **one** object in every slot
- `Array.new(3) { }` builds a fresh object per slot
- `%w[]` and `%i[]` build word / symbol arrays

CREATING AND READING ARRAYS

```ruby
nums = [10, 20, 30, 40]
mixed = [1, "two", :three, nil, [5]]
words = %w[apple banana cherry]   # ["apple", "banana", "cherry"]

Array.new(3)                  # [nil, nil, nil]
Array.new(3, 0)               # [0, 0, 0]
Array.new(3) { |i| i * i }    # [0, 1, 4]

nums[0]                       # 10
nums[-1]                      # 40
nums[10]                      # nil
nums.first                    # 10
nums.last(2)                  # [30, 40]
nums.length                   # 4
nums.fetch(10, :none)         # :none
[[1, [2, 3]]].dig(0, 1, 0)    # 2

grid = Array.new(2) { Array.new(2, 0) }  # independent rows
grid[0][0] = 1
grid                          # [[1, 0], [0, 0]]
```

CODE EXPLAINED

- → `Array.new(3) { |i| }` — block builds each element
- → `nums[10]` — out of range is `nil`, not an error
- → `last(2)` — the last N elements
- → `fetch(10, :none)` — default instead of `nil`
- → `dig(0, 1, 0)` — safe nested lookup

THE RUBY WAY

- `arr[arr.length - 1]` → `arr.last`
- `["a", "b", "c"]` → `%w[a b c]`
- `Array.new(3, [])` for a grid → `Array.new(3) { [] }`

IN RAILS

- **AS** · `Array.wrap(x)` is like `Array(x)` but doesn't split hashes
- **AS** · `arr.second`, `arr.third` … `arr.forty_two`
- **AS** · `%w[a b c].to_sentence` → `"a, b, and c"`

CREATING

CODE	RESULT
`[1, 2]`	literal
`%w[a b]`	["a", "b"]
`Array.new(2, "x")`	["x", "x"] (shared)
`Array.new(2) { [] }`	[[], []] (separate)
`Array(nil)`	[]

READING

CALL	RESULT
`a[0] / a[-1]`	first / last
`a.first(2)`	first 2
`a.fetch(i)`	IndexError if missing
`a.dig(i, j)`	nested, nil-safe
`a.size / length`	count

⚠ COMMON MISTAKES

- `Array.new(3, [])`: all three slots are the same array, so pushing to one pushes to all
- Expecting an error for a missing index; you get `nil`, which fails later and far away. Use `fetch`
- Using `count` when you mean `size`; `count` iterates when given an argument or block

✓ TIPS & BEST PRACTICES

- Use `first` / `last` instead of `[0]` / `[-1]`; they read better and take a count
- `fetch` with a block computes a default only when needed
- `Array#dig` pairs well with JSON data

QUICK SUMMARY

- `[]` / `%w[]` — create
- `a[i]` / `a[-1]` — index
- `first` / `last` / `dig` — read helpers
- `Array.new(n) { }` — fresh object per slot

Adding, Removing & Stack/Queue Use

push / << · pop · shift / unshift · insert · delete / delete_at · compact · stack & queue idioms

ADDING AND REMOVING ELEMENTS

Arrays grow and shrink at both ends, which makes them Ruby's built-in **stack** (`push` / `pop`) and **queue** (`push` / `shift`). Most mutating methods change the array in place and return either the array or the removed element, so read the return value carefully.

NOTES

- `<<` and `push` return the array, so they chain
- `pop` / `shift` return the removed item (or `nil` if empty)
- `delete` removes **every** matching element
- `shift` is O(1) in CRuby, so arrays make fine queues

PUSH, POP, SHIFT, DELETE

```ruby
a = [1, 2, 3]
a << 4                      # [1, 2, 3, 4]
a.push(5, 6)                # [1, 2, 3, 4, 5, 6]
a.pop                       # 6
a.shift                     # 1
a.unshift(0)                # [0, 2, 3, 4, 5]
a.insert(1, :x)             # [0, :x, 2, 3, 4, 5]
a.delete(:x)                # :x (returns the deleted value)
a.delete_at(0)              # 0
a.delete_if(&:even?)        # [3, 5]

[1, nil, 2, nil].compact    # [1, 2]
[1, 2] + [3]                # [1, 2, 3] (new array)
[1, 2].concat([3], [4])     # [1, 2, 3, 4] (in place)

stack = []; stack.push(1); stack.push(2); stack.pop  # 2 (LIFO)
queue = []; queue.push(1); queue.push(2); queue.shift # 1 (FIFO)
```

CODE EXPLAINED

- → `<<` / `push` — append at the end
- → `pop` / `shift` — remove from end / start
- → `unshift` / `insert` — add at start / position
- → `delete(v)` — removes all equal values
- → `delete_if` — remove by condition, in place

THE RUBY WAY

- `a = a + [x]` in a loop → `a << x`
- `a.delete_at(a.index(x))` → `a.delete(x)` (or `a.slice!`)
- `a.reject! { }` returning `nil` when unchanged → `delete_if` always returns the array

ADD

METHOD	WHERE
`<< / push`	end
`unshift / prepend`	start
`insert(i, v)`	at index
`concat(other)`	end, many

REMOVE

METHOD	RETURNS
`pop / shift`	the element
`delete(v)`	v or nil
`delete_at(i)`	element at i
`delete_if { }`	the array
`compact / compact!`	without nils

⚠ COMMON MISTAKES

- Modifying an array while iterating it with `each`; elements get skipped. Use `delete_if` or `reject`
- Expecting `delete` to remove only the first match; it removes all. Use `delete_at(index(v))` for one
- Using `+` in loops; it builds a new array every time

✓ TIPS & BEST PRACTICES

- `Thread::Queue` is the thread-safe queue (sheet 094); plain arrays are not
- `pop(2)` / `shift(2)` remove several elements at once
- `clear` empties an array in place, keeping other references valid

QUICK SUMMARY

- `<<` / `push` / `pop` — stack
- `push` / `shift` — queue
- `delete` / `delete_at` / `delete_if` — remove
- `compact` — drop nils

Array Operations

ARRAY OPERATIONS

Arrays support **set-like operators** (`&`, `\|`, `-`) and combining methods (`zip`, `product`, `flatten`, `transpose`). These return **new arrays**; the originals are unchanged. Set operators remove duplicates in their result and compare elements with `eql?` and `hash`.

NOTES

- `-` removes every occurrence of each listed element
- `\|` and `&` keep the left array's order
- `flatten(1)` flattens just one level
- `*` with a string joins: `[1, 2] * "," → "1,2"`

COMBINING ARRAYS

```
a = [1, 2, 2, 3]
b = [2, 3, 4]
a + b                       # [1, 2, 2, 3, 2, 3, 4]
a - b                       # [1]
a & b                       # [2, 3]
a | b                       # [1, 2, 3, 4]
a.uniq                      # [1, 2, 3]
[1, 2] * 2                  # [1, 2, 1, 2]

[[1, [2]], [3]].flatten     # [1, 2, 3]
[[1, [2]], [3]].flatten(1)  # [1, [2], 3]
[1, 2].zip([:a, :b])        # [[1, :a], [2, :b]]
[1, 2].product([:x, :y])    # [[1, :x], [1, :y], [2, :x], [2, :y]]
[[1, 2], [3, 4]].transpose  # [[1, 3], [2, 4]]
[1, 2, 3].rotate            # [2, 3, 1]
[3, 1, 2].reverse           # [2, 1, 3]
a.intersect?(b)             # true
```

CODE EXPLAINED

- `-` — difference; drops all matching values
- `&` / `\|` — intersection / union, deduplicated
- `flatten(1)` — limit the depth
- `zip` — pair up by position
- `intersect?` — any common element (3.1+)

THE RUBY WAY

- `(a + b).uniq` → `a \| b`
- `a.select { |x| b.include?(x) }` → `a & b`
- `(a & b).any?` → `a.intersect?(b)`

IN RAILS

- **AS** · `[1, 2, 3].excluding(2)` → `[1, 3]` ; `including(4)` adds
- **AS** · `(1..7).to_a.in_groups_of(3)` → rows padded with `nil`

SET OPERATORS

OP	[1,2,2] OP [2,3]
`+`	[1, 2, 2, 2, 3]
`-`	[1]
`&`	[2]
`|`	[1, 2, 3]

RESHAPING

METHOD	EFFECT
`flatten`	nested → flat
`zip`	pair by index
`product`	all combinations
`transpose`	rows ↔ columns
`rotate(n)`	shift positions
`each_slice(n)`	chunks of n

⚠ COMMON MISTAKES

- Using `a - b` to remove one instance of a value; it removes them all
- Calling `transpose` on ragged arrays; rows of different lengths raise `IndexError`
- Expecting `uniq` on objects to use `==` ; it uses `hash` + `eql?` (sheet 070)

✓ TIPS & BEST PRACTICES

- `uniq { |u| u.email }` dedupes by a computed key
- `zip` with a block iterates pairs without building the array
- For heavy set math on large arrays, switch to `Set` (sheet 058)

QUICK SUMMARY

- `+` / `-` — concat / difference
- `&` / `\|` — intersect / union
- `zip` / `product` / `transpose` — reshape
- `flatten(n)` — remove nesting

Array Slicing & Lookup

SLICING AND LOOKUP

Beyond single indexes, `arr[start, length]` and `arr[range]` return sub-arrays, and `take` / `drop` grab from the front. Lookup methods answer "is it there?" (`include?`), "where is it?" (`index`), and "give me several" (`values_at`). `sample` and `shuffle` pick randomly.

NOTES

- Slices return new arrays; out-of-range slices return `nil` or `[]`
- `index` takes a value or a block
- `values_at` accepts indexes and ranges
- `slice!` removes and returns the slice

SLICES AND LOOKUPS

```ruby
a = %w[a b c d e]
a[1, 2]                     # ["b", "c"]
a[1..3]                     # ["b", "c", "d"]
a[2..]                      # ["c", "d", "e"]
a[..1]                      # ["a", "b"]
a.take(2)                   # ["a", "b"]
a.drop(3)                   # ["d", "e"]
a.values_at(0, 2, 4)        # ["a", "c", "e"]

a.include?("c")             # true
a.index("c")                # 2
a.index { |x| x > "b" }     # 2
a.find_index("z")           # nil
a.sample                    # random element
a.shuffle(random: Random.new(1))  # repeatable order
b = a.dup
b.slice!(1, 2)              # ["b", "c"]; b is now ["a", "d", "e"]
Array.new(3).fill(0)        # [0, 0, 0]
```

CODE EXPLAINED

- → `a[1, 2]` — start and length
- → `a[2..]` / `a[..1]` — endless / beginless ranges
- → `take` / `drop` — first n / all but first n
- → `values_at` — pick several indexes at once
- → `slice!` — cut a piece out of the array

THE RUBY WAY

- `a[0...n]` → `a.take(n)` or `a.first(n)`
- `a.index(x) != nil` → `a.include?(x)`
- `[a[0], a[2]]` → `a.values_at(0, 2)`

IN RAILS

- **AS** · `arr.second`, `arr.third` read like English
- **AS** · `arr.from(2)` → elements from index 2; `arr.to(1)` → up to index 1

SLICE FORMS

CALL	RESULT FOR %W[A B C D]
`[1, 2]`	["b", "c"]
`[1..2]`	["b", "c"]
`[-2..]`	["c", "d"]
`[4, 1]`	[] (at end)
`[5, 1]`	nil (past end)

LOOKUP

METHOD	RETURNS
`include?(v)`	true / false
`index(v)`	first position / nil
`rindex(v)`	last position
`count(v)`	occurrences
`sample(n)`	n random items

⚠ COMMON MISTAKES

- Confusing `a[1, 2]` (length) with `a[1..2]` (end index)
- Using `include?` on large arrays in a loop; each call is O(n). Convert to a `Set`
- Calling `shuffle!` / `sample` in tests without a seed and getting flaky results

✓ TIPS & BEST PRACTICES

- `first(n)` and `last(n)` never return `nil`; they return shorter arrays
- `bsearch` finds items in sorted arrays in O(log n) (sheet 060)
- Pass `random: Random.new(seed)` to `shuffle` / `sample` for repeatable output

QUICK SUMMARY

- `a[s, n]` / `a[range]` — slices
- `take` / `drop` — from the front
- `include?` / `index` — lookup
- `sample` / `shuffle` — random

Hash Basics

{ } · key: value · "k" => v · [] vs fetch · Hash.new(0) · Hash.new { } · shared-default trap · {x:, y:} shorthand

HASH BASICS

A **Hash** maps keys to values and keeps **insertion order**. Keys can be any object, but **symbols** (`name: "Ada"`) are the norm for fixed fields and strings for external data. Missing keys return the hash's **default** (usually `nil`); `fetch` raises instead, and `Hash.new` with a block computes defaults on demand.

HASH FACTS

- `{ name: "Ada" }` is shorthand for `{ :name => "Ada" }`
- `{ x:, y: }` uses local variables `x` and `y` as values (3.1+)
- `Hash.new(0)` is great for counting
- `Hash.new([])` shares one array across all keys (a trap)

CREATING AND READING HASHES

```
user = { name: "Ada", age: 36 }      # symbol keys
headers = { "Content-Type" => "json" }  # string keys
user[:name]                  # "Ada"
user[:email]                 # nil
user.fetch(:email, "n/a")    # "n/a"
user.fetch(:email)           # KeyError: key not found: :email
user[:email] = "a@b.c"       # add or update

x, y = 1, 2
point = { x:, y: }           # {x: 1, y: 2}

counts = Hash.new(0)
%w[a b a].each { |w| counts[w] += 1 }
counts                       # {"a" => 2, "b" => 1}

groups = Hash.new { |h, k| h[k] = [] }   # fresh array per key
groups[:odd] << 1
groups                       # {odd: [1]}
user.size                    # 3
user.key?(:age)              # true
```

CODE EXPLAINED

- → `name:` — symbol key; `"k" =>` — any other key type
- → `fetch` — raises or uses a default for missing keys
- → `{ x:, y: }` — value omission shorthand
- → `Hash.new(0)` — default value for counters
- → `Hash.new { |h, k| h[k] = [] }` — default block stores per key

THE RUBY WAY

- `{ :name => "Ada" }` → `{ name: "Ada" }`
- `h[k] = [] unless h[k]; h[k] << v` → `Hash.new { |h, k| h[k] = [] }`
- `h[:required_key]` → `h.fetch(:required_key)`

IN RAILS

- **Rails** · `params` is `ActionController::Parameters`; call `permit` before mass assignment
- **AS** · `h.with_indifferent_access[:name]` and `["name"]` both work

SYNTAX

LITERAL	KEY TYPE
`{ a: 1 }`	Symbol :a
`{ "a" => 1 }`	String
`{ 1 => "x" }`	Integer
`{ [1, 2] => :pt }`	Array
`{ x:, y: }`	from locals

DEFAULTS

CONSTRUCT	MISSING KEY GIVES
`{}`	nil
`Hash.new(0)`	0 (not stored)
`Hash.new { |h, k| h[k] = [] }`	new [] (stored)
`h.fetch(k)`	KeyError
`h.fetch(k, d)`	d

⚠ COMMON MISTAKES

- `Hash.new([])` then `h[:a] << 1`; every key shares that one array and the hash stays empty
- Mixing `"name"` and `:name` keys and getting `nil` lookups
- Using string keys from JSON with symbol lookups; parse with `symbolize_names: true`

✓ TIPS & BEST PRACTICES

- Prefer `fetch` for required keys; the `KeyError` names the missing key
- Hashes preserve insertion order, so they work as ordered maps
- Freeze config hashes (`CONFIG = {...}.freeze`) to prevent accidental changes

QUICK SUMMARY

- `{ key: value }` — symbol-key hash
- `h[k]` / `fetch` — soft / strict lookup
- `Hash.new(0)` — counter default
- `Hash.new { }` — computed default

Hash Methods

HASH METHODS

Hashes come with methods to query keys and values, **merge** hashes together, **slice** out subsets, **delete** entries, and dig into nested data. `merge` returns a new hash, while `merge!` (alias `update`) changes the receiver. A block passed to `merge` resolves conflicting keys.

NOTES

- `key?` / `include?` / `member?` are the same check
- `merge` — later hashes win on duplicate keys
- `slice` keeps listed keys; `except` drops them (3.0+)
- `delete` returns the removed value

QUERY, MERGE, SLICE

```
h = { a: 1, b: 2, c: 3 }
h.keys                          # [:a, :b, :c]
h.values                        # [1, 2, 3]
h.key?(:b)                      # true
h.value?(3)                     # true
h.key(2)                        # :b (reverse lookup)

h.merge(c: 30, d: 4)            # {a: 1, b: 2, c: 30, d: 4}
h.merge({ a: 5 }) { |k, old, new| old + new }  # {a: 6, b: 2, c: 3}
h.slice(:a, :c)                 # {a: 1, c: 3}
h.except(:a)                    # {b: 2, c: 3}
h.delete(:b)                    # 2 (h is now {a: 1, c: 3})

data = { user: { address: { city: "Oslo" } } }
data.dig(:user, :address, :city)   # "Oslo"
data.dig(:user, :phone, :area)     # nil
h.to_a                          # [[:a, 1], [:c, 3]]
h.each_pair { |k, v| puts "#{k}=#{v}" }
h.any? { |k, v| v > 2 }         # true
```

CODE EXPLAINED

- → `key(2)` — find the key for a value
- → `merge` with a block — combine values on conflict
- → `slice` / `except` — keep / drop keys
- → `delete` — remove and return the value
- → `dig` — nil-safe nested lookup

THE RUBY WAY

- `h.keys.include?(k)` → `h.key?(k)`
- `h.reject { |k, _| k == :a }` → `h.except(:a)`
- `h[:a] && h[:a][:b]` → `h.dig(:a, :b)`

IN RAILS

- **AS** · `deep_merge` merges nested hashes; `reverse_merge` fills in defaults
- **AS** · `symbolize_keys` / `stringify_keys` ; `except` began in ActiveSupport before core

QUERY

METHOD	RETURNS
`keys / values`	arrays
`key?(k)`	true / false
`value?(v)`	true / false
`key(v)`	key or nil
`size / empty?`	count / boolean

CHANGE

METHOD	IN PLACE?
`merge(other)`	no
`merge!(other) / update`	yes
`delete(k)`	yes
`slice / except`	no
`compact`	no (removes nil values)

⚠ COMMON MISTAKES

- Expecting `merge` to modify the hash; it returns a new one. Use `merge!` or reassign
- Using `merge` for nested config; inner hashes are replaced, not merged
- Checking `h[k].nil?` to test presence; a key can exist with a `nil` value. Use `key?`

✓ TIPS & BEST PRACTICES

- `h.merge(defaults) { |_, mine, _| mine }` keeps existing values, like `reverse_merge`
- `filter_map` / `to_h` and `transform_values` cover most reshaping (sheet 056)
- `dig` also works across arrays: `data.dig(:items, 0, :id)`

QUICK SUMMARY

- `keys` / `values` / `key?` — query
- `merge` / `merge!` — combine
- `slice` / `except` — subsets
- `dig` — nested lookup

Hash Transformations

HASH TRANSFORMATIONS

Ruby makes it easy to reshape hashes: **`transform_values`** and **`transform_keys`** map one side, **`select / reject / filter_map`** filter, and **`to_h with a block`** builds a hash from any enumerable. `group_by`, `tally` and `sort_by` turn arrays into summary hashes.

NOTES

- `select` / `reject` on a Hash return a Hash
- `map` on a Hash returns an **Array**; use `to_h` to get a hash back
- `sort_by` returns an array of pairs
- `min_by` / `max_by` return a `[key, value]` pair

RESHAPING HASHES

```
prices = { apple: 1.2, pear: 0.8, kiwi: 2.5 }
prices.transform_values { (it * 100).round }  # {apple: 120, ...}
prices.transform_keys(&:to_s)         # {"apple" => 1.2, ...}
prices.select { |_, p| p > 1 }        # {apple: 1.2, kiwi: 2.5}
prices.reject { |_, p| p > 1 }        # {pear: 0.8}
prices.filter_map { |k, p| k if p < 2 }   # [:apple, :pear]
prices.to_h { |k, p| [k.upcase, p] }     # {APPLE: 1.2, ...}
prices.sort_by { |_, p| p }.first     # [:pear, 0.8]
prices.max_by { |_, p| p }            # [:kiwi, 2.5]
prices.sum { |_, p| p }               # 4.5

words = %w[ant bee cat ape]
words.group_by { it[0] }              # {"a" => ["ant", "ape"], ...}
words.map(&:size).tally               # {3 => 4}
words.each_with_index.to_h            # {"ant" => 0, "bee" => 1, ...}
words.zip(1..).to_h                   # {"ant" => 1, "bee" => 2, ...}
```

CODE EXPLAINED

- → `transform_values` — new hash, same keys
- → `select` / `reject` — hash in, hash out
- → `to_h { |k, v| [k2, v2] }` — map into a new hash
- → `group_by` — key → array of members
- → `tally` — counts of each value

THE RUBY WAY

- `h.map { |k, v| [k, v * 2] }.to_h` → `h.transform_values { it * 2 }`
- `h.each_with_object({}) { ... }` for simple maps → `to_h { }`
- `Hash.new(0)` counting loops → `tally`

IN RAILS

- AS · `users.index_by(&:id)` → `{1 => user1, ...}`; `ids.index_with { 0 }` → `{1 => 0, ...}`
- AS · `deep_transform_keys(&:to_s)` converts nested keys too

TRANSFORM

METHOD	RETURNS
`transform_values`	Hash
`transform_keys`	Hash
`to_h { }`	Hash
`map`	Array
`filter_map`	Array

SUMMARIZE

METHOD	RETURNS
`group_by { }`	{key => [items]}
`tally`	{item => count}
`sort_by { }`	[[k, v], ...]
`min_by / max_by`	[k, v]
`sum { }`	number

⚠ COMMON MISTAKES

- Calling `map` on a hash and expecting a hash back; you get an array of whatever the block returns
- Sorting a hash with `sort_by` and forgetting to call `to_h` if you need a hash again
- `transform_keys!` on a frozen hash, or one shared with other code

✓ TIPS & BEST PRACTICES

- `transform_keys(a: :alpha)` renames keys from a mapping hash (3.0+)
- `group_by(...).transform_values(&:count)` is a quick histogram, though `tally` is simpler
- `each_with_object(Hash.new(0))` still wins for complex accumulation

QUICK SUMMARY

- `transform_values` / `transform_keys` — map one side
- `select` / `reject` — filter, keep a hash
- `to_h { }` — build a hash
- `group_by` / `tally` — summarize

Ranges

RANGES

A **Range** represents an interval: `1..10` includes the end, `1...10` excludes it. Ranges can be **endless** (`1..`) or **beginless** (`..5`), work with any comparable values (numbers, strings, dates), and support iteration, membership tests and stepping. They're used for slicing, `case/when`, and conditions.

RANGE FACTS

- `cover?` tests `begin <= x <= end` (fast, no iteration)
- `include?` iterates for non-numeric ranges
- `(1..10) % 3` steps by 3 (Ruby 3.0+)
- `to_a` on an endless range raises (infinite)

RANGE BASICS

```ruby
r = 1..10
r.to_a                      # [1, 2, ..., 10]
(1...10).size               # 9
r.sum                       # 55
r.step(3).to_a              # [1, 4, 7, 10]
((1..10) % 4).to_a          # [1, 5, 9]
(10).downto(8).to_a         # [10, 9, 8]

r.cover?(5.5)               # true
r.include?(11)              # false
("a".."e").to_a             # ["a", "b", "c", "d", "e"]
("a".."z").cover?("cat")    # true (compares strings)
("a".."z").include?("cat")  # false (not an element)

arr = %w[a b c d e]
arr[1..]                    # ["b", "c", "d", "e"]
arr[..-2]                   # ["a", "b", "c", "d"]
(1..).each { |n| break n if n * n > 50 }  # 8
age = 42
case age when ..17 then :minor when 18.. then :adult end  # :adult
(1..10).minmax              # [1, 10]
```

CODE EXPLAINED

- `1..10` / `1...10` — inclusive / exclusive end
- `step(3)` / `% 4` — every nth value
- `cover?` — range comparison; `include?` — membership
- `arr[1..]` — slicing with endless ranges
- `(1..).each` — infinite sequence, stop with `break`

THE RUBY WAY

- `x >= 1 && x <= 10` → `(1..10).cover?(x)`
- `(0..arr.size - 1)` → `arr.each_index` or `(0...arr.size)`
- `if age < 18 … elsif age >= 65` → `case age when ..17 … when 65..`

IN RAILS

- **Rails** · `User.where(created_at: 1.week.ago..)` builds `created_at >= ?`
- **AS** · `Date.current.all_week` returns the range of this week's dates

RANGE FORMS

LITERAL	CONTAINS
`1..5`	1, 2, 3, 4, 5
`1...5`	1, 2, 3, 4
`1..`	1 and up
`..5`	up to 5
`"a".."c"`	"a", "b", "c"

RANGE METHODS

METHOD	RESULT FOR 1..10
`first(3) / last(2)`	[1, 2, 3] / [9, 10]
`min / max`	1 / 10
`size`	10
`cover?(1..3)`	true
`overlap?(9..12)`	true (3.3+)

⚠ COMMON MISTAKES

- Using `...` and forgetting the end is excluded: `(1...10).to_a.last` is `9`
- Calling `include?` on string ranges and getting `false` for strings that `cover?` accepts
- Writing `1..10.each` ; method calls bind tighter, so use `(1..10).each`

✓ TIPS & BEST PRACTICES

- Use ranges in `case/when` for buckets like grades or age groups
- `rand(1..6)` picks a random integer in a range
- `Range#step` with floats is precise: `(0.0..1.0).step(0.25).to_a`

QUICK SUMMARY

- `..` / `...` — inclusive / exclusive
- `1..` / `..5` — endless / beginless
- `cover?` — fast bounds check
- `step` / `%` — stepping

Sets

SETS

A **Set** is an unordered collection of **unique** elements with fast membership tests. In Ruby 4.0 `Set` is a **core class** reimplemented in C, so no `require "set"` is needed (and it's faster). Sets support union, intersection, difference and subset checks, and compare elements with `eql?` / `hash` like Hash keys.

SET FACTS

- `include?` is O(1) on average (vs O(n) for arrays)
- Duplicates are silently ignored on `add`
- `add?` returns `nil` if the element was already present
- Sets keep insertion order when iterated

USING SETS

```ruby
require "set"                 # needed before 3.2; harmless in 4.0
tags = Set[:ruby, :rails]
tags << :ruby                 # still 2 elements
tags.add?(:ruby)              # nil (already there)
tags.add?(:web)               # Set[:ruby, :rails, :web]
tags.include?(:rails)         # true
tags.size                     # 3

a = Set[1, 2, 3]
b = Set[2, 3, 4]
a | b                         # Set[1, 2, 3, 4]
a & b                         # Set[2, 3]
a - b                         # Set[1]
a ^ b                         # Set[1, 4]
Set[2, 3].subset?(a)          # true
a.superset?(Set[1])           # true
a.disjoint?(Set[9])           # true

[1, 1, 2].to_set              # Set[1, 2]
seen = Set.new
%w[a b a c].reject { !seen.add?(it) }  # ["a", "b", "c"]
```

CODE EXPLAINED

- → `Set[...]` — literal-style constructor
- → `<<` / `add` — duplicates ignored
- → `add?` — `nil` when already present; handy for "seen" checks
- → `\|` `&` `-` `^` — union, intersection, difference, symmetric difference
- → `subset?` / `superset?` / `disjoint?` — relationships

THE RUBY WAY

- `arr.include?(x)` inside a loop → build a `Set` once
- `arr.uniq` after every insert → use a `Set`
- `(a & b).empty?` with sets → `a.disjoint?(b)`

OPERATORS

OP	RESULT FOR {1,2,3} OP {2,3,4}
`|`	{1, 2, 3, 4}
`&`	{2, 3}
`-`	{1}
`^`	{1, 4}

SET VS ARRAY

NEED	USE
`unique items`	Set
`fast include?`	Set
`order + duplicates`	Array
`index access`	Array
`JSON output`	Array (`to_a`)

⚠ COMMON MISTAKES

- Putting mutable objects (like arrays) in a set and then changing them; their hash changes and lookups fail
- Expecting `set[0]`; sets have no index. Convert with `to_a` first
- Serializing a Set to JSON directly; call `to_a` first

✓ TIPS & BEST PRACTICES

- Use a `Set` for allow-lists and "visited" tracking in graph searches
- `SortedSet` was removed from the stdlib; sort with `to_a.sort` or use a gem
- Freeze lookup sets in constants: `ALLOWED = Set[:get, :post].freeze`

QUICK SUMMARY

- `Set[...]` — unique, unordered
- `include?` — O(1) membership
- `\|` `&` `-` `^` — set algebra
- core class in 4.0 — no require needed

Struct & Data

STRUCT AND DATA

`Struct.new` builds a small class with accessors, equality, `to_h` and pattern matching in one line, and its fields are **mutable**. **`Data.define`** (3.2+) builds **immutable** value objects: no setters, keyword or positional construction, and `with` to copy with changes. Both accept a block to add methods.

NOTES

- Struct fields have setters; Data fields don't
- Data requires every field (unless you add defaults in `initialize`)
- Both define `==` by value, not identity
- Both support `deconstruct` / `deconstruct_keys`

STRUCT AND DATA

```ruby
Point = Struct.new(:x, :y) do
  def dist = Math.hypot(x, y)
end
p = Point.new(3, 4)
p.x                          # 3
p.x = 6                      # mutable
p.dist                       # 7.211102550927978
p.to_a                       # [6, 4]
Point.new(1, 2) == Point.new(1, 2)   # true
Point.new(x: 1, y: 2)        # keyword init works (3.2+)

Money = Data.define(:amount, :currency) do
  def to_s = format("%.2f %s", amount, currency)
end
m = Money.new(amount: 5, currency: "USD")
m.to_s                       # "5.00 USD"
m.amount = 6                 # NoMethodError (immutable)
m.with(amount: 9)            # #<data Money amount=9, currency="USD">
m.to_h                       # {amount: 5, currency: "USD"}
m => { amount: }; amount     # 5 (pattern matching)
```

CODE EXPLAINED

- → `Struct.new(:x, :y) do … end` — class with accessors + methods
- → `p.x = 6` — Struct fields are writable
- → `Data.define` — read-only value object
- → `with(amount: 9)` — copy with one field changed
- → pattern matching — both work in `case/in`

THE RUBY WAY

- Tiny classes with only `attr_accessor` + `initialize` → `Struct`
- Value objects with hand-written `==` → `Data.define`
- Passing loose hashes around → named `Data` objects

IN RAILS

- **Rails** · `include ActiveModel::Model` gives validations and form helpers to plain classes
- **Rails** · `ActiveModel::Attributes` adds typed `attribute :age, :integer`

STRUCT VS DATA

FEATURE	STRUCT	DATA
`setters`	yes	no
`positional init`	yes	yes
`keyword init`	yes	yes
`missing field`	nil	ArgumentError
`with(...)`	no	yes

GENERATED METHODS

METHOD	GIVES
`members`	[:x, :y]
`to_h`	hash of fields
`to_a / deconstruct`	array of values
`== / eql? / hash`	by value
`inspect`	readable output

⚠ COMMON MISTAKES

- Subclassing with `class Point < Struct.new(:x, :y)`; it creates an extra anonymous class. Assign to a constant with a block instead
- Expecting `Data` to accept missing fields; every field is required
- Using a mutable Struct as a hash key and then changing a field

✓ TIPS & BEST PRACTICES

- Prefer `Data` for anything that shouldn't change after creation (money, coordinates, results)
- Add defaults to Data with `def initialize(amount:, currency: "USD") = super`
- Struct's `keyword_init: true` is no longer needed; both styles work since 3.2

QUICK SUMMARY

- `Struct.new` — quick mutable record
- `Data.define` — immutable value object
- `with` — copy with changes
- `to_h` / pattern matching — built in

Sorting & Searching

SORTING AND SEARCHING

`sort` uses <=> on the elements; **`sort_by`** sorts by a computed key and is usually faster and clearer. `min`, `max`, `minmax` and their `_by` forms find extremes without sorting. On a sorted array, **`bsearch`** finds elements in O(log n). Ruby's sort is **not guaranteed stable**.

NOTES

- `sort` / `sort_by` return new arrays; `!` versions sort in place
- Descending: `sort_by { -it }` for numbers or `sort.reverse`
- Multi-key: return an array from the block
- `max(2)` returns the two largest

SORTING AND SEARCHING

```ruby
nums = [5, 3, 9, 1]
nums.sort                          # [1, 3, 5, 9]
nums.sort.reverse                  # [9, 5, 3, 1]
nums.sort { |a, b| b <=> a }       # [9, 5, 3, 1]
nums.max                           # 9
nums.min(2)                        # [1, 3]
nums.minmax                        # [1, 9]

words = %w[pear Fig apple cherry]
words.sort_by(&:length)            # ["Fig", "pear", "apple", "cherry"]
words.sort_by(&:downcase)          # ["apple", "cherry", "Fig", "pear"]
words.max_by(&:length)             # "cherry"

people = [["Bo", 30], ["Al", 30], ["Cy", 25]]
people.sort_by { |name, age| [-age, name] }
# => [["Al", 30], ["Bo", 30], ["Cy", 25]]

sorted = [1, 3, 5, 7, 9]
sorted.bsearch { it >= 6 }         # 7
sorted.bsearch_index { it >= 6 }   # 3
words.each_with_index.sort_by { |w, i| [w.size, i] }  # stable
```

CODE EXPLAINED

- → `sort { |a, b| b <=> a }` — custom comparator
- → `sort_by(&:length)` — sort by a key
- → `[-age, name]` — multi-key: age descending, then name
- → `bsearch { it >= 6 }` — first element meeting the condition
- → `with index` key — forces a stable order

THE RUBY WAY

- `sort { |a, b| a.size <=> b.size }` → `sort_by(&:size)`
- `sort.first` / `sort.last` → `min` / `max`
- `sort_by { }.reverse` for numbers → `sort_by { -it }`

IN RAILS

- **Rails** · `User.order(:name)` sorts in SQL; `users.sort_by(&:name)` loads everything into Ruby
- **Rails** · `maximum(:age)` / `minimum(:age)` run as SQL aggregates

SORT METHODS

METHOD	USE
`sort`	natural order
`sort { |a, b| }`	custom compare
`sort_by { }`	by key
`sort! / sort_by!`	in place
`reverse`	flip order

FIND EXTREMES

METHOD	RETURNS
`min / max`	one element
`min(n) / max(n)`	n elements
`minmax`	[min, max]
`min_by / max_by`	by key
`bsearch { }`	O(log n) on sorted

⚠ COMMON MISTAKES

- Sorting mixed types; `<=>` returns `nil` and `sort` raises `ArgumentError`
- Calling `bsearch` on an unsorted array; it returns wrong answers without raising
- Assuming equal elements keep their order; add the index as a tie-breaker when it matters

✓ TIPS & BEST PRACTICES

- `sort_by` computes each key once, so it beats `sort` with an expensive block
- Use `max_by(n)` instead of sorting the whole array when you only need the top few
- `Comparable#clamp` and `between?` handle range checks without sorting

QUICK SUMMARY

- `sort` / `sort_by` — order elements
- `min` / `max` / `minmax` — extremes
- `[a, b]` keys — multi-key sort
- `bsearch` — fast search in sorted arrays

Classes & Objects

CLASSES AND OBJECTS

A class is a blueprint defined with `class Name ... end`; **new** allocates an object and calls **initialize**. State lives in **instance variables** (@name), which are private to the object and spring into existence when first assigned. Inside methods, **self** is the current object.

CLASS FACTS

- Class names are constants: `CamelCase`
- `@ivars` are `nil` until assigned (no error on read)
- Every class inherits from `Object` unless told otherwise
- Override `to_s` for `puts` and `inspect` for `p`

A COMPLETE SMALL CLASS

```ruby
class BankAccount
  def initialize(owner, balance = 0)
    @owner = owner
    @balance = balance
  end

  def deposit(amount)
    @balance += amount
    self                    # return self to allow chaining
  end

  def to_s = "#{@owner}: $#{@balance}"
  def inspect = "#<BankAccount #{@owner}>"
end

acct = BankAccount.new("Ada", 100)
acct.deposit(50).deposit(25)
puts acct                   # Ada: $175
p acct                      # #<BankAccount Ada>
acct.instance_variables     # [:@owner, :@balance]
```

CODE EXPLAINED

→ `initialize` — runs on `BankAccount.new(...)`

→ `@balance` — per-object state, hidden from outside

→ `self` — returning it enables method chaining

→ `to_s` / `inspect` — control `puts` and `p` output

→ `instance_variables` — introspect an object's state

THE RUBY WAY

- `def get_balance; return @balance; end` → `attr_reader :balance` (sheet 062)
- Classes with only data → `Struct` or `Data` (sheet 059)
- Printing `#<BankAccount:0x000…>` → define `to_s` / `inspect`

IN RAILS

- **Rails** · models are plain classes: `class User < ApplicationRecord`
- **Rails** · don't define `initialize` on models; use callbacks or `after_initialize`

OBJECT BASICS

CODE	RESULT
`Cls.new(args)`	calls initialize
`obj.class`	its class
`obj.is_a?(Cls)`	type check
`obj.object_id`	identity number
`obj.dup`	shallow copy

INSIDE A CLASS

THING	MEANING
`@x`	instance variable
`self`	current object
`def m`	instance method
`def self.m`	class method (sheet 063)
`CONST`	class constant

⚠ COMMON MISTAKES

- Typo in an instance variable name (`@balnce`); it's silently `nil` instead of an error
- Calling `BankAccount.initialize` ; you always call `new` , which calls `initialize`
- Forgetting that `p obj` uses `inspect` , so overriding only `to_s` doesn't change `p` output

✓ TIPS & BEST PRACTICES

- Keep `initialize` simple: assign arguments, validate, nothing more
- Return `self` from mutating methods when chaining reads naturally
- Use `Data.define` when objects shouldn't change after creation

QUICK SUMMARY

- `class … end` — define a class
- `new` → `initialize` — construct
- `@ivar` — per-object state
- `to_s` / `inspect` — printable forms

Attribute Accessors

attr_reader · attr_writer · attr_accessor · custom setters · self.x = · validation in setters

ATTRIBUTE ACCESSORS

Instance variables are private, so classes expose them through methods. **attr_reader** generates getters, **attr_writer** generates setters (name=), and **attr_accessor** both. Write a custom setter when you need validation, and call setters inside the class with **self.name =** so Ruby doesn't create a local variable instead.

ACCESSOR NOTES

- `attr_reader :a, :b` takes several names
- Setters always return the assigned value
- `self.x = 1` calls the setter; `x = 1` creates a local
- Getters can be made private: `private attr_reader :token`

READERS, WRITERS, CUSTOM SETTERS

```ruby
class Product
  attr_reader :name, :price       # getters
  attr_accessor :stock            # getter + setter

  def initialize(name, price)
    @name = name
    self.price = price            # use the validating setter
  end

  def price=(value)               # custom setter
    raise ArgumentError, "negative price" if value.negative?
    @price = value.round(2)
  end

  def restock(n)
    self.stock = (stock || 0) + n # self. required for setters
  end
end

p1 = Product.new("Pen", 1.239)
p1.price                          # 1.24
p1.stock = 5
p1.restock(3)                     # 8
p1.price = -1                     # ArgumentError
```

CODE EXPLAINED

- → `attr_reader` — `def name = @name`
- → `attr_accessor` — reader + writer
- → `self.price = price` — runs the validating setter
- → `def price=(value)` — custom setter with checks
- → `self.stock =` — without `self`, Ruby makes a local `stock`

THE RUBY WAY

- Hand-written `def name; @name; end` → `attr_reader :name`
- `attr_accessor` on everything → expose only what callers need
- `@price = v` inside `initialize` that skips validation → `self.price = v`

IN RAILS

- **Rails** · each table column becomes reader/writer methods on the model automatically
- **AS** · `mattr_accessor` / `cattr_accessor` define module- or class-level accessors

MACROS

MACRO	GENERATES
`attr_reader :x`	x
`attr_writer :x`	x=
`attr_accessor :x`	x and x=

SETTER CALLS

CODE	EFFECT
`obj.x = 5`	calls x=
`self.x = 5`	calls x= inside the class
`x = 5`	local variable!

⚠ COMMON MISTAKES

- Writing `stock = 10` inside a method; it creates a local and the object never changes
- Expecting a custom setter's return value; `obj.price = 5` always evaluates to `5`
- Exposing `attr_accessor` for internal state; callers can then break invariants

✓ TIPS & BEST PRACTICES

- Use `attr_reader` by default and add writers only when needed
- Predicate readers are just methods: `def active? = @active`
- `attr_reader` returns the method names, so `private attr_reader :x` works in one line

QUICK SUMMARY

- `attr_reader` / `attr_writer` / `attr_accessor` — generate methods
- custom `x=` — validated setter
- `self.x =` — call setter inside the class
- `private attr_reader` — hidden getter

Class Methods & Class-Level State

def self.x · class << self · @@class vars · class instance vars · constants · factory methods

CLASS METHODS AND CLASS-LEVEL STATE

Class methods are called on the class itself (`User.find`) and are defined with `def self.name` or inside `class << self`. Class-level data lives in **class instance variables** (`@count` at class level), in **class variables** (`@@count`, shared with subclasses), or in **constants**. Factory methods that return new instances are a common use.

NOTES

- `self` inside a class body (outside methods) is the class
- `@@vars` are shared across the whole inheritance tree (often surprising)
- Class instance variables are per class, not inherited
- Constants are looked up lexically and through ancestors

CLASS METHODS AND STATE

```ruby
class Temperature
  SCALES = %i[c f].freeze           # constant
  @created = 0                      # class instance variable

  class << self                     # block of class methods
    attr_reader :created
    def from_f(f) = new((f - 32) * 5 / 9.0)
  end

  def self.boiling = new(100)       # single class method

  def initialize(c)
    @c = c
    self.class.instance_variable_set(:@created, self.class.created + 1)
  end
  attr_reader :c
end

Temperature.from_f(212).c           # 100.0
Temperature.boiling.c               # 100
Temperature.created                 # 2
Temperature::SCALES                 # [:c, :f]
```

CODE EXPLAINED

- → `class << self` — opens the class's singleton class
- → `def self.boiling` — one class method
- → `from_f` — factory method returning `new(...)`
- → `@created` at class level — belongs to the class object
- → `Temperature::SCALES` — read a class constant

THE RUBY WAY

- `@@count` for per-class counters → class instance variables
- Many `def self.` methods → group them in `class << self`
- `Temperature.new(...)` with conversion logic at call sites → named factories

IN RAILS

- **Rails** · `scope :recent, -> { order(created_at: :desc) }` vs `def self.recent` — scopes always return a relation
- **AS** · `class_attribute :api_key` is an inheritable, overridable class-level setting

WAYS TO DEFINE

FORM	USE
`def self.m`	one method
`class << self … end`	several methods
`extend Module`	add module methods
`private_class_method :new`	hide new

CLASS-LEVEL DATA

KIND	INHERITED?
`CONST`	yes (lookup)
`@@cvar`	shared with subclasses
`@civar`	no (per class)
`class_attribute (AS)`	copied, overridable

⚠ COMMON MISTAKES

- Using `@@vars` and having a subclass silently change the parent's value
- Calling an instance method from a class method (or vice versa) without a receiver
- Mutating constants (`SCALES << :k`); freeze them so it raises instead

✓ TIPS & BEST PRACTICES

- `private_class_method :new` plus factories forces callers through validated constructors
- `Module#singleton_methods` (or `methods(false)` on the class) lists class methods while debugging
- Prefer class instance variables with `class << self; attr_accessor` for configuration

QUICK SUMMARY

- `def self.m` / `class << self` — class methods
- factory methods — named constructors
- `@civar` vs `@@cvar` — per class vs shared
- `CONST` — class constants

Visibility

VISIBILITY

Methods are **public** by default. **`private`** methods can only be called without an explicit receiver (except `self.x =` setters), so they're internal helpers. **`protected`** methods can be called on other instances of the same class, which is useful for comparisons. Visibility can be set as a section or per method.

NOTES

- `private` with no args affects every method below it
- `private def helper` marks a single method
- Private methods can be called with `self.` since Ruby 2.7
- `send` bypasses visibility (use `public_send` to respect it)

PUBLIC, PRIVATE, PROTECTED

```ruby
class Account
  def initialize(balance) = @balance = balance
  def >(other) = balance > other.balance    # protected reader
  def summary = "Balance: #{formatted}"     # private helper

  protected

  attr_reader :balance

  private

  def formatted = format("$%.2f", @balance)
end

a, b = Account.new(50), Account.new(20)
a > b                   # true
a.summary               # "Balance: $50.00"
a.balance               # NoMethodError: protected method
a.formatted             # NoMethodError: private method
a.send(:formatted)      # "$50.00" (bypasses)
```

CODE EXPLAINED

- → `other.balance` — protected: allowed on another `Account`
- → `formatted` — private: only callable without a receiver
- → `protected` / `private` sections — apply to methods below
- → `send` — ignores visibility (debugging, tests)

THE RUBY WAY

- Prefixing helpers with `_` to mean private → actually use `private`
- `private` sections far from the method → `private def name`
- `send` in app code to reach internals → expose a public method

IN RAILS

- **Rails** · only public controller methods become actions; put helpers under `private`
- **Rails** · `before_action :load_user` usually points at a private method

VISIBILITY

LEVEL	CALLABLE FROM
`public`	anywhere
`protected`	same class family, any instance
`private`	inside, implicit receiver (or self.)

SYNTAX

FORM	EFFECT
`private`	section below
`private def m`	one method
`private attr_reader :x`	private getter
`private_class_method :new`	class method

⚠ COMMON MISTAKES

- Placing `attr_reader` after `private` by accident and making a public API private
- Expecting `private` to affect class methods defined with `def self.`; use `private_class_method`
- Thinking `protected` means "subclasses only" as in Java; in Ruby it's about receivers of the same family

✓ TIPS & BEST PRACTICES

- Keep the public interface small; everything else goes under `private`
- Use `protected` mainly for comparison methods (`==`, `<=>`) that read other instances
- Tests should exercise public behavior rather than `send`-ing to private methods

QUICK SUMMARY

- `public` — default
- `private` — internal helpers, implicit receiver
- `protected` — same-family instances
- `send` — bypasses visibility

Inheritance & super

INHERITANCE AND SUPER

A class inherits from one parent with `<`, gaining its methods and overriding any it redefines. **super** calls the parent's version of the current method: bare `super` passes **the same arguments**, `super()` passes **none**, and `super(a, b)` passes exactly those. Method lookup walks the **ancestors** chain.

INHERITANCE FACTS

- Single inheritance only; share behavior across classes with modules
- Default parent is `Object`, whose parent is `BasicObject`
- `super` also forwards the block automatically
- `ancestors` shows the full lookup order (including modules)

OVERRIDING AND SUPER

```
class Animal
  attr_reader :name
  def initialize(name) = @name = name
  def speak = "..."
  def to_s = "#{self.class.name.downcase} #{name}"
end

class Dog < Animal
  def initialize(name, breed)
    super(name)               # parent's initialize(name)
    @breed = breed
  end

  def speak = "Woof"          # override
  def to_s = super + " (#{@breed})"   # bare super: same args
end

d = Dog.new("Rex", "lab")
d.speak                       # "Woof"
d.to_s                        # "dog Rex (lab)"
Dog.superclass                # Animal
Dog.ancestors.take(3)         # [Dog, Animal, Object]
```

CODE EXPLAINED

- → `Dog < Animal` — single inheritance
- → `super(name)` — pass only what the parent expects
- → `super` (bare) — forwards the current arguments
- → `ancestors` — method lookup order

THE RUBY WAY

- Copy-pasting methods between classes → a parent class or a module
- Deep inheritance trees → composition or modules
- `super()` vs `super` confusion → be explicit with arguments in `initialize`

IN RAILS

- **Rails** · models inherit from `ApplicationRecord`, controllers from `ApplicationController`
- **Rails** · single-table inheritance: a `type` column makes `class Admin < User` share one table

SUPER FORMS

FORM	PASSES
`super`	same args + block
`super()`	no args (block still passed)
`super(a, b)`	exactly a, b

HIERARCHY TOOLS

CALL	RETURNS
`Cls.superclass`	parent class
`Cls.ancestors`	lookup chain
`Cls < Other`	true if subclass
`Cls.subclasses`	direct subclasses (3.1+)

⚠ COMMON MISTAKES

- Calling bare `super` in an `initialize` with different parameters; the parent gets the wrong number of args
- Forgetting `super` in `initialize`, so the parent's instance variables are never set
- Using inheritance for code reuse between unrelated classes; prefer a mixin (sheet 066)

✓ TIPS & BEST PRACTICES

- Prefer shallow hierarchies (1–2 levels); combine with modules for shared behavior
- `method(:speak).super_method` shows the next implementation up the chain
- Call `super` at the start of `initialize` so parent state exists before you use it

QUICK SUMMARY

- `class B < A` — inherit
- override — redefine a method
- `super` / `super()` / `super(x)` — call parent
- `ancestors` — lookup order

Modules as Mixins

MODULES AS MIXINS

A **module** groups methods that can be mixed into classes. **`include`** adds the module's methods as **instance** methods, **`extend`** adds them as methods on a single object (often the class, giving class methods), and **`prepend`** inserts the module **before** the class so it can wrap existing methods with `super`.

MIXIN FACTS

- A class can include any number of modules
- Included modules appear in `ancestors` after the class
- Prepended modules appear before the class
- `Comparable` and `Enumerable` are core mixins

INCLUDE, EXTEND, PREPEND

```ruby
module Greet
  def greet = "Hi, I'm #{name}"
end
module Loud
  def greet = super.upcase + "!"   # wraps the original
end
module Registry
  def all = @all ||= []
end

class Person
  include Greet                    # instance methods
  prepend Loud                     # runs before Person#greet
  extend Registry                  # class methods
  attr_reader :name
  def initialize(name) = (@name = name; self.class.all << self)
end

Person.new("Ada").greet            # "HI, I'M ADA!"
Person.all.size                    # 1
Person.ancestors.take(3)           # [Loud, Person, Greet]
```

CODE EXPLAINED

- → `include Greet` — `greet` becomes an instance method
- → `prepend Loud` — `Loud#greet` runs first and calls `super`
- → `extend Registry` — `Person.all` becomes a class method
- → `ancestors` — prepend before, include after the class

THE RUBY WAY

- Copying the same methods into several classes → a module + `include`
- `alias_method` chains to wrap methods → `prepend` with `super`
- A class only used to hold helper methods → a module

IN RAILS

- **AS** · `ActiveSupport::Concern` adds `included do … end` and `class_methods do … end`
- **Rails** · concerns live in `app/models/concerns` and are included like any module

MIXING IN

KEYWORD	METHODS BECOME
`include M`	instance methods
`extend M`	methods on that object / class
`prepend M`	instance methods, before the class

LOOKUP ORDER

FOR `class C; prepend P; include I; end`	POSITION
`P`	1
`C`	2
`I`	3
`superclass`	4

⚠ COMMON MISTAKES

- Using `include` and expecting class methods; use `extend` (or a `self.included` hook)
- Two modules defining the same method; the last one included wins, silently
- Mixing in modules that depend on methods the class doesn't define; document the contract (e.g. "requires `each`")

✓ TIPS & BEST PRACTICES

- Implement `<=>` and `include Comparable`; implement `each` and `include Enumerable`
- The `self.included(base)` hook lets a module run setup or `base.extend(ClassMethods)`
- Keep modules focused on one behavior; name them as adjectives (`Comparable`, `Printable`)

QUICK SUMMARY

- `include` — instance methods
- `extend` — class/object methods
- `prepend` — wrap with `super`
- `Comparable` / `Enumerable` — core mixins

Modules as Namespaces

MODULES AS NAMESPACES

Modules also **group constants**: classes, other modules and values nested inside a module are referenced with `::` (`Billing::Invoice`). Namespaces avoid name clashes between libraries. `module_function` and `extend self` turn a module into a bag of utility functions you call directly on the module.

NAMESPACE NOTES

- `module A; class B; end; end` defines `A::B`
- `::Name` forces top-level lookup
- Constant lookup checks lexical scope first, then ancestors

NAMESPACES AND UTILITY MODULES

```ruby
module Billing
  TAX_RATE = 0.08
  class Invoice
    def total(amt) = amt * (1 + TAX_RATE)  # sees TAX_RATE
  end
  module Formatter
    module_function                 # callable as Formatter.money
    def money(n) = format("$%.2f", n)
  end
end

Billing::Invoice.new.total(100)     # 108.0
Billing::Formatter.money(5)         # "$5.00"

module Slug
  extend self                       # all methods on the module
  def call(s) = s.downcase.gsub(/\W+/, "-")
end
Slug.call("Hello World")            # "hello-world"

class Billing::Report               # compact style
  def rate = TAX_RATE               # NameError: no lexical Billing
end
```

CODE EXPLAINED

- → `Billing::Invoice` — nested class in a namespace
- → `TAX_RATE` — found lexically from inside `module Billing`
- → `module_function` — methods callable as `Formatter.money`
- → `extend self` — similar, keeps methods public on the module
- → compact `class Billing::Report` — no lexical access to `Billing` constants

THE RUBY WAY

- `class Billing::Report` → nested `module Billing; class Report` when you need its constants
- Global helper methods → a utility module with `module_function`
- Prefixing class names (`BillingInvoice`) → namespace them (`Billing::Invoice`)

IN RAILS

- **Rails** · Zeitwerk maps paths to constants: `app/models/admin/user.rb` → `Admin::User`
- **Rails** · file and constant names must match or autoloading raises `NameError`

CONSTANT LOOKUP

STEP	SEARCHES
1	lexical scopes (`Module.nesting`)
2	ancestors of the open class/module
3	`Object` (top level)
–	`::Name` jumps straight to top level

UTILITY MODULE STYLES

STYLE	NOTES
`module_function`	module + private instance copies
`extend self`	public module methods
`def self.m`	explicit, per method

⚠ COMMON MISTAKES

- Using compact `class A::B` and getting `NameError` for constants defined in `A`
- Naming a class like a stdlib one (`File` , `Logger`) at top level; namespace it instead
- Reopening a module by accident when two files use the same namespace and method names

✓ TIPS & BEST PRACTICES

- One top-level namespace per gem or app (`MyGem::`), matching the gem name
- `Module.nesting` inside a body shows the lexical scope Ruby searches
- Use `module_function` for stateless helpers that need no instance

QUICK SUMMARY

- `A::B` — namespaced constant
- nested `module … class` — lexical constant access
- `module_function` / `extend self` — utility modules
- `::Name` — top-level lookup

Polymorphism & Duck Typing

duck typing · respond_to? · overriding · NotImplementedError "abstract" methods · interfaces by convention

POLYMORPHISM AND DUCK TYPING

Ruby polymorphism is mostly **duck typing**: if an object responds to the method you call, it works, regardless of its class. Different classes implement the same method name (`to_s`, `each`, `call`), and callers just send the message. "Abstract" methods are a convention: define them to raise **`NotImplementedError`**.

NOTES

- No interfaces keyword; the contract is the method names
- `respond_to?` checks capability when you must branch
- Overriding in subclasses is classic polymorphism

DUCK TYPING IN PRACTICE

```ruby
class Pdf;  def render = "PDF bytes"; end
class Html; def render = "<html>";    end
[Pdf.new, Html.new].map(&:render)   # no shared base class

def save(io, data) = io.write(data) # File, StringIO, $stdout…
save($stdout, "hi\n")

class Shape
  def area = raise(NotImplementedError, "#{self.class}#area")
end
class Square < Shape
  def initialize(s) = @s = s
  def area = @s**2
end
Square.new(3).area            # 9
Shape.new.area                # NotImplementedError

def describe(x) = x.respond_to?(:each) ? "collection" : "single"
describe([1])                 # "collection"
describe(5)                   # "single"
```

CODE EXPLAINED

- → `map(&:render)` — no shared base class needed
- → `io.write` — any object with `write` is a valid `io`
- → `NotImplementedError` — marks an "abstract" method
- → `respond_to?(:each)` — branch on capability, not class
- → overriding `area` — subclass polymorphism

THE RUBY WAY

- `if obj.is_a?(File) … elsif obj.is_a?(StringIO)` → just call `obj.write`
- Abstract base classes with no shared code → document the duck type
- `case obj.class` dispatch → define the method on each class

IN RAILS

- **Rails** · any object with `to_param` can go in a URL helper
- **Rails** · `render @item` works for any object that responds to `to_partial_path`

COMMON DUCK TYPES

RESPONDS TO	ACTS LIKE
`each`	collection
`call`	callable / function
`to_str`	string
`to_proc`	block
`<=>`	comparable

CHECKING CAPABILITY

CHECK	PREFER WHEN
`just call it`	almost always
`respond_to?(:m)`	optional behavior
`is_a?(Cls)`	real type requirement

⚠ COMMON MISTAKES

- Type-checking everywhere with `is_a?`; it blocks valid duck-typed objects like `StringIO`
- Raising `NotImplementedError` from methods meant to be overridden but forgetting subclasses must define them; add a test per subclass
- Defining `respond_to?` without `respond_to_missing?` when using `method_missing` (sheet 110)

✓ TIPS & BEST PRACTICES

- Accept the narrowest interface: "anything with `each`" beats "an Array"
- `StringIO` is the go-to fake for any code that writes to files or streams
- Name the duck type in docs: `@param io [#write]`

QUICK SUMMARY

- duck typing — call the method, don't check the class
- `respond_to?` — check capability
- `NotImplementedError` — abstract by convention
- override — subclass polymorphism

Operator Overloading

OPERATOR OVERLOADING

Since operators are method calls, a class can define +, –, ==, <=>, [], []=, unary –@ and more. Define **<=>** and include `Comparable` to get all comparisons; define **`coerce`** so expressions like `2 * vector` work with your type on the right side.

RULES

- `!=` is derived from `==`; don't define it
- `+=` uses your `+` automatically
- Unary minus is `-@`, unary plus is `+@`
- Operators should return new objects, not mutate `self`

DEFINING OPERATORS

```ruby
class Vec
  attr_reader :x, :y
  def initialize(x, y) = (@x, @y = x, y)

  def +(o)  = Vec.new(x + o.x, y + o.y)
  def *(k)  = Vec.new(x * k, y * k)
  def -@    = Vec.new(-x, -y)
  def ==(o) = o.is_a?(Vec) && x == o.x && y == o.y
  def [](i) = [x, y].fetch(i)
  def to_s  = "(#{x}, #{y})"
  def coerce(n) = [self, n]      # lets 2 * v work
end

a = Vec.new(1, 2)
b = Vec.new(3, 4)
(a + b).to_s                     # "(4, 6)"
(-a).to_s                        # "(-1, -2)"
(a * 3).to_s                     # "(3, 6)"
(3 * a).to_s                     # "(3, 6)" via coerce
a == Vec.new(1, 2)               # true
a[1]                             # 2
a += b                           # uses + → a is (4, 6)
```

CODE EXPLAINED

- → `def +(o)` — `a + b` calls `a.+(b)`
- → `def -@` — unary minus: `-a`
- → `def ==(o)` — value equality (also gives `!=`)
- → `def [](i)` — index access
- → `coerce` — `3 * a` asks `a` to swap operands

THE RUBY WAY

- `add(other)` methods on value types → define `+`
- Defining `<`, `>`, `<=` by hand → `<=>` + `include Comparable`
- Operators that mutate `self` → return a new object

OVERLOADABLE

METHOD	CALLED BY
`+ - * / % **`	arithmetic
`== <=>`	equality / ordering
`[] []=`	indexing
`<<`	append
`-@ +@ ! ~`	unary
`=== =~`	case / match

NOT OVERLOADABLE

OPERATOR	WHY
`&& ||`	control flow
`=`	assignment
`!=`	derived from ==
`+=`	uses +

⚠ COMMON MISTAKES

- Mutating `self` in `+`; `a + b` changing `a` surprises everyone
- Defining `==` without `eql?` and `hash`; your objects then misbehave as hash keys (sheet 070)
- Forgetting `coerce`, so `2 * vec` raises `TypeError: Vec can't be coerced`

✓ TIPS & BEST PRACTICES

- Keep operator meanings familiar: `+` adds or combines, `<<` appends
- Return `NotImplemented`-style `TypeError` for unsupported operand types
- `Data.define` value objects already get value-based `==`

QUICK SUMMARY

- operators are methods — `def +(o)`
- `-@` / `+@` — unary operators
- `<=>` + `Comparable` — comparisons
- `coerce` — numbers on the left

Equality & Hash Keys

== · eql? · hash · equal? · objects as hash keys · uniq with custom objects

EQUALITY AND HASH KEYS

Ruby has three equality methods: **==** for value equality, **eql?** for stricter equality used by **Hash** and **Set**, and **equal?** for identity. To use your own objects as hash keys or in `uniq`, override **eql?** and **hash** together so equal objects produce equal hash codes.

RULES

- Objects that are `eql?` must return the same `hash`
- Default `==`, `eql?` and `equal?` all compare identity
- `1 == 1.0` but `!1.eql?(1.0)`, so they're different hash keys
- Never override `equal?`

VALUE EQUALITY FOR YOUR CLASSES

```ruby
class Point
  attr_reader :x, :y
  def initialize(x, y) = (@x, @y = x, y)

  def ==(other) = other.is_a?(Point) && [x, y] == [other.x, other.y]
  alias eql? ==
  def hash = [self.class, x, y].hash
end

a = Point.new(1, 2)
b = Point.new(1, 2)
a == b                      # true
a.equal?(b)                 # false (different objects)
{ a => "home" }[b]          # "home" (eql? + hash)
[a, b].uniq.size            # 1
Set[a, b].size              # 1

class Bare; end
Bare.new == Bare.new        # false (identity by default)
{ 1 => :int }[1.0]          # nil (1.eql?(1.0) is false)
```

CODE EXPLAINED

- → `==` — compares fields
- → `alias eql? ==` — reuse it for hash lookups
- → `hash` — combine the same fields into one code
- → `{ a => … }[b]` — works because `b.eql?(a)` and hashes match
- → default behavior — identity unless you override

THE RUBY WAY

- Overriding only `==` → also define `eql?` and `hash`
- Hand-written value classes → `Data.define` gives all three
- `hash` built with string concatenation → `[fields].hash`

IN RAILS

- **Rails** · ActiveRecord `==` compares class and `id`, so two loads of one row are equal
- **Rails** · new (unsaved) records are only equal to themselves

EQUALITY METHODS

METHOD	USED BY
`==`	comparisons, `include?`
`eql?`	Hash, Set, `uniq`
`equal?`	identity checks
`hash`	Hash, Set buckets
`===`	case / when

DEFAULTS

CLASS	== COMPARES
`Object`	identity
`String / Array`	contents
`Integer / Float`	numeric value
`Struct / Data`	fields
`ActiveRecord`	class + id

⚠ COMMON MISTAKES

- Defining `==` but not `hash`; objects compare equal but hash lookups and `uniq` miss them
- Including mutable fields in `hash` and then changing them while the object is a hash key
- Comparing with `equal?` when you mean value equality

✓ TIPS & BEST PRACTICES

- `Data.define` and `Struct` already implement value equality and hashing
- Include `self.class` in `hash` so different types with the same fields don't collide
- After mutating keys, call `hash.rehash` to rebuild the index

QUICK SUMMARY

- `==` — value equality
- `eql?` + `hash` — hash-key equality
- `equal?` — identity
- override all three together (except `equal?`)

Conversion Protocols

CONVERSION PROTOCOLS

Ruby distinguishes **explicit** conversions (`to_s`, `to_a`, `to_h`, `to_i`), which any object may offer, from **implicit** ones (`to_str`, `to_ary`, `to_hash`, `to_int`), which mean "I really am a string/array". Core methods call the implicit forms automatically, so defining them makes your object usable wherever the real type is expected.

PROTOCOL RULES

- Define `to_s` freely; define `to_str` only for string-like objects
- `to_ary` enables destructuring: `a, b = obj`
- `to_proc` enables `&obj` as a block
- `inspect` is for developers; `to_s` for users

EXPLICIT VS IMPLICIT

```ruby
class Path
  def initialize(p) = @p = p
  def to_s   = @p                 # explicit: interpolation, puts
  def to_str = @p                 # implicit: acts like a String
end
"dir/" + Path.new("a.rb")         # "dir/a.rb" (uses to_str)
File.basename(Path.new("x/y.rb")) # "y.rb"

class Pair
  def initialize(a, b) = (@a, @b = a, b)
  def to_ary = [@a, @b]           # implicit array
end
x, y = Pair.new(1, 2)             # x=1, y=2

class Doubler
  def to_proc = ->(n) { n * 2 }
end
[1, 2].map(&Doubler.new)          # [2, 4]

"n=" + 5                          # TypeError (Integer has no to_str)
"n=#{5}"                          # "n=5" (interpolation uses to_s)
```

CODE EXPLAINED

- → `to_s` — "here's a string version of me"
- → `to_str` — "treat me as a String"; `+` calls it
- → `to_ary` — multiple assignment and `flatten` call it
- → `to_proc` — `&obj` turns the object into a block
- → `"n=" + 5` — integers aren't string-like, so no implicit conversion

THE RUBY WAY

- Calling `.to_s` before every string `+` → interpolate
- Defining `to_str` just to print nicely → `to_s` (and `inspect`)
- Lambda wrappers for `&` → give the class a `to_proc`

IN RAILS

- **Rails** · `as_json` / `to_json` control how models serialize
- **Rails** · `to_param` (URL) and `to_partial_path` (`render`) are Rails protocols

EXPLICIT / IMPLICIT

EXPLICIT	IMPLICIT	CALLED BY
`to_s`	to_str	+, File methods
`to_a`	to_ary	`a, b = obj`, flatten
`to_h`	to_hash	`**obj`
`to_i`	to_int	Array#[], Integer()
—	to_proc	&obj

KERNEL CONVERTERS

CALL	USES
`String(x)`	to_str, then to_s
`Array(x)`	to_ary, then to_a
`Integer(x)`	to_int, then to_i
`Hash(x)`	to_hash, then to_h

⚠ COMMON MISTAKES

- Defining `to_str` on non-string objects; Ruby then silently treats them as strings in `+` and file APIs
- Returning a non-String from `to_s`; `puts` and interpolation then misbehave
- Overriding `to_s` but not `inspect`, so `p obj` still shows `#<Path:0x…>`

✓ TIPS & BEST PRACTICES

- Rule of thumb: implicit conversions only if your class is a drop-in replacement for the core type
- `Kernel#String()` / `Array()` are the safest way to normalize unknown input
- Implement `to_h` on value objects; it plugs into JSON and pattern matching neatly

QUICK SUMMARY

- `to_s` / `to_a` / `to_h` — explicit
- `to_str` / `to_ary` / `to_hash` — implicit
- `to_proc` — use as a block
- `String()` / `Array()` — normalize input

Copying & Freezing

COPYING AND FREEZING

Assignment copies **references**, not objects. **dup** and **clone** make **shallow** copies (nested objects are still shared); `clone` also keeps the frozen state and singleton methods. **freeze** makes an object immutable, which raises `FrozenError` on mutation, and `Ractor.make_shareable` deep-freezes a structure.

NOTES

- `freeze` is shallow; inner objects stay mutable
- `dup` returns an unfrozen copy; `clone` keeps frozen state
- `clone(freeze: false)` unfreezes the copy
- `initialize_copy` customizes what `dup` / `clone` copy

COPIES AND FREEZING

```ruby
a = [[1, 2], [3]]
b = a.dup
b << [4]                          # a unchanged
b[0] << 99                        # a[0] is [1, 2, 99] too (shared)
deep = Marshal.load(Marshal.dump(a))  # deep copy

s = "hi".freeze
s.frozen?                         # true
s << "!"                          # FrozenError
s.dup.frozen?                     # false
s.clone.frozen?                   # true
s.clone(freeze: false).frozen?    # false

CONFIG = { db: { host: "x" } }.freeze
CONFIG[:db][:host] = "y"          # allowed! inner hash not frozen
SAFE = Ractor.make_shareable({ db: { host: "x" } })
SAFE[:db].frozen?                 # true (deep)

class Order
  def initialize = @items = []
  def initialize_copy(src) = @items = @items.dup  # after copy
end
```

CODE EXPLAINED

→ `dup` — shallow copy, nested arrays shared

→ `Marshal` round-trip — quick deep copy for plain data

→ `clone` — keeps frozen state; `freeze: false` overrides

→ `.freeze` on a hash — only the outer hash is frozen

→ `initialize_copy` — deep-copy your own ivars on `dup`

THE RUBY WAY

- Defensive `dup` everywhere → freeze shared data instead
- Hand-freezing nested config → `Ractor.make_shareable`
- `Marshal` deep copy of objects with procs or IO → copy explicitly in `initialize_copy`

DUP VS CLONE

FEATURE	DUP	CLONE
`copies ivars`	yes	yes
`frozen state`	dropped	kept
`singleton methods`	dropped	kept
`depth`	shallow	shallow

FREEZING

CALL	EFFECT
`obj.freeze`	shallow immutable
`obj.frozen?`	check
`Ractor.make_shareable(obj)`	deep freeze
`# frozen_string_literal: true`	freeze literals
`+str / str.dup`	mutable copy

⚠ COMMON MISTAKES

- Expecting `dup` to copy nested arrays; `b[0] << x` changes `a[0]` too
- Freezing a constant hash and assuming its nested values are protected
- `Marshal` deep copies of objects holding procs, IO or singleton methods; they raise `TypeError`

✓ TIPS & BEST PRACTICES

- Freeze constants at definition: `LIMITS = [1, 5].freeze`
- Prefer immutable value objects (`Data`) over copying mutable ones
- `Object#then` / `tap` with `dup` keeps transformations free of side effects

QUICK SUMMARY

- `dup` / `clone` — shallow copies
- `freeze` — shallow immutability
- `make_shareable` — deep freeze
- `initialize_copy` — custom copying

Reading Files

READING FILES

For small files, **`File.read`** returns the whole contents and **`File.readlines`** returns an array of lines. For large files, **`File.foreach`** streams one line at a time with constant memory. The **block form of `File.open`** gives you the file object and closes it automatically, even when an exception is raised.

NOTES

- Lines keep their `\n` unless you pass `chomp: true`
- Paths are relative to the **current working directory**, not the script
- Missing file → `Errno::ENOENT`
- Default encoding is UTF-8; pass `encoding:` for others

READING

```ruby
text  = File.read("notes.txt")                  # whole file
lines = File.readlines("notes.txt", chomp: true)  # array, no \n

File.foreach("big.log") do |line|               # streams, low memory
  puts line if line.include?("ERROR")
end
File.foreach("big.log").count                   # line count

File.open("notes.txt") do |f|                   # auto-closed
  header = f.gets.chomp
  f.each_line.with_index(2) { |l, n| puts "#{n}: #{l}" }
end

File.read("data.bin", mode: "rb")               # binary
File.read("latin.txt", encoding: "ISO-8859-1:UTF-8")
here = File.join(__dir__, "notes.txt")          # relative to script
File.exist?(here) ? File.read(here) : ""
File.read("missing.txt")                        # Errno::ENOENT
```

CODE EXPLAINED

- → `File.read` — whole file as one string
- → `readlines(chomp: true)` — array of clean lines
- → `File.foreach` — iterate without loading everything
- → `File.open { |f| }` — closes on exit, success or error
- → `__dir__` — directory of the current source file

THE RUBY WAY

- `f = File.open(p); f.read; f.close` → `File.read(p)`
- `File.readlines(p).each` on huge files → `File.foreach(p)`
- `line.chomp` on every line → `chomp: true`

READ METHODS

METHOD	RETURNS
`File.read(p)`	String
`File.readlines(p)`	Array of lines
`File.foreach(p) { }`	yields each line
`File.open(p) { |f| }`	block value
`IO.readlines(p)`	same as File

FILE OBJECT READS

CALL	READS
`f.read`	the rest
`f.gets`	next line / nil
`f.each_line`	line by line
`f.read(1024)`	up to 1024 bytes
`f.eof?`	at end?

⚠ COMMON MISTAKES

- Using paths relative to the script while running from another directory; build them with `__dir__`
- `File.readlines` on multi-GB logs; it loads everything into memory
- Opening files without the block form and leaking file handles on exceptions

✓ TIPS & BEST PRACTICES

- `File.foreach(p).lazy.select { … }.first(10)` stops reading early
- Use `mode: "rb"` for images and archives to avoid encoding conversion
- `Pathname#read` / `#each_line` give the same methods on path objects (sheet 075)

QUICK SUMMARY

- `File.read` — whole file
- `readlines(chomp: true)` — line array
- `File.foreach` — stream lines
- `File.open { }` — auto-close

Writing & Appending

WRITING AND APPENDING

`File.write` replaces a file's contents in one call, and `mode: "a"` appends. For many writes, open the file with a **mode string** and a block: `"w"` truncates, `"a"` appends, `"r+"` reads and writes. `puts`, `print`, `write` and `<<` all work on file objects just like on `$stdout`.

NOTES

- `"w"` creates or **truncates** the file
- `"a"` creates or appends; never truncates
- Add `"b"` for binary (`"wb"`), `"x"` to fail if the file exists
- Writes are buffered; `flush` or closing writes them out

WRITING

```ruby
File.write("out.txt", "Hello\n")              # create / overwrite
File.write("out.txt", "more\n", mode: "a")    # append

File.open("report.txt", "w") do |f|           # auto-closed
  f.puts "Report"                             # with newline
  f.print "a", "b"                            # no newline
  f << "x" << "y\n"                          # chainable
  f.printf("%-5s %3d\n", "total", 42)
end

File.open("app.log", "a") { |f| f.puts "#{Time.now} started" }
File.write("img.png", bytes, mode: "wb")      # binary

File.open("new.txt", "wx") { |f| f.puts "x" } # Errno::EEXIST if present
File.open("data.txt", "w") do |f|
  f.sync = true                               # unbuffered
  f.write("line\n")
end
File.write("tmp.txt", "") && File.size("tmp.txt")   # 0
```

CODE EXPLAINED

- → `File.write(p, s)` — one-shot write, returns bytes written
- → `mode: "a"` — append instead of replacing
- → `f.puts` / `f.print` / `f <<` — same API as `$stdout`
- → `"wx"` — exclusive create, fails if the file exists
- → `f.sync = true` — write through immediately

THE RUBY WAY

- `File.open(p, "w") { |f| f.write(s) }` → `File.write(p, s)`
- Reopening a file for each line → one `File.open` block
- Manual `f.close` → block form

MODES

MODE	BEHAVIOR
`"r"`	read (default)
`"w"`	write, truncate / create
`"a"`	append / create
`"r+"`	read + write, no truncate
`"w+"`	read + write, truncate
`"wx"`	create only, error if exists

WRITE METHODS

METHOD	ADDS NEWLINE?
`f.puts(x)`	yes
`f.print(x)`	no
`f.write(x)`	no, returns bytes
`f << x`	no, chainable
`f.printf(fmt, …)`	only if in fmt

⚠ COMMON MISTAKES

- Opening with `"w"` when you meant `"a"` ; the old contents are gone
- Writing in a loop with `File.write(p, line, mode: "a")` ; it opens and closes the file each time
- Forgetting the file is buffered and reading it from another process before it's flushed

✓ TIPS & BEST PRACTICES

- For crash-safe writes, write to `file.tmp` then `File.rename` over the original
- `FileUtils.mkdir_p(File.dirname(p))` before writing into new directories
- `Tempfile.create { |f| … }` gives a self-deleting temporary file

QUICK SUMMARY

- `File.write` — one-shot write
- `"w"` / `"a"` — truncate / append
- `f.puts` / `f.write` — write on a file object
- block form — auto-close

Paths & Directories

PATHS AND DIRECTORIES

`File` class methods build and inspect paths (`join`, `basename`, `extname`, `exist?`), `Dir` lists and globs directories, and **`FileUtils`** copies, moves and deletes. **`Pathname`** wraps all of these in an object-oriented API where `/` joins path segments.

NOTES

- `File.join` uses `/`, which works on Windows too
- `Dir.glob("**/*.rb")` searches recursively
- `File.expand_path("~")` resolves the home directory
- `require "fileutils"` / `"pathname"` before using them

PATHS, DIR, FILEUTILS, PATHNAME

```
path = File.join("app", "models", "user.rb")  # "app/models/user.rb"
File.basename(path)             # "user.rb"
File.basename(path, ".rb")      # "user"
File.extname(path)              # ".rb"
File.dirname(path)              # "app/models"
File.exist?(path)               # true / false
File.expand_path("~/notes")     # "/Users/ada/notes"

Dir.glob("**/*.rb")             # all .rb files, recursive
Dir.children("app")             # names, no "." / ".."
Dir.mkdir("tmp") unless Dir.exist?("tmp")

require "fileutils"
FileUtils.mkdir_p("out/logs")  # like mkdir -p
FileUtils.cp("a.txt", "out/")
FileUtils.rm_rf("tmp")          # careful: no confirmation

require "pathname"
dir = Pathname("app") / "models"
dir.join("user.rb").read
dir.children.select(&:file?)
```

CODE EXPLAINED

- → `File.join` — portable path building
- → `basename(path, ".rb")` — strip the extension
- → `Dir.glob("**/*.rb")` — recursive pattern match
- → `FileUtils.mkdir_p` / `cp` / `rm_rf` — shell-like operations
- → `Pathname("app") / "models"` — `/` joins segments

THE RUBY WAY

- `"#{dir}/#{file}"` → `File.join(dir, file)` or `Pathname(dir) / file`
- `system("mkdir -p out")` → `FileUtils.mkdir_p("out")`
- `Dir.entries(d) - %w[. ..]` → `Dir.children(d)`

IN RAILS

- **Rails** · `Rails.root.join("config", "app.yml")` returns a `Pathname`
- **Rails** · `Rails.public_path` and `Rails.root / "tmp"` for app paths

FILE QUERIES

METHOD	RETURNS
`exist?(p)`	any file or dir
`file?(p)` / `directory?(p)`	type
`size(p)`	bytes
`mtime(p)`	Time modified

FILEUTILS

METHOD	LIKE
`mkdir_p`	mkdir -p
`cp` / `cp_r`	cp / cp -r
`mv`	mv
`rm` / `rm_rf`	rm / rm -rf

⚠ COMMON MISTAKES

- Building paths with string concatenation and doubling or missing slashes
- Calling `FileUtils.rm_rf` with a variable that can be empty or `"/"`; validate first
- Forgetting `Dir.glob` results are relative to the current directory; use `base:` or absolute patterns

✓ TIPS & BEST PRACTICES

- `Dir.glob("*.{jpg,png}")` matches several extensions
- Pass `noop: true, verbose: true` to `FileUtils` calls to dry-run them
- Ruby 4.0 moves `Pathname` toward core; check whether `require "pathname"` is still needed on your version

QUICK SUMMARY

- `File.join` / `basename` / `extname` — paths
- `Dir.glob` — find files
- `FileUtils` — copy, move, delete
- `Pathname` — OO paths with `/`

JSON

JSON

The `json` library converts between JSON text and Ruby **hashes, arrays, strings, numbers, `true` / `false` and `nil`**. `JSON.parse` returns **string keys** by default; pass `symbolize_names: true` for symbols. `to_json` / `JSON.generate` produce compact output and `JSON.pretty_generate` produces indented output.

JSON NOTES

- `require "json"` first (it's a default gem)
- JSON `null` ↔ `nil` ; symbols become strings
- Custom objects need `to_h` or a `to_json` method
- Invalid JSON raises `JSON::ParserError`

PARSE AND GENERATE

```
require "json"
text = '{"name":"Ada","langs":["ruby","c"],"admin":false}'
data = JSON.parse(text)
data["name"]                      # "Ada"
sym  = JSON.parse(text, symbolize_names: true)
sym[:langs]                       # ["ruby", "c"]

{ id: 1, tags: [:a] }.to_json     # '{"id":1,"tags":["a"]}'
JSON.generate([1, nil, true])     # '[1,null,true]'
puts JSON.pretty_generate(sym)    # indented, 2 spaces

config = JSON.load_file("config.json", symbolize_names: true)

User = Data.define(:id, :name) do
  def to_json(*args) = to_h.to_json(*args)
end
[User.new(1, "Ada")].to_json      # '[{"id":1,"name":"Ada"}]'
JSON.parse("{bad}")               # JSON::ParserError
```

CODE EXPLAINED

- → `JSON.parse` — JSON text → Hash / Array
- → `symbolize_names: true` — symbol keys
- → `to_json` / `JSON.generate` — Ruby → compact JSON
- → `JSON.load_file` — read and parse in one step
- → `def to_json(*args)` — custom serialization

THE RUBY WAY

- `JSON.parse(s)["a"]["b"]` → `JSON.parse(s, symbolize_names: true).dig(:a, :b)`
- `JSON.parse(File.read(p))` → `JSON.load_file(p)`
- Hand-building JSON strings → build a hash and call `to_json`

IN RAILS

- **Rails** · `render json: @user` calls `as_json` then encodes
- **Rails** · `@user.as_json(only: [:id, :name])` picks fields; `to_json` accepts the same options

TYPE MAPPING

JSON	RUBY
`object`	Hash
`array`	Array
`string`	String
`number`	Integer / Float
`true / false`	true / false
`null`	nil

USEFUL OPTIONS

OPTION	EFFECT
`symbolize_names: true`	symbol keys
`object_class: OpenStruct`	custom object class
`max_nesting: 10`	depth limit
`allow_nan: true`	NaN / Infinity

⚠ COMMON MISTAKES

- Accessing parsed data with symbol keys (`data[:name]`) when keys are strings; you get `nil`
- Using `JSON.load` on untrusted input; prefer `JSON.parse` , which never creates arbitrary objects
- Expecting symbols to round-trip; they come back as strings

✓ TIPS & BEST PRACTICES

- Rescue `JSON::ParserError` around user-supplied JSON and return a clear error
- `JSON.pretty_generate` for config and fixtures; compact `to_json` for APIs
- Pattern match parsed data: `case JSON.parse(s, symbolize_names: true) in { user: { id: } }`

QUICK SUMMARY

- `JSON.parse` — text → Ruby
- `symbolize_names: true` — symbol keys
- `to_json` / `pretty_generate` — Ruby → text
- `JSON.load_file` — read a file

CSV

CSV

The `csv` library reads and writes comma-separated data, handling quoting, embedded commas and newlines correctly. Since Ruby 3.4 `csv` is a **bundled gem**, so add it to your Gemfile when using Bundler. With `headers: true`, each row becomes a **`CSV::Row`** you can index by column name.

CSV NOTES

- `require "csv"` first
- `headers: true` treats the first line as column names
- `converters: :numeric` turns `"42"` into `42`
- `CSV.foreach` streams large files row by row

READING AND WRITING CSV

```ruby
require "csv"
rows = CSV.read("people.csv")           # [["name", "age"], ["Ada", "36"]]

CSV.foreach("people.csv", headers: true, converters: :numeric) do |row|
  puts "#{row["name"]} is #{row["age"] + 1}"   # Ada is 37
end

table = CSV.read("people.csv", headers: true)
table["name"]                           # ["Ada", "Bob"] (column)
table.first.to_h                        # {"name" => "Ada", "age" => "36"}

CSV.open("out.csv", "w") do |csv|
  csv << %w[name age]                   # header row
  csv << ["Lin, Jr.", 41]               # comma gets quoted
end

line = CSV.generate_line(["a", "b,c"])  # "a,\"b,c\"\n"
CSV.parse("x;y\n1;2", col_sep: ";", headers: true).map(&:to_h)
# [{"x" => "1", "y" => "2"}]
```

CODE EXPLAINED

- → `CSV.read` — all rows as arrays of strings
- → `headers: true` — rows act like hashes keyed by column
- → `converters: :numeric` — numbers instead of strings
- → `CSV.open(…, "w") + <<` — write rows, quoting handled
- → `col_sep: ";"` — other delimiters

THE RUBY WAY

- `line.split(",")` → `CSV.parse_line(line)` (handles quotes)
- `row[1]` positional access → `headers: true` and `row["age"]`
- Reading whole files with `CSV.read` for huge inputs → `CSV.foreach`

IN RAILS

- **Rails** · `send_data CSV.generate { |csv| … }, filename: "users.csv"` streams a download
- **Rails** · add `gem "csv"` to the Gemfile on Ruby 3.4+

READ OPTIONS

OPTION	EFFECT
`headers: true`	use first row as names
`header_converters: :symbol`	`:name` keys
`converters: :numeric`	parse numbers
`col_sep: "\t"`	TSV
`skip_blanks: true`	ignore empty lines

API

METHOD	USE
`CSV.read`	all rows
`CSV.foreach`	stream rows
`CSV.parse(str)`	from a string
`CSV.open(p, "w")`	write a file
`CSV.generate`	build a string

⚠ COMMON MISTAKES

- Splitting lines on commas and breaking on quoted fields like `"Lin, Jr."`
- Forgetting values are strings; `"36" + 1` raises. Use `converters: :numeric`
- On Ruby 3.4+, `require "csv"` failing under Bundler because the gem isn't in the Gemfile

✓ TIPS & BEST PRACTICES

- `header_converters: :symbol` gives `row[:age]` access and works with pattern matching
- Add `encoding: "bom\|utf-8"` for CSVs exported from Excel
- `CSV::Table#by_col` and `#each` let you process columns or rows

QUICK SUMMARY

- `CSV.read` / `foreach` — read
- `headers: true` — named columns
- `converters: :numeric` — numbers
- `CSV.open … <<` — write

YAML

YAML

YAML is the common format for Ruby config files. `require "yaml"` loads **Psych**, which maps YAML to hashes, arrays, strings, numbers, booleans and `nil`. **`YAML.load_file` is safe by default** since Psych 4 (Ruby 3.1): it refuses arbitrary Ruby objects unless you list them in `permitted_classes`.

YAML NOTES

- Keys load as strings unless you pass `symbolize_names: true`
- `yes` / `no` / `on` stay strings in YAML 1.2 style but booleans in 1.1 tools
- Dates need `permitted_classes: [Date]` or `YAML.unsafe_load`
- Aliases (`&x` / `*x`) need `aliases: true`

LOAD AND DUMP YAML

```ruby
require "yaml"
yaml = <<~YML
  app: shop
  port: 8080
  debug: true
  hosts:
    - a.example.com
    - b.example.com
YML
cfg = YAML.safe_load(yaml, symbolize_names: true)
cfg[:port]                      # 8080
cfg[:hosts].size                # 2

YAML.load_file("config.yml")    # safe by default (Psych 4)
YAML.load_file("dates.yml", permitted_classes: [Date])

{ "name" => "Ada", "langs" => %w[ruby c] }.to_yaml
# "---\nname: Ada\nlangs:\n- ruby\n- c\n"
YAML.dump([1, nil])             # "---\n- 1\n-\n"
```

CODE EXPLAINED

- → `YAML.safe_load` — plain data only, no Ruby objects
- → `symbolize_names: true` — symbol keys
- → `permitted_classes: [Date]` — allow specific types
- → `to_yaml` / `YAML.dump` — Ruby → YAML text

THE RUBY WAY

- `YAML.load(untrusted)` on old Ruby → `YAML.safe_load`
- Ruby code in config files → plain YAML (or `ERB` + YAML when values need it)
- `cfg["db"]["host"]` → `symbolize_names: true` + `cfg.dig(:db, :host)`

IN RAILS

- **Rails** · `config_for(:payment)` loads `config/payment.yml` per environment
- **Rails** · secrets live in encrypted `credentials.yml.enc`

YAML → RUBY

YAML	RUBY
`key: value`	Hash
`- item`	Array
`42 / 3.5`	Integer / Float
`true / false`	true / false
`~ / null`	nil

LOADERS

METHOD	ALLOWS
`YAML.safe_load`	basic types
`YAML.load (Psych 4)`	same as safe_load
`YAML.load_file`	safe, from file
`YAML.unsafe_load`	any object (trusted only)

⚠ COMMON MISTAKES

- Upgrading Ruby and getting `Psych::DisallowedClass` for dates or symbols; add `permitted_classes`
- Indentation mistakes; YAML is whitespace-sensitive and tabs aren't allowed
- Unquoted values like `NO` (a country code) parsing as `false` in YAML 1.1 tools; quote ambiguous strings

✓ TIPS & BEST PRACTICES

- Quote version numbers (`"3.10"`); unquoted they become floats like `3.1`
- `YAML.load_file(p, symbolize_names: true, freeze: true)` gives frozen config
- Use anchors and aliases to share defaults between environments

QUICK SUMMARY

- `YAML.safe_load` / `load_file` — safe parsing
- `symbolize_names` — symbol keys
- `permitted_classes` — allow types
- `to_yaml` — dump

require & load

require · require_relative · load · $LOAD_PATH · autoload · Zeitwerk naming conventions

REQUIRE AND LOAD

require loads a library or file **once** by searching $LOAD_PATH (and installed gems). **require_relative** loads a file relative to the current file, which is what you want inside your own project. **load** re-runs a file every time it's called. autoload defers loading a constant until first use.

NOTES

- `require` returns `true` the first time, `false` after
- The `.rb` extension is optional
- `$LOADED_FEATURES` lists everything already required
- Gems are added to the load path by RubyGems / Bundler

LOADING CODE

```ruby
require "json"                    # stdlib / gem
require "json"                    # => false (already loaded)
require_relative "lib/parser"     # ./lib/parser.rb from this file
require_relative "../config"      # relative paths work

load "settings.rb"                # runs the file again every time
load "./tasks.rb", true           # wrap in an anonymous module

$LOAD_PATH.unshift File.expand_path("lib", __dir__)
require "my_app"                  # now finds lib/my_app.rb

module MyApp
  autoload :Report, "my_app/report"   # loaded on first MyApp::Report
end

$LOADED_FEATURES.grep(/json/).first   # path of loaded json.rb
defined?(JSON)                    # "constant"
```

CODE EXPLAINED

- → `require` — search load path, load once
- → `require_relative` — path relative to this file
- → `load` — execute again each call (scripts, reloading)
- → `$LOAD_PATH.unshift` — add your `lib` directory
- → `autoload` — load a file when the constant is first used

THE RUBY WAY

- `require "./lib/parser"` → `require_relative "lib/parser"`
- `require File.expand_path("../x", __FILE__)` → `require_relative "x"`
- Requiring every file by hand in a big app → Zeitwerk autoloading

IN RAILS

- **Rails** · Zeitwerk autoloads `app/` constants; don't `require` your own app files
- **Rails** · code in `lib/` needs `config.autoload_lib` or explicit requires

LOADERS

METHOD	LOADS
`require "x"`	once, via load path
`require_relative "x"`	once, from this file's dir
`load "x.rb"`	every call
`autoload :C, "x"`	on first use of C

LOAD-PATH TOOLS

NAME	HOLDS
`$LOAD_PATH / $:`	search directories
`$LOADED_FEATURES / $"`	required files
`ruby -I lib`	add dir from CLI
`Gem.loaded_specs`	active gems

⚠ COMMON MISTAKES

- Using `require "lib/x"` and depending on the current directory; it breaks when run from elsewhere
- Using `load` in libraries; constants get redefined and warnings appear
- Circular requires (`a` requires `b` requires `a`); one file sees half-defined constants

✓ TIPS & BEST PRACTICES

- Inside a gem, `require` your own files with the gem's namespace: `require "my_gem/version"`
- Zeitwerk works outside Rails too (`gem "zeitwerk"`) for any app with conventional file names
- `ruby -rjson -e '…'` requires a library from the command line

QUICK SUMMARY

- `require` — load once via load path
- `require_relative` — load from this file's directory
- `load` — run again
- `autoload` / Zeitwerk — lazy loading

RubyGems

RUBYGEMS

RubyGems is Ruby's package system: a **gem** is a packaged library or tool, and the `gem` command installs, lists and removes them from **rubygems.org**. Ruby ships **default gems** (always installed, like `json`) and **bundled gems** (installed with Ruby but must be listed in a Gemfile under Bundler, like `csv` since 3.4).

NOTES

- Gems install into your Ruby's gem directory (no `sudo` with a version manager)
- Executables from gems go on your `PATH` (`rails` , `rubocop`)
- Versions follow semver: `MAJOR.MINOR.PATCH`
- For projects, manage gems with Bundler (sheet 081)

THE GEM COMMAND

```
gem install rubocop                # latest version
gem install rails -v 8.1.0         # specific version
gem install nokogiri --no-document  # skip docs (faster)
gem list                           # installed gems
gem list rails                     # filter by name
gem info rubocop                   # details + install path
gem outdated                       # newer versions available
gem update rubocop
gem uninstall rubocop -v 1.60.0
gem search ^rack                   # search rubygems.org
gem env                            # paths, version, sources
gem which json                     # file that `require "json"` loads
gem contents rake | head -5        # files in a gem
gem cleanup                        # remove old versions
```

CODE EXPLAINED

→ `gem install x -v` — pin a version

→ `--no-document` — skip RDoc/ri generation

→ `gem outdated` / `update` — keep tools current

→ `gem env` — where gems and executables live

→ `gem which json` — find a library's file

THE RUBY WAY

- `gem install` for every project dependency → a Gemfile + `bundle install`
- `sudo gem install` → install Ruby with a version manager
- Re-typing `--no-document` → put `gem: --no-document` in `~/.gemrc`

GEM KINDS

KIND	EXAMPLE	IN GEMFILE?
`default`	json, fileutils	optional
`bundled`	csv, minitest, rake	required (Bundler)
`third-party`	rails, rspec	required

VERSION SPECS

SPEC	MEANS
`"1.2.3"`	exactly
`">= 1.2"`	at least
`"~> 1.2"`	≥ 1.2, < 2.0
`"~> 1.2.3"`	≥ 1.2.3, < 1.3

⚠ COMMON MISTAKES

- Installing gems globally for a project and getting version conflicts between projects
- Forgetting `rbenv rehash` (older rbenv) after installing a gem with an executable
- Assuming a bundled gem like `csv` is always available; under Bundler it must be in the Gemfile

✓ TIPS & BEST PRACTICES

- `gem install --user-install` installs to your home directory when you can't use a version manager
- `gem info` shows the homepage and license before you add a dependency
- Check `rubygems.org` download counts and release dates when choosing between gems

QUICK SUMMARY

- `gem install` / `list` / `uninstall` — manage gems
- default vs bundled — always there vs Gemfile needed
- `~>` — pessimistic version constraint
- Bundler — per-project gems

Bundler

BUNDLER

Bundler manages a project's gems: you list dependencies in a **`Gemfile`**, `bundle install` resolves compatible versions and records them in **`Gemfile.lock`**, and `bundle exec` runs commands with exactly those versions. Commit both files so every machine and CI run uses the same gems.

BUNDLER NOTES

- Bundler ships with Ruby (`bundle -v`)
- `Gemfile.lock` pins exact versions; commit it for apps
- Groups (`:development` , `:test`) keep tools out of production
- `bundle add` edits the Gemfile and installs in one step

GEMFILE

```
source "https://rubygems.org"
ruby "~> 4.0"                    # required Ruby version

gem "sinatra", "~> 4.1"          # >= 4.1, < 5.0
gem "pg", ">= 1.5"
gem "csv"                        # bundled gem: list it explicitly
gem "my_lib", path: "../my_lib"  # local path
gem "toolkit", github: "acme/toolkit", branch: "main"

group :development, :test do
  gem "rspec", "~> 3.13"
  gem "debug"

end
```

CODE EXPLAINED

- → `~> 4.1` — pessimistic constraint: allow 4.x updates
- → `path:` / `github:` — gems from local dirs or git
- → `group :test` — only installed / loaded where needed
- → `bundle exec` — guarantees lock-file versions

THE RUBY WAY

- `gem install` per project → Gemfile + `bundle install`
- `bundle update` (everything) → `bundle update gem_name`
- Typing `bundle exec` constantly → binstubs (`bin/rspec`)

COMMANDS

```
bundle install           # resolve + write Gemfile.lock
bundle add faraday       # add to Gemfile + install
bundle exec rspec        # run with locked versions
bundle update sinatra    # upgrade one gem
```

IN RAILS

- **Rails** · `bin/rails` and other binstubs already run under Bundler
- **Rails** · `Bundler.require(*Rails.groups)` auto-requires gems for the current environment

GEMFILE SYNTAX

LINE	MEANING
`gem "x"`	any version
`gem "x", "~> 2.3"`	≥ 2.3, < 3.0
`gem "x", "1.2.0"`	exact
`gem "x", require: "y"`	require a different name

COMMANDS

COMMAND	DOES
`bundle install`	install from lock / resolve
`bundle exec cmd`	run with bundle
`bundle update x`	upgrade x
`bundle info x`	version + path

⚠ COMMON MISTAKES

- Running `rspec` instead of `bundle exec rspec` and getting a different gem version
- Not committing `Gemfile.lock` for an application; teammates resolve different versions
- Running `bundle update` with no arguments and upgrading every dependency at once

✓ TIPS & BEST PRACTICES

- `bundle config set --local path vendor/bundle` keeps gems inside the project
- `bundle binstubs rspec-core` creates `bin/rspec` so you can skip `bundle exec`
- Gems (libraries) usually don't commit `Gemfile.lock` ; apps always do

QUICK SUMMARY

- `Gemfile` — declare dependencies
- `Gemfile.lock` — exact versions (commit it)
- `bundle exec` — run with locked gems
- `~>` — pessimistic constraint

Creating a Gem

CREATING A GEM

`bundle gem name` scaffolds a gem with the conventional layout: code in **`lib/`**, a **`.gemspec`** describing metadata and dependencies, a **`version.rb`** constant, and test setup. `gem build` packages it into a `.gem` file, and `gem push` publishes it to rubygems.org.

CONVENTIONS

- Gem name `my_gem` → `lib/my_gem.rb` → `module MyGem`
- Dashes mean namespaces: `my-gem` → `lib/my/gem.rb`
- Put runtime deps in the gemspec, dev tools in the Gemfile
- Executables go in `exe/` and are listed in `spec.executables`

SCAFFOLD, BUILD, PUBLISH

```
bundle gem slugify --test=rspec --mit
# slugify/
#   lib/slugify.rb          main file
#   lib/slugify/version.rb  VERSION constant
#   slugify.gemspec         metadata
#   spec/                   tests
cd slugify
bundle exec rake spec       # run tests
gem build slugify.gemspec   # → slugify-0.1.0.gem
gem push slugify-0.1.0.gem  # publish (needs an account)
bundle exec rake release    # tag + build + push in one step
```

CODE EXPLAINED

- → `bundle gem` — generates the standard layout
- → `version.rb` — single source for the version number
- → `gem build` — packages into a `.gem` file
- → `rake release` — tags git, builds and pushes
- → `add_dependency` — runtime dependencies users get

SLUGIFY.GEMSPEC (EXCERPT)

```
Gem::Specification.new do |spec|
  spec.name    = "slugify"
  spec.version = Slugify::VERSION
  spec.required_ruby_version = ">= 3.2"
  spec.files   = Dir["lib/**/*.rb", "README.md"]
  spec.add_dependency "unicode_normalize", "~> 1.0"
end
```

THE RUBY WAY

- Hard-coding the version in several files → `MyGem::VERSION`
- Dev tools in `add_dependency` → put them in the Gemfile
- `Dir["**/*"]` in `spec.files` → list only shipped files

LAYOUT

PATH	HOLDS
`lib/name.rb`	entry point, requires the rest
`lib/name/*.rb`	namespaced code
`exe/`	command-line executables
`spec/ or test/`	tests

GEMSPEC FIELDS

FIELD	PURPOSE
`name / version`	identity
`summary / description`	listing text
`required_ruby_version`	Ruby constraint
`add_dependency`	runtime deps

⚠ COMMON MISTAKES

- Forgetting to bump `VERSION`; rubygems.org rejects re-pushing an existing version
- Leaving `spec.files` using `git ls-files` with untracked files missing from the package
- Defining top-level constants or monkey patches in a gem; everything should live under its module

✓ TIPS & BEST PRACTICES

- Enable MFA on your rubygems.org account before pushing
- Use trusted publishing from GitHub Actions instead of storing API keys
- Run `gem build` then `gem install ./x.gem` locally to test the packaged version

QUICK SUMMARY

- `bundle gem` — scaffold
- `.gemspec` — metadata + deps
- `gem build` / `gem push` — package / publish
- `lib/name.rb` + `module Name` — layout

Standard Library — Math & Random

STANDARD LIBRARY: MATH AND RANDOM

The **Math** module provides floating-point functions and constants (`sqrt`, `log`, `sin`, `PI`). **rand** and the **Random** class generate pseudo-random numbers, with a seed for repeatable sequences. For tokens, passwords and anything security-related, use **SecureRandom**.

NOTES

- `Math` functions return Floats: `Math.sqrt(16)` is `4.0`
- `rand` with no args gives a Float in `[0, 1)`
- `Random.new(seed)` repeats the same sequence
- `require "securerandom"` for `SecureRandom`

MATH, RAND, SECURERANDOM

```
Math.sqrt(16)                 # 4.0
Integer.sqrt(17)              # 4 (exact integer)
Math.cbrt(27)                 # 3.0
Math.hypot(3, 4)              # 5.0
Math.log(Math::E)             # 1.0
Math.log10(1000)              # 3.0
Math.log2(8)                  # 3.0
Math.sin(Math::PI / 2)        # 1.0
[12.gcd(18), 12.lcm(18)]      # [6, 36]

rand                          # 0.734... (Float)
rand(10)                      # 0..9
rand(1..6)                    # 1..6 (dice)
[1, 2, 3].sample              # random element
rng = Random.new(42)
rng.rand(100)                 # 51 (same every run with seed 42)

require "securerandom"
SecureRandom.hex(8)           # "3f9a1c2b7d4e6f80"
SecureRandom.uuid             # "b7f1…-…" (v4)
SecureRandom.alphanumeric(12) # "aZ3kP9qL2mXb"
```

CODE EXPLAINED

- → `Math.sqrt` — Float result; `Integer.sqrt` for exact ints
- → `Math.hypot(3, 4)` — distance without overflow issues
- → `rand(1..6)` — inclusive integer range
- → `Random.new(42)` — seeded, repeatable generator
- → `SecureRandom` — cryptographically secure values

THE RUBY WAY

- `rand(6) + 1` → `rand(1..6)`
- `arr[rand(arr.size)]` → `arr.sample`
- `rand.to_s` for tokens → `SecureRandom.hex` / `urlsafe_base64`

MATH FUNCTIONS

CALL	RETURNS
`sqrt(x) / cbrt(x)`	roots
`log(x) / log(x, b)`	natural / base b
`exp(x)`	e^x
`sin / cos / tan`	radians
`PI / E`	constants

RANDOM SOURCES

CALL	USE
`rand / Random.rand`	quick, non-secure
`Random.new(seed)`	repeatable tests
`arr.sample / shuffle`	pick / reorder
`SecureRandom.hex(n)`	tokens
`SecureRandom.uuid`	unique ids

⚠ COMMON MISTAKES

- Using `rand` for passwords or session tokens; it's predictable. Use `SecureRandom`
- Passing degrees to `Math.sin`; it expects radians (`deg * Math::PI / 180`)
- `Math.sqrt(-1)` raises `Math::DomainError`; use `Complex` if you need imaginary results

✓ TIPS & BEST PRACTICES

- Use `Random.new(seed)` and pass it into code under test (`shuffle(random: rng)`)
- `SecureRandom.uuid_v7` gives time-ordered UUIDs (Ruby 3.3+)
- `Float#round(n)` before display; `Math` results carry float noise

QUICK SUMMARY

- `Math.sqrt` / `log` / `sin` — float math
- `rand(range)` — random numbers
- `Random.new(seed)` — repeatable
- `SecureRandom` — security-grade randomness

Standard Library — Dates & Times

STANDARD LIBRARY: DATES AND TIMES

Time (core) represents a moment with date, time and UTC offset. **Date** (`require "date"`) represents calendar days without a time. Arithmetic on `Time` uses **seconds**, on `Date` uses **days**; `strftime` formats both, and `Time.parse` / `Date.parse` / `iso8601` read strings.

NOTES

- `Time.now` uses the system time zone; `Time.now.utc` for UTC
- Subtracting two Times gives seconds as a Float
- `Date.today + 1` is tomorrow
- `require "time"` adds `Time.parse` and `Time#iso8601`

TIME AND DATE

```ruby
now = Time.now                      # 2026-10-17 14:05:09 -0500
now.year / now.month / now.day      # 2026 / 10 / 17
now + 3600                          # one hour later
(Time.now - now).round(2)           # elapsed seconds
now.strftime("%Y-%m-%d %H:%M")      # "2026-10-17 14:05"
now.strftime("%b %-d, %Y")          # "Oct 17, 2026"
Time.at(0).utc                      # 1970-01-01 00:00:00 UTC
now.to_i                            # Unix timestamp

require "time"
Time.parse("2026-10-17 09:30")      # local time
Time.iso8601("2026-10-17T09:30:00Z")
now.utc.iso8601                     # "2026-10-17T19:05:09Z"

require "date"
d = Date.new(2026, 12, 25)
(d - Date.today).to_i               # days until Christmas
d.saturday?                         # false (it's a Friday)
d >> 1                              # 2027-01-25 (add a month)
Date.parse("Oct 17 2026").yday      # 290
```

CODE EXPLAINED

- → `now + 3600` — Time math is in seconds
- → `strftime` — format with `%Y`, `%m`, `%d`, `%H`, `%M`
- → `Time.at(0)` — from a Unix timestamp
- → `iso8601` — standard machine-readable format
- → `Date - Date` — a `Rational` number of days

THE RUBY WAY

- `Time.now.to_s` for logs or APIs → `Time.now.utc.iso8601`
- `t + 86400 * 30` for "a month" → `Date#>>` (calendar aware)
- Parsing with `Date.parse` for user input → `Date.strptime(s, "%m/%d/%Y")`

IN RAILS

- **AS** · `Time.zone.now` / `Time.current` respect the app's configured zone
- **AS** · `2.days.ago`, `1.week.from_now`, `beginning_of_day`, `end_of_month`
- **AS** · `Date.current` instead of `Date.today` in Rails apps

STRFTIME

CODE	EXAMPLE
`%Y-%m-%d`	2026-10-17
`%H:%M:%S`	14:05:09
`%b %-d`	Oct 17
`%A`	Saturday
`%z / %Z`	-0500 / CDT
`%s`	Unix seconds

PARSE / BUILD

CALL	FROM
`Time.now`	clock
`Time.at(n)`	Unix seconds
`Time.parse(s)`	loose string
`Time.iso8601(s)`	ISO string
`Date.strptime(s, fmt)`	exact format

⚠ COMMON MISTAKES

- Storing local times without offsets and getting DST bugs; store UTC and convert for display
- Adding `86400` seconds for "tomorrow" across a DST change; use `Date + 1`
- Using `Date.parse` on ambiguous strings like `"01/02/03"`; use `strptime` with a format

✓ TIPS & BEST PRACTICES

- Measure durations with the monotonic clock (`Process::CLOCK_MONOTONIC`), not `Time.now`
- `Time#round` / `floor(2)` trim fractional seconds
- `DateTime` is deprecated in practice; use `Time` for moments and `Date` for days

QUICK SUMMARY

- `Time.now` / `Time.at` — moments
- `Date.new` / `Date.today` — calendar days
- `strftime` — format
- `iso8601` / `strptime` — parse

Standard Library — OS & Processes

STANDARD LIBRARY: OS AND PROCESSES

ENV reads environment variables, `system` runs a command and returns `true` / `false`, and backticks capture output. **`Open3.capture3`** is the safe, complete option: it returns stdout, stderr and the exit status separately. `exit` / `abort` end the program, and **`OptionParser`** handles command-line flags.

NOTES

- `ENV["X"]` returns a String or `nil`
- Pass commands as **separate arguments** to avoid shell injection
- `$?` holds the last child process status
- `abort("msg")` prints to stderr and exits with status 1

ENV, COMMANDS, OPTIONPARSER

```ruby
ENV["HOME"]                          # "/Users/ada"
ENV.fetch("API_KEY")                 # KeyError if unset
ENV.fetch("PORT", "3000").to_i       # 3000

system("ls", "-l")                   # true (output to terminal)
out = `git rev-parse --short HEAD`   # captured stdout
$?.success?                          # true

require "open3"
stdout, stderr, status = Open3.capture3("grep", "-c", "ERR", "app.log")
status.exitstatus                    # 0

require "optparse"
opts = { verbose: false }
OptionParser.new do |o|
  o.banner = "Usage: tool [options] FILE"
  o.on("-v", "--verbose", "Talk more") { opts[:verbose] = true }
  o.on("-n", "--count N", Integer)    { |n| opts[:count] = n }
end.parse!                           # removes flags from ARGV
abort("missing FILE") if ARGV.empty?
at_exit { puts "done" }
exit(0)
```

CODE EXPLAINED

- → `ENV.fetch` — fail fast when a variable is required
- → `system("ls", "-l")` — args as a list, no shell parsing
- → backticks — capture stdout as a string
- → `Open3.capture3` — stdout, stderr and status
- → `OptionParser#parse!` — parse and remove flags from `ARGV`

THE RUBY WAY

- `system("rm #{file}")` → `system("rm", file)`
- Backticks when you need stderr → `Open3.capture3`
- Hand-parsing `ARGV` for flags → `OptionParser`

IN RAILS

- **Rails** · `Rails.env.production?` instead of reading `ENV["RAILS_ENV"]`
- **Rails** · keep secrets in `Rails.application.credentials`

RUNNING COMMANDS

METHOD	RETURNS
`system(cmd, *args)`	true / false / nil
`cmd` / `%x(cmd)`	stdout String
`Open3.capture3`	[out, err, status]
`spawn(cmd)`	pid (async)
`exec(cmd)`	replaces this process

EXITING

CALL	EFFECT
`exit` / `exit(0)`	normal exit, runs at_exit
`exit(1)`	failure status
`abort("msg")`	stderr + status 1
`exit!`	skip at_exit and ensure

⚠ COMMON MISTAKES

- Interpolating user input into a command string; that's shell injection. Pass arguments separately
- Forgetting `ENV` values are strings: `ENV["DEBUG"] == true` is never true
- Ignoring exit statuses; check `$?.success?` or the `status` from `capture3`

✓ TIPS & BEST PRACTICES

- `system(..., exception: true)` raises if the command fails (Ruby 2.6+)
- `ENV.fetch("X") { raise "set X" }` gives a clear startup error
- OptionParser builds `--help` output automatically from your `on` descriptions

QUICK SUMMARY

- `ENV` / `ENV.fetch` — environment
- `system` / backticks / `Open3` — run commands
- `exit` / `abort` / `at_exit` — program end
- `OptionParser` — CLI flags

Minitest

MINITEST

Minitest is Ruby's built-in testing framework (a bundled gem), fast and plain Ruby. Test classes inherit from `Minitest::Test`, test methods start with **`test_`**, and assertions like `assert_equal expected, actual` check results. It also has a spec-style DSL and simple mocks and stubs.

MINITEST NOTES

- Run with `ruby -Ilib test/x_test.rb` or `rake test`; add `-n /name/` to filter
- Order is `assert_equal expected, actual`
- Tests run in random order to catch hidden dependencies
- `setup` / `teardown` run around every test

A TEST FILE

```
require "minitest/autorun"
require_relative "../lib/cart"

class CartTest < Minitest::Test
  def setup
    @cart = Cart.new
  end

  def test_total_adds_prices
    @cart.add(price: 2.5, qty: 2)
    assert_equal 5.0, @cart.total
  end

  def test_rejects_negative_qty
    error = assert_raises(ArgumentError) { @cart.add(price: 1, qty: -1) }
    assert_match(/qty/, error.message)
  end
end
```

CODE EXPLAINED

→ `Minitest::Test` — base class for test cases

→ `setup` — fresh state before each test

→ `assert_equal 5.0, @cart.total` — expected first

→ `assert_raises` — returns the exception for further checks

THE RUBY WAY

- `assert x == y` → `assert_equal y, x` (better failure messages)
- Shared state in class variables → `setup` per test
- Huge test methods → one behavior per `test_` method

IN RAILS

- **Rails** · tests inherit from `ActiveSupport::TestCase` and use YAML fixtures
- **Rails** · `bin/rails test test/models` runs a directory; `test "name" do` defines tests

ASSERTIONS

ASSERTION	PASSES WHEN
`assert x`	x truthy
`assert_equal e, a`	e == a
`assert_nil x`	x is nil
`assert_includes coll, x`	coll includes x
`assert_raises(E) { }`	block raises E
`refute_*`	negated forms

SPEC STYLE

SPEC	EQUIVALENT
`describe Cart do`	test class
`it "is empty" do`	test method
`_(x).must_equal 5`	assert_equal 5, x
`before do`	setup

⚠ COMMON MISTAKES

- Swapping expected and actual; failure messages then read backwards
- Naming a method without the `test_` prefix; Minitest silently skips it
- Depending on test order; random order will make the suite flaky

✓ TIPS & BEST PRACTICES

- `Minitest::Mock` and `obj.stub(:method, value) { }` cover simple doubles (require `minitest/mock`)
- Run with `--seed 1234` to reproduce an order-dependent failure
- Add `minitest-reporters` for colored, readable output

QUICK SUMMARY

- `Minitest::Test` + `test_` methods — test cases
- `assert_equal expected, actual` — main assertion
- `setup` / `teardown` — per-test hooks
- `rake test` — run the suite

RSpec Basics

RSPEC BASICS

RSpec is the most widely used Ruby testing framework, built around a readable DSL: **`describe`** groups examples, **`context`** describes a situation, **`it`** defines an example, and **`expect(actual).to matcher`** states the expectation. `let` defines lazy helpers and `before` runs setup.

RSPEC NOTES

- Files end in `_spec.rb` under `spec/`
- Run with `bundle exec rspec` , or `rspec spec/x_spec.rb:12` for one example
- `let` is lazy and memoized per example; `let!` runs eagerly
- `subject` names the object under test

A SPEC FILE

```ruby
require "cart"

RSpec.describe Cart do
  subject(:cart) { described_class.new }
  let(:pen) { { price: 2.5, qty: 2 } }

  it "starts empty" do
    expect(cart.items).to be_empty
  end

  context "with items" do
    before { cart.add(**pen) }

    it "totals the prices" do
      expect(cart.total).to eq(5.0)
    end
  end
end
```

CODE EXPLAINED

- → `RSpec.describe Cart` — example group for the class
- → `subject(:cart)` — named object under test
- → `let(:pen)` — lazy, memoized helper
- → `context` + `before` — shared setup for a situation
- → `expect(x).to eq(y)` — the expectation

THE RUBY WAY

- Instance variables in `before` → `let`
- `it "works"` → describe the behavior: `it "totals the prices"`
- Repeated setup in each `it` → `before` in a `context`

IN RAILS

- **Rails** · `rspec-rails` adds model, request and system specs (`type: :request`)
- **Rails** · FactoryBot builds records: `let(:user) { create(:user) }`

STRUCTURE

KEYWORD	ROLE
`describe`	group by class / method
`context`	group by situation
`it / specify`	one example
`before / after`	hooks
`let / let!`	lazy / eager helpers
`subject`	object under test

CLI OPTIONS

OPTION	EFFECT
`file.rb:LINE`	run one example
`--only-failures`	rerun failures
`--format documentation`	readable output
`--tag focus`	filter by tag
`--seed N`	reproduce order

⚠ COMMON MISTAKES

- Overusing `let!` ; eager setup slows every example even when it isn't needed
- Sharing state between examples through instance variables set outside hooks
- Putting several unrelated expectations in one `it` ; the first failure hides the rest

✓ TIPS & BEST PRACTICES

- Name contexts "when …" / "with …" so output reads like documentation
- `--only-failures` (with `example_status_persistence_file_path`) shortens the fix-test loop
- `described_class` keeps specs working when the class is renamed

QUICK SUMMARY

- `describe` / `context` / `it` — structure
- `expect(x).to matcher` — expectations
- `let` / `subject` / `before` — setup
- `rspec file:line` — run one example

RSpec Matchers & Mocks

RSPEC MATCHERS AND MOCKS

Matchers describe expectations: equality (`eq`), truthiness (`be_truthy`), collections (`include`, `contain_exactly`), errors (`raise_error`) and changes (`change { }`). **Test doubles** stand in for collaborators: `double` creates a fake object, `allow(...).to receive` stubs a method, and `expect(...).to receive` verifies a call happened.

NOTES

- `to` / `not_to` flip any matcher
- Predicate matchers: `be_empty` calls `empty?`
- `instance_double(Class)` verifies stubbed methods really exist
- Spies: `have_received` checks calls after the fact

MATCHERS AND DOUBLES

```
expect(total).to eq(5.0)
expect(user).to be_admin                    # calls admin?
expect(list).to include(3)
expect(list).to contain_exactly(3, 1, 2)  # any order
expect(name).to match(/^A/)
expect(price).to be_within(0.01).of(9.99)
expect { cart.add(qty: -1) }.to raise_error(ArgumentError, /qty/)
expect { cart.add(qty: 1) }.to change { cart.count }.by(1)

mailer = instance_double(Mailer)
allow(mailer).to receive(:deliver).and_return(true)
Signup.new(mailer).call("a@b.c")
expect(mailer).to have_received(:deliver).with("a@b.c")

api = double("api", fetch: { id: 1 })      # quick fake
expect(api).to receive(:fetch).once
api.fetch
allow(Time).to receive(:now).and_return(Time.at(0))
```

CODE EXPLAINED

- → `be_admin` — predicate matcher for `admin?`
- → `contain_exactly` — same elements, any order
- → `raise_error` / `change { }.by` — block expectations
- → `instance_double` — fake that only allows real methods
- → `have_received` — verify after the call (spy)

THE RUBY WAY

- `expect(x == y).to be true` → `expect(x).to eq(y)`
- `double` for real classes → `instance_double` (catches typos)
- Stubbing the object under test → stub its collaborators

COMMON MATCHERS

MATCHER	CHECKS
`eq(x) / eql(x)`	== / eql?
`be(x)`	same object
`be_nil / be_truthy`	nil / truthiness
`include(x)`	collection / string contains
`start_with / end_with`	prefix / suffix
`have_attributes(k: v)`	readers match

DOUBLES

API	USE
`double(name, m: v)`	simple fake
`instance_double(Cls)`	verified fake
`allow(o).to receive(:m)`	stub
`expect(o).to receive(:m)`	must be called
`.with(args) / .and_return(v)`	arguments / result

⚠ COMMON MISTAKES

- Using `be` when you mean `eq`; `be` checks identity, so `expect("a").to be("a")` fails
- Stubbing methods that don't exist on the real class with plain `double`; tests pass while code is broken
- Setting `expect(...).to receive` after the call; use `have_received` (spy style) instead

✓ TIPS & BEST PRACTICES

- Compose matchers: `include(a_hash_including(id: 1))`
- `aggregate_failures` reports every failed expectation in one example
- Freeze time with `travel_to` (Rails) or stub `Time.now` only at the edges

QUICK SUMMARY

- `eq` / `include` / `match` — value matchers
- `raise_error` / `change` — block matchers
- `instance_double` — verified doubles
- `allow` / `expect … to receive` — stub / verify

Rake

RAKE

Rake is Ruby's task runner (a bundled gem), like `make` written in Ruby. Tasks are defined in a **`Rakefile`** with `task`, documented with `desc`, grouped with `namespace`, and can depend on other tasks. `rake -T` lists documented tasks, and `file` tasks only rebuild outputs when their inputs change.

RAKE NOTES

- Only tasks with a `desc` appear in `rake -T`
- `task default: :test` sets what plain `rake` runs
- Arguments: `rake "greet[Ada]"`
- `Rake::TestTask` wires up Minitest suites

RAKEFILE

```ruby
require "rake/testtask"

desc "Say hello"
task :hello, [:name] do |_t, args|
  puts "Hello, #{args.fetch(:name, 'world')}"
end

namespace :db do
  task :create do
    puts "creating…"
  end

  task reset: [:drop, :create]       # prerequisites run first
  task(:drop) { puts "dropping…" }
end

file "build/app.css" => ["src/app.scss"] do |t|   # only if stale
  sh "sass #{t.prerequisites.first} #{t.name}"
end

Rake::TestTask.new(:test) { |t| t.pattern = "test/**/*_test.rb" }
task default: :test
```

CODE EXPLAINED

- → `desc` — documents the task for `rake -T`
- → `task :hello, [:name]` — task with arguments
- → `namespace :db` — `rake db:create`
- → `reset: [:drop, :create]` — prerequisites
- → `file` task — runs only when the output is older than inputs

THE RUBY WAY

- Shell scripts for project chores → Rake tasks in Ruby
- Copy-pasted setup in tasks → prerequisites
- Long task names → namespaces

IN RAILS

- **Rails** · custom tasks go in `lib/tasks/*.rake`
- **Rails** · `task cleanup: :environment do` loads the app (models, config) first
- **Rails** · `bin/rails db:migrate` and `bin/rails -T` run Rake tasks

DSL

METHOD	PURPOSE
`task :name`	define a task
`desc "…"`	describe next task
`namespace :ns`	group tasks
`file "out" => [in]`	build rule

CLI

COMMAND	DOES
`rake`	default task
`rake -T`	list documented tasks
`rake ns:task`	namespaced task
`rake "t[a,b]"`	pass arguments

⚠ COMMON MISTAKES

- Forgetting the quotes around `rake "task[arg]"` in zsh; brackets are glob characters there
- Omitting `desc` and wondering why the task doesn't show in `rake -T`
- Putting slow `require`s at the top of the Rakefile; every task pays the load time

✓ TIPS & BEST PRACTICES

- `sh` raises if the command fails, so the task stops instead of continuing silently
- Use `Rake::FileList["src/**/*.md"]` for file sets with exclusions
- Keep task bodies thin; call into your library code so it can be tested

QUICK SUMMARY

- `task` / `desc` / `namespace` — define tasks
- prerequisites — `task a: [:b]`
- `file` — rebuild only when stale
- `rake -T` — list tasks

Debugging & Linting

DEBUGGING AND LINTING

Ruby ships the **debug** gem: put **`binding.break`** (or `debugger`) in code to stop and inspect, or `binding.irb` for a quick REPL at that point. **RuboCop** (or **Standard**) lints style and common bugs, and the stdlib **Logger** writes leveled log messages.

NOTES

- `ruby -r debug/start script.rb` starts under the debugger
- `binding.irb` works without extra gems
- `rubocop -a` auto-corrects safe offenses
- `Logger.new($stdout)` logs to the terminal

BREAKPOINTS, LOGGER, LINTING

```ruby
require "debug"
def total(items)
  sum = items.sum(&:price)
  binding.break if sum.negative?   # stop here (debug gem)
  sum
end

def parse(row)
  binding.irb                      # REPL with locals in scope
  row.split(",")
end

require "logger"
log = Logger.new($stdout, level: :info)
log.info("starting")               # I, [2026-10-17T…] INFO -- : starting
log.debug("details")               # hidden at :info level
log.error { "expensive #{dump}" }  # block: built only if logged

p obj.method(:save).source_location
```

CODE EXPLAINED

- → `binding.break` — breakpoint with the full debugger
- → `binding.irb` — quick interactive inspection
- → `Logger.new($stdout, level: :info)` — leveled logging
- → `log.error { }` — lazy message, built only if needed
- → `source_location` — where a method is defined

THE RUBY WAY

- `puts` debugging left in code → `binding.break` or `Logger`
- Logging with string building at debug level → block form `log.debug { }`
- Style debates in reviews → RuboCop / Standard in CI

IN RAILS

- **Rails** · `bin/rails console` is an IRB session with your app loaded
- **Rails** · `Rails.logger.info` writes to `log/development.log`; `debug(obj)` dumps in views

DEBUGGER COMMANDS

CMD	DOES
`n` / `s`	next line / step into
`c`	continue
`info`	show locals
`bt`	backtrace
`q`	quit

LOGGER LEVELS

LEVEL	USE
`debug`	details while developing
`info`	normal events
`warn`	something odd
`error`	an operation failed
`fatal`	the app can't continue

⚠ COMMON MISTAKES

- Committing `binding.irb` / `binding.break`; CI or production hangs waiting for input. Add a RuboCop rule (`Lint/Debugger`)
- Running `rubocop -A` across a large codebase without reviewing; unsafe corrections can change behavior
- Logging secrets or full request bodies at info level

✓ TIPS & BEST PRACTICES

- `p` returns its argument, so `p(expr)` is the quickest temporary probe
- Use `.rubocop.yml` to tune cops and `rubocop --auto-gen-config` to adopt it gradually
- VS Code's Ruby LSP integrates the `debug` gem for breakpoints in the editor

QUICK SUMMARY

- `binding.break` / `binding.irb` — stop and inspect
- `Logger` — leveled logs
- `rubocop -a` — lint + auto-fix
- `source_location` — find definitions

HTTP Clients

HTTP CLIENTS

The stdlib **`Net::HTTP`** handles HTTP without extra gems: `Net::HTTP.get` for quick reads, `Net::HTTP.start` for full control over headers, bodies and timeouts. **Faraday** is the most popular gem for real applications, adding middleware for JSON, retries and logging. Parse JSON responses with `JSON.parse`.

NOTES

- `require "net/http"` also loads `URI`
- Check `res.is_a?(Net::HTTPSuccess)` or `res.code` (a String)
- Always set `open_timeout` / `read_timeout`
- `Net::HTTP.get(uri)` handles https itself; with `start`, pass `use_ssl: true`

NET::HTTP AND FARADAY

```ruby
require "net/http"
require "json"

uri = URI("https://api.example.com/users/1")

res = Net::HTTP.start(uri.host, uri.port, use_ssl: true,
                      open_timeout: 3, read_timeout: 5) do |http|
  req = Net::HTTP::Post.new("/users",
                            "Content-Type" => "application/json")
  req["Authorization"] = "Bearer #{ENV.fetch('TOKEN')}"
  req.body = { name: "Ada" }.to_json
  http.request(req)
end
res.code                                        # "201"
JSON.parse(res.body) if res.is_a?(Net::HTTPSuccess)

require "faraday"
conn = Faraday.new("https://api.example.com") do |f|
  f.request :json
  f.response :json
end
conn.get("/users/1").body["name"]               # parsed hash
```

CODE EXPLAINED

- → `Net::HTTP.get(uri)` — one-line GET, body only
- → `Net::HTTP.start(...) do |http|` — reuse a connection, set timeouts
- → `Net::HTTP::Post.new` + `req.body` — build a request
- → `res.code` / `Net::HTTPSuccess` — check the status
- → Faraday `request :json` / `response :json` — encode / decode automatically

THE RUBY WAY

- `open-uri` `URI.open(url).read` in apps → `Net::HTTP` with timeouts
- Manual JSON encode/decode on every call → Faraday JSON middleware
- Ignoring status codes → `res.value` raises on non-2xx

NET::HTTP

CALL	DOES
`Net::HTTP.get(uri)`	GET body
`Net::HTTP.get_response(uri)`	GET response
`Net::HTTP::Get.new(path)`	build request
`http.request(req)`	send it

RESPONSE

METHOD	GIVES
`res.code`	"200" (String)
`res.body`	String
`res["Content-Type"]`	header
`res.is_a?(Net::HTTPSuccess)`	2xx?

⚠ COMMON MISTAKES

- Comparing `res.code == 200`; codes are strings, so use `"200"` or check the class
- No timeouts; a slow server can hang your process for a long time
- Building URLs by string concatenation; use `URI.join` and `URI.encode_www_form`

✓ TIPS & BEST PRACTICES

- Reuse one `Net::HTTP.start` block (or a Faraday connection) for many requests to the same host
- `URI.encode_www_form(q: "ruby", page: 2)` builds query strings safely
- Add `faraday-retry` for automatic retries with backoff

QUICK SUMMARY

- `Net::HTTP.get` — quick GET
- `Net::HTTP.start` — headers, body, timeouts
- `res.code` / `HTTPSuccess` — status
- Faraday — middleware-based client

Regular Expressions — Basics

/pattern/ · =~ · match · MatchData · named captures (?<name>) · $~ $1 · flags i m x

REGULAR EXPRESSIONS: BASICS

Regex literals are written `/pattern/flags`. `=~` returns the match position or `nil`, `match` returns a **MatchData** object (or `nil`), and `match?` returns a boolean without setting match globals. Named captures (`(?<year>\d+)`) let you read groups by name, and Ruby uses `\A` / `\z` for true string start / end.

REGEX NOTES

- `^` / `$` match at **line** boundaries; `\A` / `\z` at string boundaries
- After `=~`, `$~` is the MatchData and `$1`, `$2` the groups
- Interpolation works: `/#{Regexp.escape(word)}/`
- `Regexp.new(str)` builds a pattern at runtime

MATCHING

```
s = "Order 2026-10-17 total $42.50"
s =~ /\d{4}/                      # 6
$~[0]                             # "2026"
s.match?(/total/i)                # true

m = s.match(/(\d{4})-(\d\d)-(\d\d)/)
m[0]                              # "2026-10-17"
m[1]                              # "2026"
m.captures                        # ["2026", "10", "17"]

d = s.match(/(?<year>\d{4})-(?<mon>\d\d)/)
d[:year]                          # "2026"

if /\$(?<amount>[\d.]+)/ =~ s     # literal on the LEFT
  amount                          # "42.50" (local variable!)
end
"line1\nline2" =~ /^line2$/       # 6 (matches a line)
"line1\nline2".match?(/\Aline2/) # false (string start)
```

CODE EXPLAINED

- `=~` — position or `nil`; sets `$~`, `$1` …
- `match` — MatchData with groups, `pre_match`, `post_match`
- `(?<year>…)` — named capture, read with `m[:year]`
- `/(?<amount>…)/ =~ s` — creates local variables from names
- `\A` vs `^` — whole string vs any line

THE RUBY WAY

- `^…$` for validating input → `\A…\z`
- `!!(s =~ /x/)` → `s.match?(/x/)`
- Numbered groups in long patterns → named groups

IN RAILS

- **Rails** · `validates :zip, format: { with: /\A\d{5}\z/ }`; Rails rejects `^` / `$` here unless you set `multiline: true`

SYNTAX

PATTERN	MATCHES
`.`	any char except newline
`\d \w \s`	digit, word, space
`* + ? {n,m}`	repetition
`(…) / (?<n>…)`	group / named group
`\A \z \b`	string start / end, word edge

FLAGS

FLAG	EFFECT
`i`	case-insensitive
`m`	`.` matches newline
`x`	free-spacing, comments allowed
`o`	interpolate once

⚠ COMMON MISTAKES

- Validating with `^` and `$`; `"ok\n<script>"` passes `/^ok$/` because `$` matches before the newline
- Expecting `m` to mean multiline like in other languages; in Ruby `m` makes `.` match newlines (`^` / `$` are always per-line)
- Forgetting the named-capture local variables only appear when the regex literal is on the left of `=~`

✓ TIPS & BEST PRACTICES

- Use the `x` flag with comments for complex patterns
- `Regexp.escape(user_input)` before interpolating text into a pattern
- Ruby has a regex timeout: `Regexp.timeout = 1.0` guards against ReDoS

QUICK SUMMARY

- `/re/` — literal; `=~` — position
- `match` — MatchData
- `(?<name>…)` — named groups
- `\A` / `\z` — string anchors

Regex in String Methods

REGEX IN STRING METHODS

Most string methods accept a regex: **scan** finds all matches, **sub / gsub** replace the first / every match, **split** splits on a pattern, and `slice` / `[]` extract matches. `gsub` takes a replacement string with backreferences, a **hash**, or a **block** that computes each replacement.

NOTES

- `scan` with groups returns arrays of captures
- Backrefs in replacement strings: `'\1'` or `'\k<name>'`
- In a `gsub` block, `$1` / `$~` refer to the current match
- `Regexp.union` combines patterns or literal strings safely

SCAN, GSUB, SPLIT

```
s = "a1 b22 c333"
s.scan(/\d+/)                    # ["1", "22", "333"]
s.scan(/([a-z])(\d+)/)           # [["a", "1"], ["b", "22"], …]
s.gsub(/\d+/, "#")               # "a# b# c#"
s.gsub(/(\w)(\d+)/, '\2\1')      # "1a 22b 333c"
s.gsub(/\d+/) { it.size.to_s }   # "a1 b2 c3"

iso = /(?<y>\d+)-(?<m>\d+)-(?<d>\d+)/
"2026-10-17".sub(iso, '\k<d>/\k<m>/\k<y>')     # "17/10/2026"
"cat hat".gsub(/[ch]/, "c" => "b", "h" => "m")   # "bat mat"
"a, b;c  d".split(/[,;\s]+/)     # ["a", "b", "c", "d"]
"price: $42"[/\$(\d+)/, 1]       # "42"

stop = Regexp.union("cat", "dog", "a.b")   # literals escaped
"a.b and cats".scan(stop)        # ["a.b", "cat"]
Regexp.escape("1+1=2")           # "1\\+1=2"
```

CODE EXPLAINED

- → `scan` — all matches, or all capture groups
- → `'\2\1'` — swap groups (single quotes keep backslashes)
- → `gsub { }` — computed replacement per match
- → hash replacement — lookup table for each match
- → `Regexp.union` — one pattern from many alternatives

THE RUBY WAY

- `s.match(/x/)[1]` → `s[/x(…)/, 1]`
- Chained `gsub` for each word → `gsub(Regexp.union(words), map)`
- `split(" ")` with messy separators → `split(/[,;\s]+/)`

METHODS

METHOD	RETURNS
`scan(re)`	Array of matches
`sub / gsub(re, rep)`	new String
`sub! / gsub!`	String or nil
`split(re)`	Array
`s[re, n]`	group n or nil
`match?(re)`	boolean

REPLACEMENT FORMS

REPLACEMENT	EXAMPLE
`string`	`gsub(/x/, "y")`
`backrefs`	`'\1'`, `'\k<name>'`
`hash`	`gsub(/[ab]/, "a" => "1")`
`block`	`gsub(/\d/) { it * 2 }`
`\0`	whole match

⚠ COMMON MISTAKES

- Writing backrefs in double quotes (`"\1"`); that's an octal escape. Use `'\1'` or `"\\1"`
- Expecting `scan` with one group to return full matches; it returns the group contents
- Using `gsub!` in a chain; it returns `nil` when nothing changed

✓ TIPS & BEST PRACTICES

- `gsub` with a hash is a fast template filler: `tpl.gsub(/\{(\w+)\}/) { vals[$1] }`
- `String#tr` beats `gsub` for single-character mappings
- Test patterns interactively in `irb` or on rubular.com

QUICK SUMMARY

- `scan` — find all
- `sub` / `gsub` — replace first / all
- block / hash — dynamic replacements
- `Regexp.union` / `escape` — safe patterns

Threads & Mutex

THREADS AND MUTEX

`Thread.new` runs a block concurrently; `join` waits for it and `value` returns its result. CRuby has a **Global VM Lock (GVL)**, so threads don't run Ruby code in parallel, but they **do overlap I/O** (network, files, sleep), which makes them useful for concurrent requests. Shared mutable state needs a **`Mutex`**, and **`Thread::Queue`** passes work between threads safely.

THREAD NOTES

- Exceptions in a thread surface on `join` / `value`
- Ruby's GVL means CPU-bound work needs Ractors or processes
- `Thread::Queue#pop` blocks until an item is available

THREADS, MUTEX, QUEUE

```ruby
urls = %w[https://a.com https://b.com https://c.com]
threads = urls.map do |url|
  Thread.new(url) { |u| Net::HTTP.get(URI(u)).size }   # I/O overlaps
end
threads.map(&:value)             # [1256, 3471, 902]

count = 0
lock = Mutex.new
10.times.map {
  Thread.new { 1000.times { lock.synchronize { count += 1 } } }
}.each(&:join)
count                            # 10000 (always)

queue = Thread::Queue.new
workers = 3.times.map do
  Thread.new do
    while (job = queue.pop)      # nil after close
      puts "done #{job}"
    end
  end
end
5.times { queue << it }
queue.close
workers.each(&:join)
```

CODE EXPLAINED

- → `Thread.new(url) { |u| }` — pass values in as arguments
- → `value` — joins and returns the block's result
- → `lock.synchronize { }` — one thread at a time in the block
- → `Thread::Queue` — thread-safe work queue
- → `queue.close` — `pop` returns `nil`, ending the workers

THE RUBY WAY

- `count += 1` from many threads → `Mutex#synchronize`
- Polling shared arrays → `Thread::Queue`
- Unbounded `Thread.new` per item → a fixed worker pool

THREAD API

CALL	DOES
`Thread.new { }`	start a thread
`t.join`	wait
`t.value`	wait + result
`Thread.current[:k]`	fiber-local storage

SYNC TOOLS

TOOL	USE
`Mutex`	exclusive section
`Thread::Queue`	producer / consumer
`SizedQueue`	bounded queue (backpressure)

⚠ COMMON MISTAKES

- Expecting threads to speed up CPU-heavy work in CRuby; the GVL serializes Ruby code
- Updating shared counters or arrays without a lock; `+=` isn't atomic
- Forgetting to `join`; the main program exits and kills unfinished threads

✓ TIPS & BEST PRACTICES

- `concurrent-ruby` provides thread pools, futures and atomic counters
- Keep the critical section inside `synchronize` as small as possible
- Use `SizedQueue` so fast producers can't exhaust memory

QUICK SUMMARY

- `Thread.new` / `join` / `value` — run and wait
- GVL — concurrency for I/O, not CPU
- `Mutex#synchronize` — protect shared state
- `Thread::Queue` — pass work safely

Fibers & the Fiber Scheduler

FIBERS AND THE FIBER SCHEDULER

A **Fiber** is a lightweight, manually scheduled coroutine: `resume` runs it until it calls `Fiber.yield`, which hands a value back. Since Ruby 3.0 a **Fiber Scheduler** can make blocking I/O non-blocking, so thousands of fibers can wait on sockets concurrently. The **async** gem provides a production scheduler.

FIBER NOTES

- Fibers never run in parallel; they switch only at yield points or I/O
- `Fiber.new(blocking: false)` fibers use the scheduler
- `async` gives you tasks, timeouts and HTTP clients

FIBERS AND ASYNC

```ruby
gen = Fiber.new do
  Fiber.yield 1
  Fiber.yield 2
  3                              # final value
end
gen.resume                       # 1
gen.resume                       # 2
gen.resume                       # 3

echo = Fiber.new { |x| loop { x = Fiber.yield(x * 2) } }
echo.resume(5)                   # 10
echo.resume(7)                   # 14

require "async"
require "async/http/internet"
Async do |task|
  internet = Async::HTTP::Internet.new
  pages = %w[https://a.com https://b.com].map do |url|
    task.async { internet.get(url).read.size }   # concurrent
  end
  p pages.map(&:wait)            # [1256, 3471]
ensure
  internet&.close
end
```

CODE EXPLAINED

- → `Fiber.yield 1` — pause and return 1 to `resume`
- → `resume(7)` — the value becomes `Fiber.yield`'s return
- → `alive?` — false after the block finishes
- → `Async do |task|` — starts a scheduler event loop
- → `task.async { }` — child task; `wait` gets its result

THE RUBY WAY

- Manual fiber juggling for generators → `Enumerator.new` (sheet 105)
- Threads for thousands of mostly idle connections → `async` fibers
- Callback-style I/O → plain blocking calls inside `Async`

FIBER API

CALL	DOES
`Fiber.new { }`	create
`f.resume(x)`	run / continue
`Fiber.yield(v)`	pause, return v
`Fiber.scheduler`	current scheduler

CONCURRENCY OPTIONS

TOOL	BEST FOR
`Thread`	moderate I/O, simple code
`Fiber + async`	many concurrent I/O tasks
`Ractor`	parallel CPU work (sheet 096)
`Process / fork`	isolation, CPU work

⚠ COMMON MISTAKES

- Calling `resume` on a finished fiber; it raises `FiberError: dead fiber called`
- Expecting fibers to run CPU work in parallel; they're cooperative and single-threaded
- Mixing blocking C extensions inside `Async`; calls the scheduler can't hook will block every task

✓ TIPS & BEST PRACTICES

- `Async::Barrier` waits for a group of tasks; `Async::Semaphore` limits concurrency
- Falcon is a fiber-based web server built on `async`
- Most programs never touch `Fiber` directly; use `Enumerator` or `async`

QUICK SUMMARY

- `Fiber.new` / `resume` / `Fiber.yield` — coroutines
- Fiber Scheduler — non-blocking I/O
- `async` — production scheduler
- cooperative — no parallel CPU

Ractors

RACTORS

Ractors (Ruby 3.0+) run Ruby code in **true parallel**, each with its own GVL. They share almost nothing: objects pass between Ractors by **copying or moving**, and only **shareable** (deeply frozen) objects can be read from several Ractors. Ruby 4.0 replaces `Ractor.yield` / `take` with **`Ractor::Port`** for communication.

RACTOR NOTES

- Still marked experimental; Ruby warns on first use
- Many gems aren't Ractor-safe (global state, C extensions)
- `Ractor.make_shareable` deep-freezes an object
- Ruby 4.0: `Ractor#join` / `#value` and `Ractor::Port` replace `take` / `yield`

RACTOR BASICS (RUBY 4.0)

```ruby
def fib(n) = n < 2 ? n : fib(n - 1) + fib(n - 2)

rs = 4.times.map { |i| Ractor.new(30 + i) { |n| fib(n) } }
rs.map(&:value)                  # computed in parallel

port = Ractor::Port.new
worker = Ractor.new(port) do |out|
  while (msg = Ractor.receive) != :stop
    out << msg.upcase            # send results back
  end
end
worker << "a"                    # Ractor#send
worker << "b"
port.receive                     # "A"
port.receive                     # "B"
worker << :stop
worker.join

CONFIG = Ractor.make_shareable({ level: 3 })
Ractor.new { CONFIG[:level] }.value   # 3 (shareable constant)
```

CODE EXPLAINED

- → `Ractor.new(arg) { |n| }` — pass arguments in (copied)
- → `value` — wait for the Ractor's final result
- → `Ractor::Port` — a channel others can send to
- → `Ractor.receive` / `<<` — the Ractor's own inbox
- → `make_shareable` — deep-freeze so any Ractor can read it

THE RUBY WAY

- Threads for CPU-bound math → Ractors (or processes)
- Mutable globals read inside Ractors → shareable constants
- `Ractor.yield` / `take` (3.x) → `Ractor::Port` (4.0)

RACTOR API

CALL	DOES
`Ractor.new(*args) { }`	start
`r << msg / r.send(msg)`	to its inbox
`Ractor.receive`	read own inbox
`r.value / r.join`	result / wait
`Ractor::Port.new`	create a channel

SHARING RULES

OBJECT	SHAREABLE?
`Integer, Symbol, nil`	yes
`frozen String`	yes
`deep-frozen Hash / Array`	yes
`mutable object`	no (copied or moved)

⚠ COMMON MISTAKES

- Accessing non-shareable constants or globals inside a Ractor; it raises `Ractor::IsolationError`
- Using 3.x `Ractor.yield` / `take` code on 4.0; switch to ports and `value`
- Assuming every gem works inside Ractors; check before relying on them in production

✓ TIPS & BEST PRACTICES

- Start with pure functions on plain data; that's where Ractors work best today
- Pass `move: true` to `send` to transfer large objects without copying
- Benchmark against processes (`fork` , `parallel` gem); they're often simpler for CPU work

QUICK SUMMARY

- `Ractor.new` — parallel execution
- `Ractor::Port` — 4.0 message channels
- `value` / `join` — get results
- shareable objects — deeply frozen data

Databases

DATABASES

Ruby talks to databases through driver gems: **sqlite3** for SQLite, pg for PostgreSQL, mysql2 for MySQL. Always use **placeholders** (? or $1) for values to avoid SQL injection. **Sequel** is a lightweight toolkit with a query DSL, and **ActiveRecord** (Rails' ORM) also works standalone.

NOTES

- `gem "sqlite3"` / `gem "pg"` in your Gemfile
- `results_as_hash = true` returns rows as hashes
- Wrap related writes in a transaction
- Close connections (or use a connection pool)

SQLITE3 AND SEQUEL

```ruby
require "sqlite3"
db = SQLite3::Database.new("app.db")
db.results_as_hash = true
db.execute <<~SQL
  CREATE TABLE IF NOT EXISTS users
    (id INTEGER PRIMARY KEY, name TEXT, age INT)
SQL
db.execute("INSERT INTO users (name, age) VALUES (?, ?)", ["Ada", 36])
db.execute("SELECT * FROM users WHERE age > ?", [30]).each do |row|
  puts row["name"]                          # Ada
end
db.transaction do
  db.execute("UPDATE users SET age = age + 1 WHERE name = ?", ["Ada"])
end

require "sequel"
DB = Sequel.sqlite("app.db")
DB[:users].where { age > 30 }.order(:name).select_map(:name)  # ["Ada"]
DB[:users].insert(name: "Lin", age: 41)
DB[:users].where(name: "Lin").update(age: 42)
```

CODE EXPLAINED

- → `SQLite3::Database.new` — open (or create) a database file
- → `results_as_hash` — rows as `{"name" => …}`
- → `execute(sql, [values])` — placeholders keep values safe
- → `transaction do` — all-or-nothing writes
- → Sequel `DB[:users].where { }` — composable query DSL

THE RUBY WAY

- `"... WHERE name = '#{name}'"` → placeholders: `"name = ?", [name]`
- Hand-written SQL strings for every query → Sequel datasets
- Opening a connection per query → one connection or a pool

IN RAILS

- **Rails** · `User.where("age > ?", 30)` and `User.find_by(name:)` (see a Rails set for the full ORM)
- **Rails** · migrations (`bin/rails g migration`) manage schema instead of `CREATE TABLE`

DRIVERS

GEM	DATABASE
`sqlite3`	SQLite
`pg`	PostgreSQL
`mysql2 / trilogy`	MySQL
`sequel`	any (toolkit)
`activerecord`	any (ORM)

SQLITE3 API

CALL	RETURNS
`execute(sql, args)`	array of rows
`get_first_row(sql)`	one row
`get_first_value(sql)`	one value
`last_insert_row_id`	new id
`transaction { }`	commit / rollback

⚠ COMMON MISTAKES

- Interpolating user input into SQL; that's SQL injection. Always use placeholders
- Forgetting `results_as_hash` and indexing rows by position after the schema changes
- Running many inserts without a transaction; each commits separately and is slow

✓ TIPS & BEST PRACTICES

- `db.execute_batch(sql)` runs a multi-statement script
- Sequel's `DB.loggers << Logger.new($stdout)` prints every query
- Use SQLite's `:memory:` database for fast tests

QUICK SUMMARY

- driver gems — sqlite3, pg, mysql2
- placeholders — never interpolate values
- `transaction` — atomic writes
- Sequel / ActiveRecord — higher-level access

Marshal & Serialization

MARSHAL AND SERIALIZATION

Marshal converts almost any Ruby object to a binary string and back, preserving classes and object graphs. It's fast and convenient for caches, but the format is **Ruby-only** and **Marshal.load on untrusted data is a security risk** because it can instantiate arbitrary classes. For data that leaves your process, prefer JSON, YAML or MessagePack.

NOTES

- Procs, IO objects and singleton methods can't be marshaled
- Format can change between major Ruby versions
- `marshal_dump` / `marshal_load` customize what's saved
- `Struct` / `Data` convert cleanly with `to_h`

MARSHAL AND ALTERNATIVES

```ruby
data = { user: "Ada", tags: [:admin], at: Time.now }
bytes = Marshal.dump(data)              # binary String
copy  = Marshal.load(bytes)             # same structure, new objects
copy[:tags]                             # [:admin] (symbols survive)


class Session
  def initialize(id, conn) = (@id, @conn = id, conn)
  def marshal_dump = [@id]              # skip the connection
  def marshal_load((id)) = @id = id
end
Marshal.load(Marshal.dump(Session.new(1, $stdout)))

Point = Data.define(:x, :y)
json = Point.new(1, 2).to_h.to_json     # '{"x":1,"y":2}'
Point.new(**JSON.parse(json, symbolize_names: true))  # round trip

require "msgpack"
MessagePack.unpack({ a: 1 }.to_msgpack) # {"a" => 1}
```

CODE EXPLAINED

- → `Marshal.dump` / `load` — full Ruby object round-trip
- → symbols and `Time` survive, unlike JSON
- → `marshal_dump` / `marshal_load` — save only some state
- → `to_h` + JSON — portable round-trip for value objects

THE RUBY WAY

- `Marshal.load(params[:data])` → JSON with explicit fields
- `Marshal` as a deep copy for objects with IO → `initialize_copy`
- Custom string formats → JSON or MessagePack

IN RAILS

- **Rails** · `serialize :prefs, coder: JSON` stores a column as JSON
- **Rails** · ActiveJob serializes arguments; pass ids or simple values, not whole objects

FORMATS

FORMAT	STRENGTHS
`Marshal`	any Ruby object, fast, Ruby-only
`JSON`	portable, human-readable
`YAML`	config files
`MessagePack`	compact binary, cross-language

MARSHAL LIMITS

OBJECT	DUMPABLE?
`Hash, Array, String`	yes
`Struct, Data, Time`	yes
`Proc / lambda`	no
`IO / File / Socket`	no
`objects with singleton methods`	no

⚠ COMMON MISTAKES

- Calling `Marshal.load` on cookies, uploads or network data; it can lead to remote code execution
- Caching marshaled objects across Ruby upgrades and failing to load them later
- Marshaling objects that hold procs or IO; it raises `TypeError: no _dump_data is defined`

✓ TIPS & BEST PRACTICES

- Version your cache keys so a format change just misses the cache
- For APIs and storage, define `to_h` and rebuild with `new(**hash)`
- `Marshal.load(bytes, freeze: true)` returns deeply frozen objects

QUICK SUMMARY

- `Marshal.dump` / `load` — Ruby object serialization
- never load untrusted Marshal data
- `marshal_dump` / `marshal_load` — customize
- JSON / MessagePack — portable formats

Type Signatures: RBS & Sorbet

TYPE SIGNATURES: RBS AND SORBET

Ruby stays dynamically typed, but you can add **gradual typing**. **RBS** (bundled with Ruby) describes types in separate `.rbs` files under `sig/`, checked by **Steep**; **TypeProf** infers them from code. **Sorbet** (from Stripe) uses inline `sig { }` blocks in Ruby code and its own fast checker.

NOTES

- RBS files don't change runtime behavior
- `rbs-inline` lets you write RBS in comments next to the code
- Sorbet has `typed: false / true / strict` levels per file
- Editors use these signatures for completion and hover types

RBS FILE

```
# sig/cart.rbs
class Cart
  attr_reader items: Array[Item]
  def initialize: () -> void
  def add: (Item item, ?qty: Integer) -> self
  def total: () -> Float
  def find: (String sku) -> Item?          # may return nil
end
```

SORBET INLINE

```
# typed: strict
class Cart
  extend T::Sig

  sig { params(item: Item, qty: Integer).returns(T.self_type) }
  def add(item, qty: 1) = tap { @items << [item, qty] }

  sig { returns(Float) }
  def total = @items.sum { |i, q| i.price * q }
end
```

CODE EXPLAINED

- → `sig/*.rbs` — separate files; check with `steep check` (Sorbet: `srb tc`)
- → `?qty: Integer` — optional keyword
- → `Item?` — nilable return type
- → `sig { params(...).returns(...) }` — Sorbet inline signature
- → `# typed: strict` — Sorbet strictness level

THE RUBY WAY

- Type checks with `is_a?` everywhere → signatures + a checker
- Documenting types only in YARD comments → RBS or Sorbet sigs
- Typing everything at once → start with core classes and strict files

RBS SYNTAX

TYPE	MEANING
`Integer / String`	class instance
`Array[String]`	generic
`String?`	String or nil
`(A | B)`	union
`() -> void`	method returning nothing useful

TOOLS

TOOL	ROLE
`rbs`	signature language + stdlib sigs
`Steep`	type checker for RBS
`TypeProf`	infers RBS from code
`Sorbet / Tapioca`	inline sigs + RBI files
`Ruby LSP`	editor integration

⚠ COMMON MISTAKES

- Expecting RBS or Sorbet sigs to check types at runtime by default; RBS never does, Sorbet only with `sorbet-runtime`
- Mixing RBS and Sorbet RBI in one project without a plan; pick one toolchain
- Using `untyped` everywhere and getting no benefit

✓ TIPS & BEST PRACTICES

- `rbs prototype rb lib/cart.rb` generates a starting signature
- Use `typeprof` on small libraries to bootstrap signatures
- Gems can ship `sig/` folders; `rbs collection` downloads community signatures

QUICK SUMMARY

- RBS — separate signature files
- Steep / TypeProf — check / infer
- Sorbet — inline `sig { }`
- gradual — type the important parts first

Performance & Memory

PERFORMANCE AND MEMORY

Measure before optimizing: **Benchmark** (stdlib) and the **benchmark-ips** gem compare code, and `GC.stat` / `ObjectSpace` / **memory_profiler** show allocations. Common wins are fewer allocations (frozen strings, << instead of +), better algorithms (Sets, hashes), and enabling the **YJIT** JIT for long-running processes.

NOTES

- `ruby --yjit` or `RUBY_YJIT_ENABLE=1` turns on YJIT
- Ruby 4.0 adds the experimental **ZJIT** (`--zjit`)
- Allocations drive GC time; fewer objects = faster
- Profile with `stackprof` or `vernier` to find hot spots

MEASURING

```ruby
require "benchmark"
Benchmark.realtime { 1_000_000.times { "a" + "b" } }   # 0.08 (seconds)
Benchmark.bm(6) do |x|
  x.report("+=")  { s = +""; 10_000.times { s += "x" } }
  x.report("<<")  { s = +""; 10_000.times { s << "x" } }
end

require "benchmark/ips"
Benchmark.ips do |x|
  x.report("map+flatten") { [[1], [2]].map { it }.flatten }
  x.report("flat_map")    { [[1], [2]].flat_map { it } }
  x.compare!
end

GC.stat[:total_allocated_objects]      # running total
ObjectSpace.count_objects[:T_STRING]   # live strings
RubyVM::YJIT.enabled?                  # true with --yjit

require "memory_profiler"
MemoryProfiler.report { load_data }.pretty_print(scale_bytes: true)
```

CODE EXPLAINED

- → `Benchmark.realtime` — wall time of one block
- → `Benchmark.bm` — side-by-side timings
- → `benchmark-ips` — iterations per second with error margins
- → `GC.stat` / `ObjectSpace` — allocation and object counts
- → `MemoryProfiler.report` — where memory is allocated

THE RUBY WAY

- `str += x` in loops → `str << x` on a mutable buffer
- `arr.include?` in a loop → a `Set`
- `map { }.flatten` → `flat_map` ; `select { }.map { }` → `filter_map`

IN RAILS

- **Rails** · most slowness is N+1 queries; fix with `includes(:posts)` (the bullet gem detects them)
- **Rails** · `Rails.cache.fetch` and fragment caching avoid repeated work
- **Rails** · Rails 7.2+ enables YJIT by default when available

TOOLS

TOOL	FINDS
`Benchmark / benchmark-ips`	which version is faster
`stackprof / vernier`	CPU hot spots
`memory_profiler`	allocations by line
`derailed_benchmarks`	boot-time memory (Rails)
`GC.stat`	GC runs, heap size

QUICK WINS

CHANGE	WHY
`frozen_string_literal`	fewer allocations
`Set / Hash lookups`	O(1) vs O(n)
`each_with_object / <<`	avoid intermediate arrays
`lazy enumerators`	stop early
`--yjit`	JIT for hot methods

⚠ COMMON MISTAKES

- Optimizing without measuring; intuition about Ruby performance is often wrong
- Benchmarking with too few iterations; use `benchmark-ips` for stable numbers
- Turning on YJIT for short scripts and CLIs; warm-up cost can outweigh the gain

✓ TIPS & BEST PRACTICES

- Measure in production-like conditions (same Ruby, YJIT setting, data size)
- `GC.compact` and `RUBY_GC_HEAP_*` settings tune long-running processes
- Use `vernier` for profiling apps with threads or fibers

QUICK SUMMARY

- `Benchmark` / `benchmark-ips` — measure speed
- `memory_profiler` / `GC.stat` — measure memory
- fewer allocations — main optimization
- `--yjit` / `--zjit` — JIT compilers

Enumerable — Iterate & Transform

each · map · flat_map · filter_map · each_with_index · with_index(1) · each_with_object · zip

ENUMERABLE: ITERATE AND TRANSFORM

Enumerable is the mixin behind Array, Hash, Range, Set and any class that defines `each`. Its transform methods build new collections: **`map`** converts each element, **`flat_map`** maps and flattens one level, **`filter_map`** maps and drops `nil`/`false`, and **`each_with_object`** accumulates into an object you choose.

NOTES

- These return new arrays; the receiver is unchanged
- `map!` / `select!` exist on Array for in-place versions
- Without a block, most methods return an Enumerator
- `with_index(1)` adds a 1-based counter to any of them

TRANSFORMING COLLECTIONS

```
nums = [1, 2, 3, 4]
nums.map { it * 10 }                    # [10, 20, 30, 40]
nums.each_with_index.map { |n, i| n * i }   # [0, 2, 6, 12]
nums.map.with_index(1) { |n, i| "#{i}. #{n}" }  # ["1. 1", "2. 2", …]

[[1, 2], [3]].flat_map { it.map(&:succ) }   # [2, 3, 4]
%w[a1 b c3].filter_map { it[/\d/]&.to_i }   # [1, 3]

words = %w[apple avocado banana]
words.each_with_object(Hash.new(0)) { |w, h| h[w[0]] += 1 }
# {"a" => 2, "b" => 1}

nums.zip(%w[a b c d])                   # [[1, "a"], [2, "b"], …]
nums.each_slice(2).map(&:sum)           # [3, 7]
{ a: 1, b: 2 }.map { |k, v| "#{k}=#{v}" }   # ["a=1", "b=2"]
(1..3).map { it ** 2 }                  # [1, 4, 9]
```

CODE EXPLAINED

- → `map` — one output per input
- → `each_with_index.map` — use the index while mapping
- → `flat_map` — map then flatten one level
- → `filter_map` — map and drop falsy results in one pass
- → `each_with_object(obj)` — accumulate into `obj`, returned at the end

THE RUBY WAY

- `result = []; arr.each { result << f(x) }` → `arr.map { f(it) }`
- `map { }.compact` → `filter_map { }`
- `inject({}) { |h, x| h[x] = 1; h }` → `each_with_object({})`

IN RAILS

- **Rails** · `User.pluck(:email)` selects one column in SQL; `User.all.map(&:email)` loads every record
- **Rails** · `find_each` iterates large tables in batches of 1000

TRANSFORM METHODS

METHOD	RETURNS
`map / collect`	Array
`flat_map`	Array (flattened 1 level)
`filter_map`	Array (truthy results)
`each_with_object(o)`	o
`each_with_index`	receiver (or Enumerator)
`zip(other)`	Array of pairs

WORKS ON

CLASS	YIELDS
`Array / Set`	element
`Hash`	[key, value] → `|k, v|`
`Range`	each value
`IO / File`	each line
`Enumerator`	whatever it produces

⚠ COMMON MISTAKES

- Using `each` and expecting a transformed array; `each` returns the original receiver
- Mutating inside `map` (`map { it << x }`); you change the originals and get them back too
- Forgetting that `map` on a Hash returns an Array; add `.to_h` or use `transform_values`

✓ TIPS & BEST PRACTICES

- Chain without blocks: `arr.each_with_index.select { |x, i| i.even? }.map(&:first)`
- `sum`, `min_by`, `tally` (sheet 103) replace many hand-rolled `each_with_object` loops
- Any class with `each` + `include Enumerable` gets all of these (sheet 106)

QUICK SUMMARY

- `map` — transform each
- `flat_map` / `filter_map` — map + flatten / map + filter
- `each_with_object` — accumulate
- `with_index(1)` — numbered iteration

Enumerable — Filter & Find

ENUMERABLE: FILTER AND FIND

Filtering methods select elements by a condition: **`select`** (alias `filter`) keeps matches, **`reject`** drops them, and **`partition`** splits into both groups. **`find`** (alias `detect`) returns the first match, `grep` filters with ===, and the predicates **`any?` / `all?` / `none?` / `one?`** answer yes/no questions.

NOTES

- `find` stops at the first match; `select` always scans everything
- `grep(pattern)` uses `===`, so it accepts classes, ranges and regexes
- Predicates accept a pattern too: `nums.all?(Integer)`
- `take_while` / `drop_while` stop at the first non-match

FILTERING AND FINDING

```
nums = [3, 8, 1, 12, 7, 6]
nums.select(&:even?)              # [8, 12, 6]
nums.reject(&:even?)              # [3, 1, 7]
evens, odds = nums.partition(&:even?)  # [[8, 12, 6], [3, 1, 7]]
nums.find { it > 5 }              # 8
nums.find_index { it > 5 }        # 1
nums.take_while { it < 10 }       # [3, 8, 1]
nums.drop_while { it < 10 }       # [12, 7, 6]

%w[a.rb b.txt c.rb].grep(/\.rb\z/)    # ["a.rb", "c.rb"]
[1, "a", :b, 2.5].grep(Numeric)       # [1, 2.5]
nums.grep(5..10)                      # [8, 7, 6]
%w[a.rb b.txt].grep_v(/\.rb/)         # ["b.txt"]

nums.any? { it > 10 }             # true
nums.all?(Integer)                # true
nums.none?(&:negative?)           # true
[nil, false, 1].one?              # true (one truthy)
nums.count(&:even?)               # 3
```

CODE EXPLAINED

- → `select` / `reject` — keep / drop by condition
- → `partition` — both groups at once
- → `find` — first match or `nil`
- → `grep` — filter with `===` (regex, class, range)
- → `any?` / `all?` / `none?` / `one?` — boolean questions

THE RUBY WAY

- `select { }.first` → `find { }`
- `select { }.size > 0` → `any? { }`
- `select { |s| s =~ /x/ }` → `grep(/x/)`

IN RAILS

- **Rails** · `User.where(active: true)` filters in SQL; `User.all.select(&:active?)` filters in Ruby after loading all rows
- **AS** · `exclude?` reads better than `!include?`

FILTERS

METHOD	RETURNS
`select / filter`	matching elements
`reject`	non-matching elements
`partition`	[matching, non-matching]
`grep(pat) / grep_v(pat)`	by === / not
`take_while / drop_while`	prefix / rest

FINDERS & PREDICATES

METHOD	RETURNS
`find / detect`	first match or nil
`find_index`	index or nil
`any? / all?`	boolean
`none? / one?`	boolean
`count { }`	number of matches

⚠ COMMON MISTAKES

- Calling `[].all? { … }`; it's `true` for an empty collection, which can hide missing data
- Using `select` + `first` on large collections; `find` stops early
- Confusing `filter_map` (map + compact) with `select` (keep elements as they are)

✓ TIPS & BEST PRACTICES

- `grep` with a block maps the matches: `lines.grep(/ERROR (\d+)/) { $1 }`
- Use `lazy.select { }.first(5)` on huge or infinite sources (sheet 105)
- `min_by` / `max_by` beat `sort_by { }.first` when you only need one element

QUICK SUMMARY

- `select` / `reject` / `partition` — filter
- `find` — first match
- `grep` — filter with `===`
- `any?` / `all?` / `none?` — predicates

Enumerable — Reduce & Aggregate

ENUMERABLE: REDUCE AND AGGREGATE

Aggregation methods boil a collection down to one value or a summary: **sum**, **count**, **min_by / max_by**, **minmax**, **tally** (counts) and **group_by** (buckets). **reduce** (alias `inject`) is the general tool: it carries an accumulator through every element.

NOTES

- `reduce(:+)` / `reduce(0) { |acc, x| … }` — symbol or block form
- Without an initial value, `reduce` starts with the first element
- `sum` takes a start value and a block: `sum(0.0) { it.price }`
- `tally` counts equal values; `map` first to count by a key

AGGREGATING

```
nums = [4, 8, 15, 16, 23, 42]
nums.sum                          # 108
nums.sum { it * 2 }               # 216
nums.reduce(:*)                   # 7418880
nums.reduce(1) { |acc, n| acc * n }   # same, explicit start
nums.count(&:even?)               # 4
nums.minmax                       # [4, 42]
(nums.sum.to_f / nums.size).round(1)  # 18.0 (average)

people = [{ name: "Ada", dept: :eng, age: 36 },
          { name: "Bo",  dept: :ops, age: 29 },
          { name: "Cy",  dept: :eng, age: 41 }]
people.max_by { it[:age] }[:name]          # "Cy"
people.sum { it[:age] }                    # 106
people.group_by { it[:dept] }.transform_values(&:size)  # {eng: 2, …}
people.map { it[:dept] }.tally             # {eng: 2, ops: 1}
%w[a b a c a].tally.max_by(&:last)         # ["a", 3]
```

CODE EXPLAINED

- → `sum { }` — sum of computed values
- → `reduce(:*)` — fold with an operator
- → `reduce(1) { |acc, n| }` — fold with an initial accumulator
- → `group_by` — key → array of elements
- → `tally` — value → count

THE RUBY WAY

- `inject(0) { |s, x| s + x }` → `sum`
- `group_by { }.map { |k, v| [k, v.size] }.to_h` → `map { }.tally`
- `sort_by { }.last` → `max_by { }`

IN RAILS

- **Rails** · `Order.sum(:total)` and `Order.count` run as SQL on a relation
- **Rails** · `Order.group(:status).count` → `{"paid" => 12, …}` in one query

REDUCERS

METHOD	RESULT
`sum / sum { }`	number
`count / count { }`	integer
`min / max / minmax`	elements
`min_by / max_by`	element
`reduce(init) { }`	accumulator

SUMMARIES

METHOD	RESULT
`tally`	{value => count}
`group_by { }`	{key => [items]}
`chunk_while { }`	runs (sheet 104)
`each_with_object`	custom accumulator

⚠ COMMON MISTAKES

- `reduce` without an initial value on an empty array returns `nil`; pass a start value
- Summing floats for money; rounding errors accumulate. Use `BigDecimal` or integer cents
- Integer average: `sum / size` truncates; convert to a float first

✓ TIPS & BEST PRACTICES

- `sum` is faster and more accurate than `reduce(:+)` for floats (it uses Kahan summation)
- `minmax_by { }` returns both extremes in one pass
- Combine `group_by` with `transform_values` for per-group stats

QUICK SUMMARY

- `sum` / `count` / `minmax` — quick stats
- `reduce` — general fold
- `tally` — counts
- `group_by` — buckets

Enumerable — Slice & Chunk

ENUMERABLE: SLICE AND CHUNK

These methods split a collection into groups by **size** or by **relationships between neighbors**. `each_slice(n)` makes fixed-size chunks, `each_cons(n)` makes overlapping windows, and `chunk_while` / `slice_when` start a new group when adjacent elements stop (or start) satisfying a condition. `chunk` groups consecutive elements with the same key.

NOTES

- These return Enumerators; add `.to_a` or chain `map`
- `chunk` only groups **consecutive** equal keys (unlike `group_by`)
- `cycle(n)` repeats the collection n times (forever without n)
- `uniq { }` deduplicates by a computed key

SLICING AND CHUNKING

```
(1..7).each_slice(3).to_a         # [[1, 2, 3], [4, 5, 6], [7]]
(1..5).each_cons(2).to_a          # [[1, 2], [2, 3], [3, 4], [4, 5]]
[3, 5, 9].each_cons(2).map { |a, b| b - a }  # [2, 4]

nums = [1, 2, 3, 7, 8, 10]
nums.chunk_while { |a, b| b == a + 1 }.to_a
# [[1, 2, 3], [7, 8], [10]]
nums.slice_when { |a, b| b > a + 1 }.to_a     # same groups

temps = [12, 14, 21, 25, 19, 8]
temps.chunk { it > 18 ? :warm : :cool }.to_a
# [[:cool, [12, 14]], [:warm, [21, 25, 19]], [:cool, [8]]]

lines = ["# A", "x", "# B", "y", "z"]
lines.slice_before { it.start_with?("#") }.to_a
# [["# A", "x"], ["# B", "y", "z"]]
%w[apple Avocado bean].uniq { it[0].downcase }  # ["apple", "bean"]
```

CODE EXPLAINED

- → `each_slice(3)` — fixed-size, non-overlapping chunks
- → `each_cons(2)` — sliding window of neighbors
- → `chunk_while` — keep grouping while the condition holds
- → `chunk` — consecutive runs with the same key
- → `slice_before` — start a group at each matching element

THE RUBY WAY

- Index math for pairs (`arr[i], arr[i + 1]`) → `each_cons(2)`
- Manual "current group" loops → `chunk_while` / `slice_when`
- `group_by { }.values` when order matters → `chunk`

IN RAILS

- **Rails** · `User.in_batches(of: 500) { |rel| rel.update_all(flag: true) }` batches in SQL
- **AS** · `arr.in_groups_of(3, false)` is like `each_slice(3)`; `in_groups(3)` splits into 3 groups

FIXED SIZE

METHOD	FOR [1,2,3,4,5]
`each_slice(2)`	[1,2], [3,4], [5]
`each_cons(2)`	[1,2], [2,3], [3,4], [4,5]
`cycle.first(7)`	1,2,3,4,5,1,2

BY CONDITION

METHOD	SPLITS WHEN
`chunk_while { }`	block becomes false
`slice_when { }`	block becomes true
`slice_before { }`	element matches (start)
`chunk { }`	key changes

⚠ COMMON MISTAKES

- Expecting `chunk` to merge non-adjacent equal keys; use `group_by` for that
- Forgetting `.to_a` and printing an `Enumerator` object
- Calling `cycle` without a limit inside `map`; it never ends

✓ TIPS & BEST PRACTICES

- `each_cons(2).all? { |a, b| a <= b }` checks that an array is sorted
- `slice_when` is perfect for detecting gaps in sequences or time series
- `each_slice` keeps batch API calls under rate limits

QUICK SUMMARY

- `each_slice(n)` — chunks
- `each_cons(n)` — sliding windows
- `chunk_while` / `slice_when` — split by neighbors
- `chunk` — consecutive runs

Enumerators & Lazy Evaluation

ENUMERATORS AND LAZY EVALUATION

An **Enumerator** is an object that produces values on demand. Calling an iterator **without a block** returns one, and **external iteration** (`next`, `peek`, `rewind`) lets you pull values one at a time. `Enumerator.new` builds custom sequences, `Enumerator.produce` generates from a seed, and **`lazy`** chains transformations that only run as far as needed.

NOTES

- `next` raises `StopIteration` at the end (`loop` rescues it)
- Lazy chains need a terminal: `first(n)`, `to_a`, `force`, `each`
- `.eager` turns a lazy enumerator back into a normal one
- Infinite sequences are fine with `lazy` or `first(n)`

ENUMERATORS AND LAZY

```
e = [10, 20, 30].each            # no block → Enumerator
e.next                           # 10
e.peek                           # 20 (doesn't advance)
e.next                           # 20
e.rewind; e.next                 # 10

fib = Enumerator.new do |y|
  a, b = 0, 1
  loop { y << a; a, b = b, a + b }
end
fib.take(8)                      # [0, 1, 1, 2, 3, 5, 8, 13]

Enumerator.produce(1) { it * 2 }.take(5)   # [1, 2, 4, 8, 16]
Enumerator.produce([0, 1]) { |a, b| [b, a + b] }.take(4)

(1..Float::INFINITY).lazy
  .map { it * it }
  .select(&:even?)
  .first(3)                      # [4, 16, 36]
File.foreach("big.log").lazy.grep(/ERROR/).first(10)
```

CODE EXPLAINED

- → `each` without a block — returns an Enumerator
- → `next` / `peek` / `rewind` — external iteration
- → `Enumerator.new { |y| y << v }` — custom generator
- → `Enumerator.produce(seed) { }` — next value from the previous
- → `lazy … first(3)` — computes only what's needed

THE RUBY WAY

- Building a huge array to take a few → `lazy … first(n)`
- Fibers for generators → `Enumerator.new`
- `while` loops with manual state → `Enumerator.produce`

ENUMERATOR API

CALL	DOES
`e.next`	next value
`e.peek`	look ahead
`e.rewind`	start over
`e.size`	size if known
`e.with_index(1)`	add a counter

LAZY CHAIN

STEP	EXAMPLE
`start`	`(1..).lazy`
`transform`	`.map`, `.select`, `.reject`
`limit`	`.take(n)`, `.take_while`
`finish`	`.first(n)`, `.to_a`, `.force`

⚠ COMMON MISTAKES

- Calling `to_a` on an infinite enumerator; it never returns
- Using `lazy` for small arrays; the overhead makes it slower than eager methods
- Sharing one external enumerator between threads; `next` isn't thread-safe

✓ TIPS & BEST PRACTICES

- Return `enum_for(__method__)` when no block is given so your methods chain (sheet 106)
- `Enumerator::Lazy#filter_map` and `#compact` work lazily too
- `lazy` + `File.foreach` processes huge files with constant memory

QUICK SUMMARY

- Enumerator — values on demand
- `next` / `peek` — external iteration
- `Enumerator.new` / `produce` — generators
- `lazy` — compute only what's needed

Making Classes Enumerable

MAKING CLASSES ENUMERABLE

Define an **each** method that yields every element and **`include Enumerable`**, and your class gains `map`, `select`, `sort`, `min_by`, `first`, `lazy` and dozens more. Return an **Enumerator** when `each` is called without a block so chaining works. Add **`<=>`** and `include Comparable` to make the objects themselves comparable.

NOTES

- `sort` / `min` / `max` need the elements to implement `<=>`
- `to_enum(:each) { size }` lets enumerators report a size

A CUSTOM COLLECTION

```ruby
class Playlist
  include Enumerable

  def initialize(*songs) = @songs = songs

  def each
    return to_enum(:each) { @songs.size } unless block_given?
    @songs.each { yield it }
    self
  end
end

Song = Data.define(:title, :secs) do
  include Comparable
  def <=>(other) = secs <=> other.secs
end

list = Playlist.new(Song["A", 200], Song["B", 95], Song["C", 310])
list.map(&:title)                  # ["A", "B", "C"]
list.select { it.secs > 150 }.size # 2
list.min.title                     # "B" (uses Song#<=>)
list.sort.map(&:title)             # ["B", "A", "C"]
list.sum(&:secs)                   # 605
```

CODE EXPLAINED

- → `include Enumerable` — all collection methods from `each`
- → `return to_enum(:each) { size }` — blockless `each` returns an Enumerator
- → `yield it` — hand each element to the caller
- → `include Comparable` + `<=>` — `min`, `sort`, `<`, `between?`
- → `list.min` — Enumerable uses the elements' `<=>`

THE RUBY WAY

- Exposing the internal array (`attr_reader :songs`) → implement `each`
- Re-implementing `map`, `select` on your class → `include Enumerable`
- Forgetting the blockless case → `return to_enum(:each) unless block_given?`

YOU DEFINE

YOU GET	EXAMPLES	
`each`	Enumerable	map, select, find, sort_by, lazy
`<=>`	Comparable	<, >, between?, clamp
`each + <=> on elements`	min / max / sort	ordered output
`size (optional)`	faster count	`count` without iterating

DELEGATION OPTIONS

APPROACH	WHEN
`def each(&) = @items.each(&)`	thin wrapper
`Forwardable`	forward many methods
`inherit from Array`	avoid (leaky)

⚠ COMMON MISTAKES

- Subclassing `Array` or `Hash`; many methods return plain arrays and break your invariants
- Not returning an Enumerator from a blockless `each`; `list.each.with_index` then raises `LocalJumpError`
- Defining `<=>` that returns non-integers or never `nil` for incomparable types

✓ TIPS & BEST PRACTICES

- `def each(&) = @songs.each(&)` is the shortest correct delegation (it returns an Enumerator too)
- Override `size` (and `include?` if you have a faster lookup) for big collections
- Implement `to_a` / `to_h` only if they differ from the Enumerable defaults

QUICK SUMMARY

- `each` + `include Enumerable` — collection superpowers
- `to_enum` — blockless `each`
- `<=>` + `Comparable` — ordering
- don't subclass Array — wrap and delegate

Open Classes & Refinements

OPEN CLASSES AND REFINEMENTS

Ruby classes are **open**: you can reopen any class, including core ones like `String`, and add or replace methods. This **monkey patching** is powerful but changes behavior globally. **Refinements** scope changes: `refine` defines the patch in a module, and it only applies in files (or modules) that call **`using`**.

NOTES

- Reopening a class adds to it; it doesn't replace it
- Monkey patches affect every gem in the process
- Refinements are lexically scoped (file / module after `using`)
- Refinements don't apply to `send` or `method` lookups in other scopes

MONKEY PATCHING VS REFINEMENTS

```ruby
class String                    # reopen a core class (global!)
  def shout = upcase + "!"
end
"hi".shout                      # "HI!"

module Shouting
  refine String do
    def whisper = downcase + "…"
  end
end

"HI".respond_to?(:whisper)      # false (not active yet)
using Shouting                  # active for the rest of this file
"HI".whisper                    # "hi…"

class Integer                   # safer: add, don't override
  def minutes = self * 60
end
5.minutes                       # 300

String.instance_method(:shout).source_location  # [file, line]
```

CODE EXPLAINED

- → `class String … end` — reopens it; existing methods stay
- → `refine String do` — define a scoped patch
- → `using Shouting` — activate it from here to end of file
- → `respond_to?` — false before `using`
- → `source_location` — find who defined a patched method

THE RUBY WAY

- Patching core classes in a library → refinements or helper modules
- Overriding existing core methods → add new, clearly named methods
- `class Foo` reopening by accident → check for name clashes first

IN RAILS

- **AS** · ActiveSupport core extensions (`blank?`, `5.minutes`, `squish`) are monkey patches on core classes
- **AS** · each extension can be required on its own, e.g. `core_ext/string/filters`

APPROACHES

APPROACH	SCOPE
`monkey patch`	whole process
`refinement + using`	file / module
`helper module function`	explicit calls
`subclass / wrapper`	your objects only

REFINEMENT RULES

RULE	DETAIL
`using at top level`	rest of the file
`using in module`	rest of that module body
`not via send`	dynamic calls skip refinements
`import_methods`	reuse module methods in a refine

⚠ COMMON MISTAKES

- Overriding a core method (like `String#to_s`) and breaking unrelated gems
- Expecting a refinement to be active in another file because you called `using` once
- Two libraries patching the same method name; the last one loaded wins silently

✓ TIPS & BEST PRACTICES

- Prefix patch methods with your project name when you must monkey patch
- Keep patches in one place (`lib/core_ext/`) so they're easy to find
- Prefer refinements in gems; leave global patches to applications

QUICK SUMMARY

- open classes — reopen anything
- monkey patch — global change
- `refine + using` — scoped change
- add, don't override

Introspection

INTROSPECTION

Ruby objects can describe themselves at runtime: list their **methods**, **instance variables**, **ancestors** and **constants**, check what they respond to, and find where a method is defined. Introspection powers debugging, REPL exploration, documentation tools and metaprogramming.

NOTES

- `methods(false)` / `instance_methods(false)` show only the class's own methods
- `public_methods`, `private_methods` filter by visibility
- `source_location` returns `nil` for methods written in C
- `Module#const_source_location` finds where a constant is defined

INSPECTING OBJECTS AND CLASSES

```ruby
class Dog
  NOISE = "woof"
  attr_reader :name
  def initialize(name) = @name = name
  def bark = NOISE
end
d = Dog.new("Rex")

d.class                              # Dog
d.instance_variables                 # [:@name]
d.instance_variable_get(:@name)      # "Rex"
Dog.instance_methods(false)          # [:name, :bark]
d.public_methods(false)              # methods defined on Dog
d.respond_to?(:bark)                 # true
Dog.ancestors.first(3)               # [Dog, Object, Kernel]
Dog.constants                        # [:NOISE]
Dog.const_get(:NOISE)                # "woof"

d.method(:bark).owner                # Dog
d.method(:bark).source_location      # ["dog.rb", 5]
"x".method(:upcase).source_location # nil (C method)
```

CODE EXPLAINED

- → `instance_variables` / `instance_variable_get` — read state by name
- → `instance_methods(false)` — only methods this class defines
- → `ancestors` — modules and superclasses in lookup order
- → `const_get` — constant by name (strings like `"A::B"` work too)
- → `method(:m).source_location` — file and line of the definition

THE RUBY WAY

- Searching files for a method → `method(:x).source_location`
- `d.methods.include?(:x)` → `d.respond_to?(:x)`
- Reading private state with `instance_variable_get` in app code → add a reader

OBJECT QUESTIONS

CALL	RETURNS
`obj.class`	class
`obj.methods`	all public + protected
`obj.singleton_methods`	per-object methods
`obj.instance_variables`	ivar names

CLASS QUESTIONS

CALL	RETURNS
`C.instance_methods(false)`	own methods
`C.ancestors`	lookup chain
`C.constants`	constant names
`C.method_defined?(:m)`	boolean

⚠ COMMON MISTAKES

- Reading `obj.methods` and drowning in `Object` methods; pass `false` or subtract `Object.instance_methods`
- Using `instance_variable_set` to bypass validations in production code
- Assuming `source_location` always works; C-implemented methods return `nil`

✓ TIPS & BEST PRACTICES

- In irb, `ls obj` and `show_source Dog#bark` are faster than calling these by hand
- `obj.method(:m).super_method` walks up overridden implementations
- `Module#name` and `#to_s` help identify anonymous classes (`nil` name means anonymous)

QUICK SUMMARY

- `methods` / `instance_methods(false)` — what it can do
- `instance_variables` — what it holds
- `ancestors` / `constants` — class structure
- `source_location` — where it's defined

Dynamic Dispatch & define_method

send · public_send · define_method · instance_variable_get / set · const_set · remove_method

DYNAMIC DISPATCH AND DEFINE_METHOD

send / public_send call methods by name, and **define_method** creates methods at runtime with a block body that is a closure. Combined with `instance_variable_get` / `set` and `const_get` / `set`, they let you generate repetitive methods from data instead of writing them by hand.

NOTES

- `define_method(name) { |args| … }` — block becomes the body
- Called inside a class body, `self` is the class
- `remove_method` deletes from this class; `undef_method` blocks lookup entirely
- Prefer `public_send` with user-derived names

GENERATING METHODS

```ruby
class Settings
  FIELDS = %i[theme lang timezone]

  FIELDS.each do |field|
    define_method(field) { @data[field] }               # getter
    define_method("#{field}=") { |v| @data[field] = v } # setter
    define_method("#{field}?") { !@data[field].nil? }   # predicate
  end

  def initialize = @data = {}
end

s = Settings.new
s.theme = "dark"
s.theme                          # "dark"
s.lang?                          # false
s.public_send(:theme)            # "dark"
Settings.instance_methods(false).size   # 9

s.instance_variable_set(:@data, { theme: "light" })
s.theme                          # "light"
```

CODE EXPLAINED

→ `FIELDS.each { define_method … }` — three methods per field

→ `"#{field}="` — method names can be strings

→ block body — closes over `field`

→ `public_send(:theme)` — call by name, respecting visibility

→ `const_set` / `const_get` — define and read constants by name

THE RUBY WAY

- Copy-pasted getters and setters → loop over a field list with `define_method`
- `eval("def #{name} …")` → `define_method(name) { }`
- `send(params[:action])` → `public_send` with an allow-list

IN RAILS

- **Rails** · ActiveRecord defines attribute methods (`name` , `name=` , `name?` , `name_changed?`) from the table schema
- **Rails** · `enum :status, %i[draft live]` generates `draft?` , `live!` and scopes the same way

DYNAMIC TOOLS

METHOD	DOES
`send / public_send`	call by name
`define_method`	create a method
`instance_variable_get / set`	read / write ivars
`const_get / const_set`	read / write constants

DEFINE_METHOD VS DEF

FEATURE	DEF	DEFINE_METHOD
`name from data`	no	yes
`closure over locals`	no	yes
`speed`	fastest	slightly slower

⚠ COMMON MISTAKES

- Generating methods nobody can find; add a comment listing the generated names
- Using `send` with user input; it can reach private methods like `system` or `exit`
- Defining methods inside instance methods with `define_method` ; that changes the class for every instance

✓ TIPS & BEST PRACTICES

- `define_method` returns the method name, so `private define_method(:x) { }` works
- `respond_to_missing?` + `method_missing` is the alternative when names aren't known up front (sheet 110)
- Generate methods once at class load, not per object

QUICK SUMMARY

- `define_method` — methods from data
- `public_send` — call by name safely
- `instance_variable_get/set` — dynamic state
- `const_get/set` — dynamic constants

method_missing & Delegation

METHOD_MISSING AND DELEGATION

When a method isn't found, Ruby calls **`method_missing(name, *args, &block)`**, which you can override to handle "ghost methods" dynamically. Always pair it with **`respond_to_missing?`** and call **`super`** for names you don't handle. For plain forwarding, **`Forwardable`** and **`SimpleDelegator`** are simpler and faster.

NOTES

- `super` in `method_missing` raises the normal `NoMethodError`
- Without `respond_to_missing?`, `respond_to?` and `method` lie
- Ghost methods are slower than real ones

GHOST METHODS AND DELEGATION

```
class Config
  def initialize(data) = @data = data
  def method_missing(name, *args)
    return @data[name] if @data.key?(name) && args.empty?
    super                                  # NoMethodError for others
  end
  def respond_to_missing?(name, priv = false) = @data.key?(name) || super
end
c = Config.new(port: 8080)
c.port                                     # 8080
c.respond_to?(:port)                       # true
c.host                                     # NoMethodError

require "forwardable"
class Stack
  extend Forwardable
  def_delegators :@items, :size, :last, :push, :pop
  def initialize = @items = []
end

require "delegate"
class Loud < SimpleDelegator
  def to_s = __getobj__.to_s.upcase        # override one, forward the rest
end
```

CODE EXPLAINED

- → `method_missing(name, *args)` — catch unknown calls
- → `super` — fall back to `NoMethodError`
- → `respond_to_missing?` — keep `respond_to?` honest
- → `def_delegators :@items, …` — forward named methods
- → `SimpleDelegator` — wrap an object and forward everything

THE RUBY WAY

- `method_missing` for a fixed list of names → `define_method` (sheet 109)
- Hand-written forwarding methods → `Forwardable`
- Decorators that copy every method → `SimpleDelegator`

IN RAILS

- **AS** · `delegate :name, :email, to: :user, prefix: true` → `user_name`, `user_email`
- **Rails** · old dynamic finders like `find_by_email` used `method_missing`; today use `find_by(email:)`

HOOKS

METHOD	CALLED WHEN
`method_missing`	method not found
`respond_to_missing?`	respond_to? / method

DELEGATION TOOLS

TOOL	USE
`Forwardable`	forward named methods
`SimpleDelegator`	wrap everything
`AS delegate`	Rails models / services

⚠ COMMON MISTAKES

- Forgetting `super` in `method_missing`; typos then silently return `nil`
- Skipping `respond_to_missing?`; `respond_to?`, `method(:x)` and duck-typing checks break
- Using `method_missing` in hot paths; every call goes through a failed lookup first

✓ TIPS & BEST PRACTICES

- Define the real method on first use (`define_singleton_method`) to speed up repeated ghost calls
- `SimpleDelegator` is ideal for presenters and decorators around models
- Keep the handled-name check identical in both hooks; extract it into one private method

QUICK SUMMARY

- `method_missing` — handle unknown methods
- `respond_to_missing?` + `super` — always pair them
- `Forwardable` — forward named methods
- `SimpleDelegator` — wrap an object

Singleton Classes

SINGLETON CLASSES

Every object has a hidden **singleton class** (eigenclass) that holds methods for **that object only**. `def obj.method`, `define_singleton_method` and `obj.extend(Module)` all add methods there. **Class methods are singleton methods of the class object**, which is why `class << self` works for defining them.

NOTES

- Method lookup checks the singleton class first, then the class
- `obj.singleton_class` returns it; `singleton_methods` lists its methods
- `extend` mixes a module into the singleton class

PER-OBJECT METHODS

```ruby
dog = Object.new
def dog.speak = "woof"              # singleton method
dog.speak                           # "woof"
Object.new.respond_to?(:speak)      # false (only this object)

cat = Object.new
cat.define_singleton_method(:speak) { "meow" }
cat.singleton_methods               # [:speak]

module Loud
  def shout = "#{speak.upcase}!"
end
dog.extend(Loud)                    # module on dog's singleton class
dog.shout                           # "WOOF!"
dog.singleton_class.ancestors.first(2)  # [#<Class:#<Object…>>, Loud]

class Report
  class << self                     # Report's singleton class
    def build = new
  end
end
Report.singleton_methods            # [:build]
5.singleton_class                   # TypeError: can't define singleton
```

CODE EXPLAINED

- → `def dog.speak` — method only on `dog`
- → `define_singleton_method` — same, with a block (closure)
- → `extend(Loud)` — add a module's methods to one object
- → `class << self` — opens the class's singleton class
- → `5.singleton_class` — immediates don't have one

THE RUBY WAY

- Subclassing just to change one object → `extend` or a singleton method
- `def self.a; def self.b` scattered → `class << self` block
- Patching a shared object (like a constant) → wrap it instead

ADDING SINGLETON METHODS

FORM	EXAMPLE
`def obj.m`	`def config.reload = …`
`define_singleton_method`	dynamic name / closure
`obj.extend(Mod)`	several methods at once
`class << obj`	open the singleton class

LOOKUP ORDER FOR OBJ.M

STEP	CHECKS
1	singleton class (+ extended modules)
2	prepended modules of the class
3	the class
4	included modules, superclasses
5	method_missing

⚠ COMMON MISTAKES

- Expecting `dup` to keep singleton methods; only `clone` copies them
- Adding singleton methods to objects that get serialized; `Marshal` can't dump them
- Forgetting that class methods aren't inherited by instances; they live on the class's singleton class

✓ TIPS & BEST PRACTICES

- `extend self` in a module puts its methods on the module's own singleton class
- Tests can stub one object with `define_singleton_method` without mocking libraries
- `singleton_class?` tells you whether a class object is a singleton class

QUICK SUMMARY

- singleton class — per-object methods
- `def obj.m` / `define_singleton_method` — add one
- `extend` — add a module to one object
- class methods — singleton methods of the class

eval Family & Hooks

EVAL FAMILY AND HOOKS

instance_eval runs a block with `self` set to an object, **class_eval** (alias `module_eval`) runs it inside a class so `def` adds instance methods, and **instance_exec** does the same while passing arguments. **Hook methods** like `inherited`, `included`, `extended` and `method_added` let classes and modules react when they're used.

NOTES

- Prefer blocks over string `eval`; strings can't be checked and are an injection risk
- `class_eval` with a block can reopen classes held in variables
- `included(base)` is the standard way to add class methods from a module

EVAL VARIANTS AND HOOKS

```
class Box; def initialize = @secret = 42; end
Box.new.instance_eval { @secret }        # 42 (self is the box)
Box.class_eval { def size = 3 }          # adds an instance method
Box.new.instance_exec(10) { |n| @secret + n }  # 52

module Trackable
  def self.included(base) = base.extend(ClassMethods)  # hook
  module ClassMethods
    def tracked? = true
  end
end
class Order; include Trackable; end
Order.tracked?                           # true

class Plugin
  def self.registry = @registry ||= []   # class-level ivar
  def self.inherited(sub) = (super; registry << sub)   # hook
end
class CsvPlugin < Plugin; end
Plugin.registry                          # [CsvPlugin]
```

CODE EXPLAINED

- → `instance_eval` — `self` is the receiver, ivars readable
- → `class_eval { def … }` — define methods on a class
- → `instance_exec(10)` — like `instance_eval` with arguments
- → `self.included(base)` — runs when the module is included
- → `self.inherited(sub)` — runs for each new subclass

THE RUBY WAY

- `eval("def #{n}; end")` → `define_method(n) { }`
- `instance_eval` to read private state in app code → add a public method
- Manual plugin lists → `inherited` hook (or `subclasses`)

IN RAILS

- **AS** · `ActiveSupport::Concern` wraps `included` with `included do … end` and `class_methods do … end`
- **Rails** · `ApplicationRecord` uses `inherited` to set up each model class

EVAL VARIANTS

METHOD	SELF INSIDE
`instance_eval { }`	the object
`instance_exec(args) { }`	the object, with args
`class_eval { }`	the class (defs → instance methods)
`eval("code")`	current binding (avoid)

HOOKS

HOOK	FIRES WHEN
`inherited(sub)`	class subclassed
`included(base)`	module included
`extended(obj)`	module extended
`method_added(name)`	method defined

⚠ COMMON MISTAKES

- Forgetting `super` in `inherited`; other libraries' hooks on the parent stop running
- Using string `eval` with interpolated input; it can execute arbitrary code
- Expecting `instance_eval { def x }` to add an instance method; it defines a singleton method

✓ TIPS & BEST PRACTICES

- `instance_eval` with a block is the core of most Ruby DSLs (sheet 113)
- Use `Module#prepended` / `included` to register behavior without monkey patching
- `__method__` and `caller_locations` help hooks report where they were triggered

QUICK SUMMARY

- `instance_eval` / `instance_exec` — run as the object
- `class_eval` — run inside the class
- `included` / `inherited` — react to use
- avoid string `eval`

Building DSLs

BUILDING DSLS

A **DSL** (domain-specific language) is Ruby code shaped to read like configuration. The usual ingredients are **blocks**, **`instance_eval`** (so bare method calls hit a builder object), **method calls without parentheses**, and **keyword arguments**. Rake tasks, RSpec, Gemfiles and Rails routes are all Ruby DSLs.

DSL INGREDIENTS

- A builder object that collects settings
- `instance_eval(&block)` or yield the builder (`|config|`)
- Methods named like keywords: `title "x"` , `route "/a"`
- `Class.new` / `Module.new` for anonymous classes

A SMALL CONFIG DSL

```ruby
class Server
  Route = Data.define(:verb, :path, :to)
  attr_reader :routes

  def self.build(&block) = new.tap { it.instance_eval(&block) }

  def initialize = @routes = []
  def port(n = nil) = n ? @port = n : @port
  def get(path, to:)  = @routes << Route.new(:get, path, to)
end

app = Server.build do
  port 8080
  get  "/users", to: "users#index"
end
app.port                                # 8080
app.routes.first.to                     # "users#index"

Config = Struct.new(:env, :debug)
def configure = (@config ||= Config.new).tap { yield it }
configure { |c| c.env = :prod; c.debug = false }   # yield style
```

CODE EXPLAINED

- → `instance_eval(&block)` — bare calls (`port` , `get`) go to the builder
- → `def port(n = nil)` — one method as setter and getter
- → `get "/users", to:` — keywords read like configuration
- → `tap { yield it }` — the explicit-receiver style (`c.env =`)

THE RUBY WAY

- Long hashes of options → a block DSL with named methods
- `instance_eval` where outer methods are needed → yield the builder instead
- DSLs for one-off scripts → plain method calls

IN RAILS

- **Rails** · `routes.rb` , `has_many :posts` , `validates :email, presence: true` are DSLs on class bodies
- **Rails** · `Rails.application.configure do … end` uses `instance_eval`

TWO DSL STYLES

STYLE	PROS / CONS
`instance_eval`	clean syntax; hides outer `self`
`yield builder`	explicit `c.x =`; outer methods still work

FAMOUS RUBY DSLS

DSL	EXAMPLE
Rake	`task :build do … end`
RSpec	`describe … it … expect`
Bundler	`gem "rails", "~> 8.1"`
Rails routes	`resources :users`

⚠ COMMON MISTAKES

- Using `instance_eval` and then calling your own helper methods inside the block; `self` changed, so they aren't found
- Building a DSL where plain method calls would do; it adds indirection for no gain
- Letting DSL methods silently ignore typos; raise on unknown settings

✓ TIPS & BEST PRACTICES

- Support both styles: `block.arity == 1 ? yield(self) : instance_eval(&block)`
- Freeze the built object after the block runs so configuration can't drift
- Name DSL methods after the domain (`route` , `task`), not after implementation details

QUICK SUMMARY

- blocks + `instance_eval` — DSL core
- builder object — collects settings
- yield style — explicit receiver
- Ruby DSLs — Rake, RSpec, Gemfile, routes

Sheet 114

Quick Reference — Syntax, Types & Control Flow

running · output · types & conversion · strings · operators · conditionals · case & pattern matching · loops · exceptions · methods & blocks

RUNNING & OUTPUT

SYNTAX	DOES	#
`ruby app.rb`	run a script	003
`irb`	interactive shell	003
`puts / print / p`	line / raw / inspect	005
`gets.chomp`	read a line	006
`ARGV`	command-line args	006

TYPES

SYNTAX	DOES	#
`42 / 3.14 / 1r`	Integer / Float / Rational	009
`:sym`	Symbol	013
`nil`	nothing; falsy	012
`false / nil`	the only falsy values	011
`obj&.name`	safe navigation	012
`Integer("42") / "42".to_i`	strict / lenient	014

STRINGS

SYNTAX	DOES	#
`"Hi #{name}"`	interpolation	017
`s.include? / s.start_with?`	search	018
`s.upcase / s.strip / s.sub`	transform	019
`s[0, 3] / s[-1]`	slice	020
`s.split(",")`	to array	020
`<<~TEXT`	squiggly heredoc	021

OPERATORS

OPERATOR	DOES	#
`a <=> b`	spaceship (-1, 0, 1)	023
`== / eql? / equal?`	value / strict / identity	023
`&& || !`	logic (prefer over `and`/`or`)	024
`x ||= 5`	assign if nil / false	026
`*rest / **opts`	splat / double splat	026
`1..5 / 1...5`	inclusive / exclusive range	057

CONDITIONALS

SYNTAX	DOES	#
`if / elsif / else / end`	branch	028
`unless cond`	if not	028
`x = 1 if ok`	modifier form	029
`cond ? a : b`	ternary	029
`case x when 1..5 then`	case with ranges	030

PATTERN MATCHING

SYNTAX	DOES	#
`case h in {name: String => n}`	hash pattern	031
`in [x, *rest]`	array pattern	031
`in Integer | Float`	alternatives	031
`in {age:} if age > 18`	guard	032
`h => {name:}`	rightward destructure	032

LOOPS & ITERATION

SYNTAX	DOES	#
`5.times { |i| }`	repeat	034
`xs.each { |x| }`	iterate	034
`each_with_index`	with index	034
`while / until / loop do`	loops	033
`next / break / redo`	loop control	035

EXCEPTIONS

SYNTAX	DOES	#
`begin … rescue E => e`	catch	036
`else / ensure`	no error / always	036
`raise ArgumentError, "msg"`	raise	037
`class MyError < StandardError`	custom	038
`retry`	re-run begin block	039

METHODS & BLOCKS

SYNTAX	DOES	#
`def greet(name, greeting: "Hi")`	keyword arg	041
`def m(*args, **kw, &blk)`	variadic + block	042
`yield / block_given?`	call block	044
`->(x) { x * 2 }`	lambda	046
`proc { |x| }`	proc	045
`def m = expr`	endless method	040

⚠ COMMON MISTAKES

- `0` and `""` are truthy — only `nil` and `false` are falsy
- `and` / `or` have very low precedence; use `&&` / `||` in expressions
- Rescuing `Exception` instead of `StandardError` (catches interrupts and exits)

✓ TIPS & BEST PRACTICES

- Use `p` for debugging; it shows `inspect` output
- Prefer guard clauses and modifier `if` for short checks
- Use keyword arguments for anything beyond two parameters

QUICK SUMMARY

- Everything is an object; methods return the last expression
- `#{}` interpolation, symbols for names, `nil` and `false` falsy
- `case` / `in` for pattern matching
- Blocks with `yield`; lambdas with `->`

Sheet 115

Quick Reference — Collections, OOP & Tooling

arrays · hashes · Enumerable · sorting · classes & modules · visibility · equality · files & JSON · gems & Bundler · testing · metaprogramming

ARRAYS

SYNTAX	DOES	#
`%w[a b c]`	word array	050
`xs << x / push / pop`	stack	051
`shift / unshift`	queue	051
`xs.compact / uniq / flatten`	clean up	052
`xs & ys / xs | ys / xs - ys`	set ops	052

HASHES

SYNTAX	DOES	#
`{ name: "A", age: 3 }`	symbol keys	054
`h.fetch(:k, default)`	lookup or fallback	055
`h.dig(:a, :b)`	nested lookup	055
`h.transform_values(&:to_s)`	map values	056
`h.select { |k, v| } / reject`	filter	056
`Hash.new(0)`	default value	054

ENUMERABLE

METHOD	DOES	#
`map / filter_map`	transform	101
`select / reject / find`	filter / first match	102
`reduce(:+) / sum`	aggregate	103
`group_by / tally`	group / count	103
`each_slice(2) / chunk_while`	batches	104
`xs.lazy.map … .first(5)`	lazy pipeline	105

SORTING & STRUCTS

SYNTAX	DOES	#
`sort_by { _1.age }`	sort by key	060
`max_by / min_by`	extremes	060
`sort.reverse / sort_by { -x }`	descending	060
`Struct.new(:x, :y)`	mutable record	059
`Data.define(:x, :y)`	immutable value	059

CLASSES

SYNTAX	DOES	#
`class Dog < Animal`	inheritance	065
`def initialize(name)`	constructor	061
`attr_accessor :name`	getter + setter	062
`def self.create`	class method	063
`private / protected`	visibility	064
`super / super()`	call parent (with / no args)	065
`include Comparable`	mixin	066

OBJECT PROTOCOLS

METHOD	ENABLES	#
`to_s / inspect`	printing	071
`<=> + Comparable`	comparisons	069
`== / eql? + hash`	equality, hash keys	070
`each + Enumerable`	collection methods	106
`dup / clone / freeze`	copying	072

FILES & DATA

SYNTAX	DOES	#
`File.read(p) / readlines`	read	073
`File.write(p, s)`	write	074
`File.open(p, "a") { |f| }`	append + auto-close	074
`JSON.parse(s, symbolize_names: true)`	JSON in	076
`obj.to_json / CSV.foreach`	JSON out / CSV	077

GEMS & TOOLING

COMMAND	DOES	#
`gem install x`	install gem	080
`bundle init / add / install`	Gemfile deps	081
`bundle exec cmd`	use Gemfile versions	081
`rspec / ruby -Itest`	RSpec / Minitest	087
`rubocop -a`	lint + autofix	090

METAPROGRAMMING

API	DOES	#
`define_method(:x) { }`	method at runtime	109
`send / public_send`	dynamic call	109
`method_missing + respond_to_missing?`	catch-all	110
`refine String do`	scoped monkey patch	107
`class << self`	singleton class	111

⚠ COMMON MISTAKES

- `Hash#[]` returning `nil` silently — use `fetch` when a key must exist
- Mutating a frozen string literal (`FrozenError`)
- `method_missing` without `respond_to_missing?`

✓ TIPS & BEST PRACTICES

- Use `bundle exec` so commands run with the Gemfile's gem versions
- Prefer `Data.define` for small immutable value objects
- Chain Enumerable methods instead of manual loops

QUICK SUMMARY

- Arrays and hashes + Enumerable do most work
- `attr_accessor` , `include` , `super` for OOP
- `File` / `JSON` / `CSV` for data
- Bundler + Rake + RSpec / Minitest for projects